STUDIO USER MANUAL · 사용자 매뉴얼

NeoFlow

NeoFlow Studio 사용자 매뉴얼

설치 · 화면 구성 · 표준 컴포넌트 · NeoFlow 전용 기능 · 실습

항목	내용
문서명	NeoFlow Studio 사용자 매뉴얼
문서 버전	v1.0
작성일	2026-10-07
제조사	주식회사 위블리스
대상 제품	NeoFlow v1.0 — NeoFlow Studio (v1.0.0)

© 2026 주식회사 위블리스

개정 이력

버전	일자	작성자	변경 내용	승인
v1.0	2026-10-07	주식회사 위블리스	최초 작성 — 시작하기 가이드·컴포넌트 사용 매뉴얼·사용자 매뉴얼을 한 권으로 통합	—

목차

개정 이력 2
목차 3
1. 개요 6
1.1 NeoFlow Studio 가 하는 일 6
1.2 기능 구성 6
1.3 읽는 순서 7
1.4 이 매뉴얼 보는 법 7
2. 설치와 제거 8
2.1 시스템 요구 사항 8
2.2 압축 해제 8
2.3 JDK 지정 (NEO_BD-win-x86_64.ini) 8
2.4 첫 실행과 버전 확인 9
2.5 제거 11
3. License 등록 12
3.1 라이선스 등록 방법 12
3.2 라이선스 저장 위치 12
3.3 자주 보는 메시지 13
4. 화면 구성 14
4.1 툴 바 영역 14
4.2 Repository 영역 15
4.3 Design 영역 (캔버스) 16
4.4 Outline / Code Viewer 영역 17
4.5 Configuration 영역 18
4.6 Palette 영역 19
4.7 뷰 추가 · 배치 되돌리기 20
5. Repository 사용법 22
5.1 Job Designs — Job 과 폴더 만들기 22
5.2 Job 열기와 버전 관리 23
5.3 내보내기 · 가져오기 · 빌드 24
5.4 Contexts — 환경별 값 관리 25
5.5 Metadata — DB 연결과 스키마 27
5.6 Code > Routines — 사용자 정의 함수 29
5.7 Recycle bin 과 그 밖의 노드 29
6. 컴포넌트 기본 사용법 30
6.1 컴포넌트 기본 조작 30
6.2 주요 컴포넌트 패밀리 30
6.3 처음 익혀 두면 좋은 컴포넌트 31
6.4 스키마(Schema) 정의 31
6.5 context 변수 32
6.6 NeoFlow 전용 컴포넌트 32
7. 연결(Connection) 33
7.1 Row — 데이터 흐름 33
7.2 Iterate — 반복 34
7.3 Trigger — 실행 순서 제어 35
7.4 서브잡(Subjob) 이해하기 35
8. 데이터베이스 컴포넌트 37

8.1 tDBInput — 조회 37
8.2 tDBOutput — 적재 37
8.3 연결 공유 — tDBConnection 패턴 38
8.4 tDBRow — 임의 SQL 38
9. 파일 컴포넌트 39
9.1 tFileInputDelimited — 구분자 파일 읽기 39
9.2 tFileOutputDelimited — 구분자 파일 쓰기 39
9.3 tFileList — 파일 목록 순회 39
10. 변환·처리 컴포넌트 41
10.1 tMap — 개요와 Map Editor 41
10.2 tMap — Lookup(조인) 41
10.3 tMap — 출력·필터·Reject 42
10.4 tFilterRow / tAggregateRow / tSortRow 42
10.5 tUniqRow / tReplicate / tUnite 42
11. 오케스트레이션 컴포넌트 43
11.1 tPrejob / tPostjob 43
11.2 tRunJob — 자식 Job 호출 43
11.3 tFlowToIterate / tIterateToFlow 43
11.4 tLoop 44
12. 로그·에러 처리 컴포넌트 45
12.1 tLogRow — 콘솔 출력 45
12.2 tDie / tWarn 45
12.3 tLogCatcher — 에러 수집 패턴 45
12.4 tStatCatcher — 실행 통계 46
13. 코드·기타 컴포넌트 47
13.1 tJava / tJavaRow / tJavaFlex 47
13.2 tFixedFlowInput / tRowGenerator 47
13.3 tContextLoad — context 동적 로딩 47
13.4 tSetGlobalVar / globalMap 48
14. NeoFlow Server 연결 정보 등록 49
14.1 API token 발급 (NeoFlow Server) 49
14.2 Preferences 페이지 열기 52
14.3 프로파일 관리 52
14.4 프로파일별 입력 항목 53
14.5 연결 테스트 53
14.6 저장 및 Job 으로의 주입 원리 54
15. NeoFlow 컴포넌트 56
15.1 tNeoFlowPre — BizDate 구간 가져오기 56
15.2 tNeoFlowPost — BizDate 워터마크 커밋 58
15.3 tNeoFlowThread — 서브잡 병렬 실행(시차 기동) 60
16. Job 파라미터 받기 (NeoFlowContext · NeoFlowJobApi) 62
16.1 파라미터가 Job 에 도달하는 경로 62
16.2 NeoFlowContext routine (기본 포함) 62
16.3 Job 에서 사용 62
16.4 동작 규칙과 제약 63
16.5 NeoFlowJobApi 로 서버 값 조회 64
17. Sync to NeoFlow Server 66
17.1 실행 66
17.2 마법사 단계 66
17.3 내부 동작 요약 69

17.4 Sync 전후의 Job 처리 70
18. Build Job (NeoFlow Deploy) 71
18.1 실행 71
18.2 주요 항목 71
18.3 배포 실행과 결과 72
19. 따라 하기 73
19.1 실습 준비 73
19.2 실습 1 — 첫 Job 만들기 73
19.3 실습 2 — DB 연결과 테이블 생성 75
19.4 실습 3 — 테스트 데이터 생성·적재 77
19.5 실습 4 — 조회 → 정렬 → 파일 출력 79
19.6 실습 5 — 두 소스 조인 81
19.7 실습 6 — 필터링과 조건 분기 82
19.8 실습 7 — 그룹핑·집계 84
19.9 실습 8 — 자식 Job 병렬 호출 86
19.10 실습 9 — NeoFlow Server 로 동기화 87
20. 부록 91
20.1 빠른 점검 체크리스트 91
20.2 자주 쓰는 단축키 91
20.3 자주 쓰는 globalMap 변수 92
20.4 컴포넌트 빠른 참조 92
20.5 자주 묻는 질문 93
20.6 용어 정리 94
20.7 참고 문서 95

1. 개요

NeoFlow Studio 는 Eclipse 기반 오픈소스 ETL 도구인 Talend Open Studio 8.0.1 을 기반으로, NeoFlow 전용 기능을 추가한 ETL 개발 도구입니다. 컴포넌트를 캔버스에 끌어다 놓고 선으로 연결하면 Studio 가 그에 해당하는 Java 코드를 자동으로 생성해 실행하고, 완성한 Job 은 NeoFlow Server 로 동기화해 운영합니다.

이 매뉴얼은 NeoFlow Studio 의 기능 전체를 한 권에 담았습니다. 설치와 화면 구성, Repository·컴포넌트·연결 같은 기본 기능, 자주 쓰는 표준 컴포넌트의 사용법, 그리고 License·NeoFlow Server 연동·NeoFlow 컴포넌트·Sync·Build Job 같은 NeoFlow 전용 기능을 차례로 다루고, 끝에 실습과 부록을 둡니다. 대상 제품은 NeoFlow v1.0 이며, NeoFlow Studio 버전은 v1.0.0 입니다.

1.1 NeoFlow Studio 가 하는 일

기업 데이터는 여러 시스템에 흩어져 있습니다. 운영 DB, 파일 서버, 외부 연계 시스템의 데이터를 모아 정제하고 데이터 웨어하우스나 다른 업무 시스템으로 옮기는 작업이 ETL 입니다. 이 작업을 프로그램으로 직접 짜면 소스마다 접속 코드·변환 로직·오류 처리를 매번 새로 작성해야 합니다.

NeoFlow Studio 는 이 반복 작업을 **컴포넌트**로 표준화합니다. DB 에서 읽는 일은 `tDBInput`, 매핑·변환은 `tMap`, 파일로 쓰는 일은 `tFileOutputDelimited` 처럼 각 단계가 하나의 아이콘이 되고, 개발자는 이들을 연결해 데이터 흐름을 그립니다.

```
    EXTRACT                 TRANSFORM           LOAD
    -------                 ---------           ----

    tDBInput   --(Main)---> tMap   --(Main)---> tDBOutput
    tFileInput --(Lookup)-→  |
                              +--(Reject)------>  tLogRow
```

작성한 Job 은 Studio 에서 바로 실행해 결과를 확인할 수 있고, 완성되면 **NeoFlow Server** 로 동기화해 스케줄 운영·모니터링 체계에 올립니다. Studio 가 개발 도구, Server 가 운영 관제 플랫폼입니다.

1.2 기능 구성

NeoFlow Studio 의 기능은 Talend Open Studio 8.0.1 이 제공하는 기본 기능과, 주식회사 위블리스가 자체 개발한 NeoFlow 전용 기능으로 나뉩니다. 각 기능을 다루는 장은 다음과 같습니다.

구분	기능 영역	주요 내용	장
공통	설치와 제거	시스템 요구 사항, 압축 해제, JDK 지정, 첫 실행, 제거	2 장
NeoFlow 전용	License 등록	라이선스 키 검증·등록, Sync·Build Job 사용 제한	3 장
기본 기능	화면 구성	툴 바, Repository, Design, Outline, Configuration, Palette 영역	4 장
기본 기능	Repository	Job·폴더, 버전, 내보내기·가져오기·빌드, Contexts, Metadata, Routines	5 장
기본 기능	컴포넌트 기본 사용법	배치·속성 탭·패밀리·스키마·context 변수	6 장
기본 기능	연결(Connection)	Row·Iterate·Trigger 연결, 서브잡	7 장
기본 기능	표준 컴포넌트	데이터베이스·파일·변환·오케스트레이션·로그·코드 컴포넌트	8~13 장
NeoFlow 전용	NeoFlow Server 연결	API token 발급, 서버 프로파일, 연결 테스트, 연결 정보	14 장

구분	기능 영역	주요 내용	장
		주입	
NeoFlow 전용	NeoFlow 컴포넌트	tNeoFlowPre · tNeoFlowPost · tNeoFlowThread	15 장
NeoFlow 전용	Job 파라미터	NeoFlowContext routine · NeoFlowJobApi	16 장
NeoFlow 전용	Sync to NeoFlow Server	서버 등록·빌드·업로드 마법사, Sync 전후의 Job 처리	17 장
기본 + 전용	Build Job	표준 빌드와 NeoFlow Deploy(서버 배포)	18 장
실습	따라 하기	첫 Job 만들기부터 서버 동기화까지 실습 9 개	19 장
부록	부록	점검 체크리스트, 단축키, globalMap 변수, FAQ, 용어, 참고 문서	20 장

참고 기본 기능의 조작 방법은 Talend Open Studio 8.0.1 과 같으므로, 개별 컴포넌트의 세부 동작은 Talend Open Studio 공식 문서도 함께 참고할 수 있습니다(20.7).

1.3 읽는 순서

목적에 따라 다음 순서로 읽으면 됩니다.

독자	읽을 장
NeoFlow Studio 를 처음 쓰는 사용자	2~7 장으로 설치·화면·기본 조작을 익힌 뒤 19 장 실습을 따라 합니다.
특정 컴포넌트 사용법이 필요한 개발자	8~13 장 표준 컴포넌트, 20.3 globalMap 변수, 20.4 컴포넌트 빠른 참조
Job 을 NeoFlow Server 운영에 올리는 담당자	3 장 License, 14~18 장 NeoFlow 전용 기능, 20.1 점검 체크리스트

TIP 19 장 「따라 하기」는 앞 장의 내용을 실제로 써 보는 실습입니다. 2~7 장을 훑어본 뒤 19 장을 손으로 따라 해 보면 화면과 용어가 훨씬 빨리 익숙해집니다.

참고 NeoFlow Server 웹 콘솔(Job 등록·스케줄·실행 이력·알림)은 «NeoFlow Server 사용자 매뉴얼»에서 다룹니다.

1.4 이 매뉴얼 보는 법

메뉴·버튼·컴포넌트명·코드는 `Job Designs`, `Basic settings`, `tMap`, `context.NEOFLOW_BASE_URL` 처럼 고정폭 글꼴로 표기합니다. 메뉴를 따라 들어가는 경로는 [Window] → [Preferences] 형태로, Job 흐름은 `--(Main)-->` 같은 ASCII 다이어그램으로 나타냅니다. 중요한 내용은 아래 세 가지 안내 상자로 강조합니다.

참고 참고 — 알아두면 도움이 되는 부가 정보입니다.

TIP TIP — 더 빠르고 편하게 작업하는 방법입니다.

주의 주의 — 데이터 손실·오류를 유발할 수 있어 각별히 주의해야 하는 항목입니다.

2. 설치와 제거

NeoFlow Studio 는 설치 프로그램(installer) 없이 배포 압축 파일을 풀어 사용합니다. 이 장은 PC 요구 사항, 압축 해제, JDK 지정, 첫 실행과 버전 확인, 제거 방법을 다룹니다. 화면 구성과 기본 조작은 4 장부터 다룹니다.

2.1 시스템 요구 사항

항목	요구 사항	비고
OS	Windows 10 / 11 (64bit)	32bit Windows 는 지원하지 않습니다.
Java	JDK 11 (64bit)	Studio 는 JDK 11 로 실행됩니다(ini 의 -Dosgi.requiredJavaVersion=11).
메모리	8GB 이상	Studio 최대 힙이 4GB(-Xmx4096m)로 설정되어 있습니다.
디스크	5GB 이상 여유 공간	workspace·빌드 산출물이 쌓입니다.
압축 해제 경로	짧은 영문 경로	예: C:\NEO_BD-V1.0.0
네트워크	NeoFlow Server 접속	Sync·Build Job(Deploy) 때 서버 Base URL 로 접속합니다.

2.2 압축 해제

1. 배포받은 NeoFlow Studio 압축 파일을 C 드라이브 바로 아래 같은 짧은 영문 경로에 풉니다(예: `C:\NEO_BD-V1.0.0`).
2. 압축을 푼 폴더에 아래 항목이 있는지 확인합니다.

항목	설명
`NEO_BD-win-x86_64.exe`	Studio 실행 파일 (Windows 64bit)
`NEO_BD-win-x86_64.ini`	JDK 경로·메모리 등 실행 옵션 (2.3)
`workspace`	프로젝트·Job 이 저장되는 작업 폴더. 처음 실행할 때 만들어집니다 (백업 대상)
`license.bin`	라이선스 등록 후 생성되는 파일 (3 장)
`plugins, features, configuration`	Studio 본체 — 수정하지 마세요
`custom-components`	컴포넌트 업데이트용 폴더 — 수정하지 마세요

> **주의** 압축 해제 경로에 한글·공백·특수문자가 들어가거나 경로가 길면 실행이나 Job 빌드가 실패할 수 있습니다. 바탕 화면·사용자 폴더 대신 드라이브 루트 아래 짧은 영문 경로를 쓰세요(Windows 경로 길이 제한 260 자).

2.3 JDK 지정 (NEO_BD-win-x86_64.ini)

Studio 가 쓸 JDK 는 실행 파일과 같은 폴더의 `NEO_BD-win-x86_64.ini` 첫 두 줄에 지정합니다. 첫 줄은 `-vm`, 둘째 줄은 `javaw.exe` 의 전체 경로입니다. 배포 압축 파일의 ini 에는 이 두 줄이 없을 수 있으므로, 없으면 파일 맨 위(`-vmargs` 보다 위)에 추가합니다. 지정한 예는 다음과 같습니다.

```
-vm
```

```
C:\Program Files\Java\jdk-11\bin\javaw.exe
-vmargs
-Xms1536m
-Xmx4096m
```

둘째 줄에는 실제로 설치한 JDK 11 의 경로를 씁니다.

1. JDK 11(64bit)을 설치하고, 설치 폴더 안의 `bin\javaw.exe` 전체 경로를 확인합니다.
2. Studio 를 종료한 상태에서 `NEO_BD-win-x86_64.ini` 를 메모장으로 엽니다.
3. 맨 위에 `-vm` 줄과 확인한 `javaw.exe` 전체 경로 줄을 넣습니다(이미 있으면 둘째 줄의 경로만 고칩니다). 경로에 공백이 있어도 따옴표로 감싸지 않습니다.
4. `-vm` 과 경로는 각각 한 줄씩 두고, 두 줄 모두 `-vmargs` 보다 위에 있어야 합니다. 나머지 줄은 수정하지 않습니다.
5. 저장한 뒤 Studio 를 실행합니다.

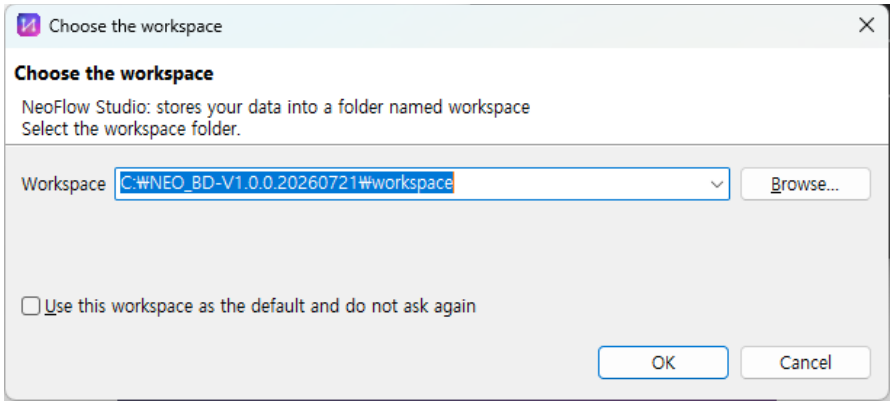
> **주의** `-vm` 경로가 틀리면 Studio 를 실행할 때 Java 가상 머신을 찾을 수 없다는 오류 창이 뜨고 Studio 가 시작되지 않습니다. 이 경우 ini 둘째 줄의 경로와 파일 이름(`javaw.exe`)을 다시 확인하세요.

> **참고** `-vmargs` 아래의 `-Xms1536m`·`-Xmx4096m` 이 Studio 의 시작·최대 메모리입니다. 요구 사항(8GB 이상)을 만족하는 PC 에서는 그대로 둡니다.

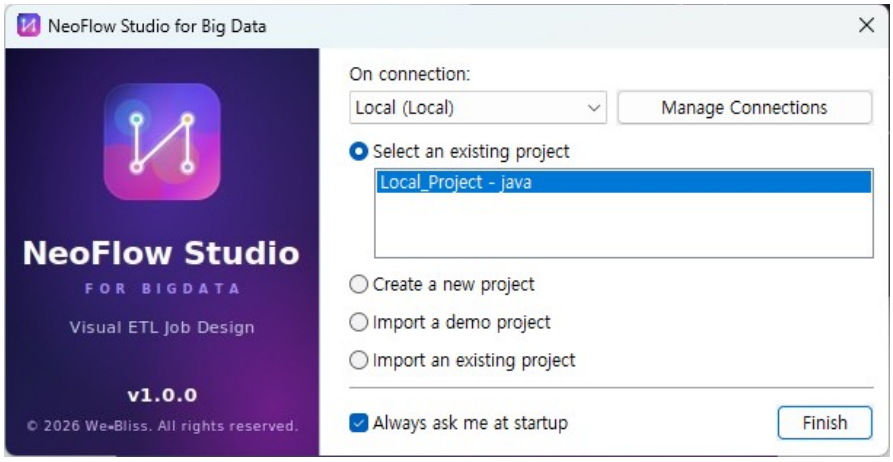
2.4 첫 실행과 버전 확인

워크스페이스는 프로젝트가 저장되는 폴더이고, **프로젝트**는 Job·메타데이터·context 를 담는 단위입니다. 하나의 워크스페이스 안에 여러 프로젝트를 둘 수 있습니다.

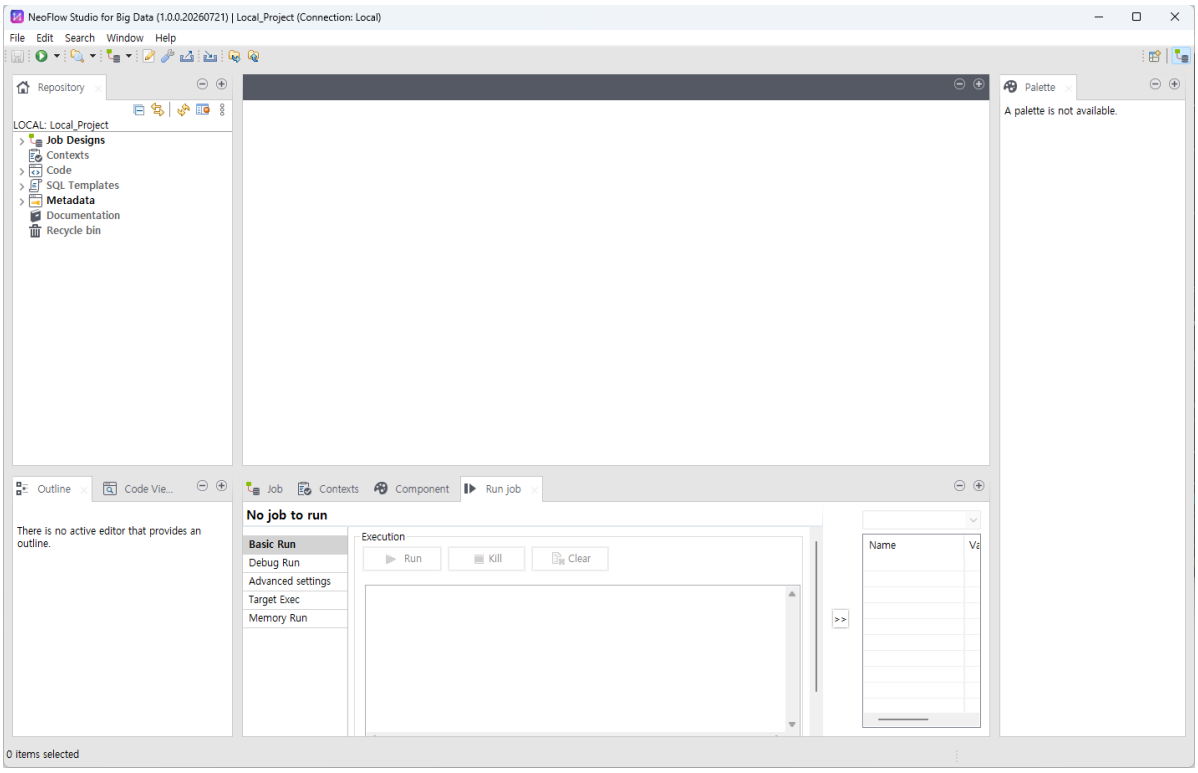
1. 압축을 푼 폴더에서 `NEO_BD-win-x86_64.exe` 를 더블 클릭합니다. 시작 화면이 뜬 뒤 워크스페이스 선택 창이 나타납니다.
2. 워크스페이스 선택 창이 나타나면 경로를 확인합니다. 특별한 이유가 없으면 기본값(설치 폴더 아래 `workspace`)을 그대로 사용하세요. `Use this workspace as the default and do not ask again` 을 체크하면 다음 실행부터 이 창을 건너뜁니다. 처음 실행할 때는 이 창 없이 설치 폴더 아래 `workspace` 가 바로 쓰일 수 있습니다.
3. [OK] 를 누르면 프로젝트 선택 창이 열립니다. `On connection` 은 로컬 작업이면 `Local (Local)` 그대로 둡니다.
4. 기존 프로젝트가 있으면 `Select an existing project` 목록에서 고르고, 없으면 `Create a new project` 를 선택해 이름을 입력합니다(예: `Local_Project`). 이름에는 영문·숫자·언더스코어를 사용합니다. 프로젝트가 하나도 없으면 `Create a new project`·`Import a demo project`·`Import an existing project` 중에서 고르는 창이 뜹니다.
5. [Finish] 를 누르면 Studio 메인 화면이 열립니다.
6. 창 제목 표시줄에 `NeoFlow Studio for Big Data` 와 버전(1.0.0 으로 시작)이 표시되는지 확인합니다. [Help] → [About NeoFlow Studio] 창에서도 제품 정보를 확인할 수 있습니다.
7. 이어서 3 장의 License 등록을 진행합니다.



▲ 워크스페이스 선택 창



▲ 프로젝트 선택 창 — Select an existing project / Create a new project



▲ Studio 메인 화면 — 창 제목에 NeoFlow Studio for Big Data 와 버전 v1.0.0 표시

> **참고** 워크스페이스 폴더는 Job 소스와 생성 코드가 모두 들어가는 실제 작업 산출물입니다. **백업 대상은 이 폴더**이며, 설치 폴더 전체를 백업할 필요는 없습니다.

> **TIP** 워크스페이스를 바꾸려면 `[File] → [Switch Workspace]`, 프로젝트를 바꾸려면 `[File] → [Switch Project]` 를 사용하세요. 화면 배치가 흐트러졌다면 `[Window] → [Perspective] → [Reset Perspective...]` 로 기본 배치를 복구합니다.

2.5 제거

설치 프로그램 없이 압축을 풀어 쓰므로 제거도 폴더를 지우는 방식입니다.

1. 보관할 Job 이 있으면 먼저 백업합니다. Studio 를 종료하고 `workspace` 폴더를 다른 위치로 복사하거나, Repository 에서 Job 을 [Export items] 로 내보냅니다.
2. Studio 가 종료되었는지 확인한 뒤 압축을 풀었던 설치 폴더 전체를 삭제합니다. `license.bin` 과 `workspace` 도 이 폴더 안에 있으므로 함께 지워집니다.
3. Studio 용으로만 설치한 JDK 가 더 필요 없으면 Windows [설정] → [앱] 에서 제거합니다.

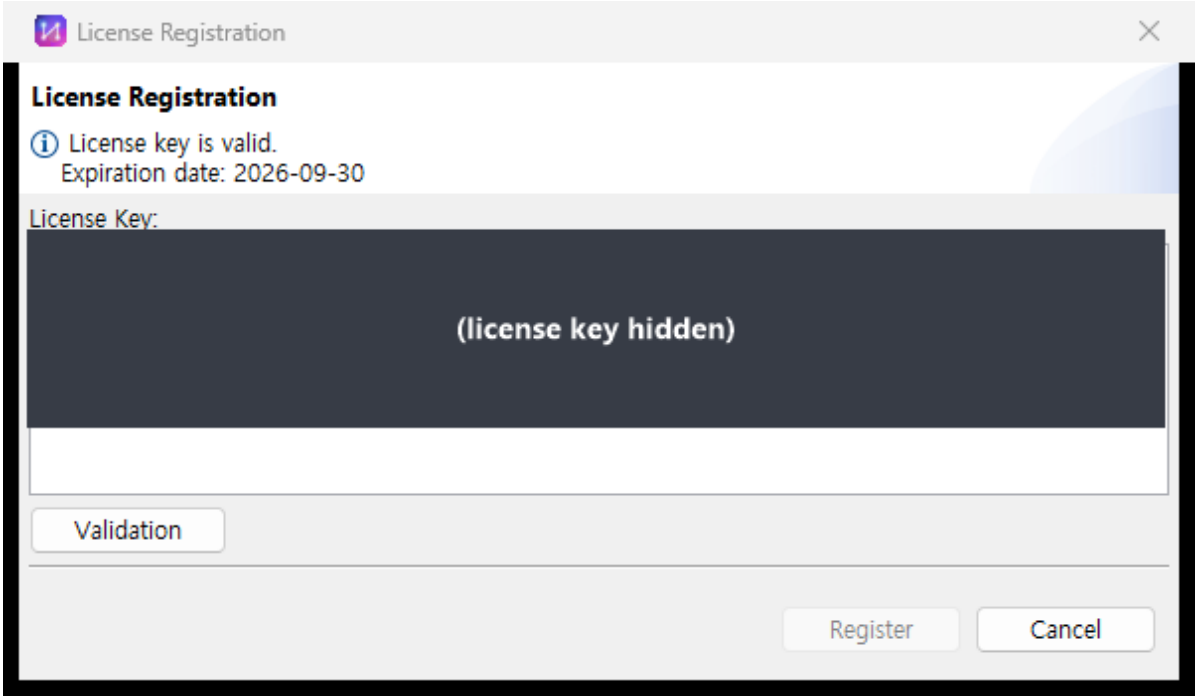
> **참고** 다른 PC 로 옮길 때는 백업한 `workspace` 를 새 설치 폴더에 두고 첫 실행에서 그 워크스페이스를 선택합니다. 특정 PC 의 MAC 주소로 발급된 라이선스는 새 PC 에서 유효하지 않으므로 다시 발급받아야 합니다 (3.2).

3. License 등록

NeoFlow Studio 의 핵심 기능(**Sync to NeoFlow Server**, **Build Job**)은 유효한 라이선스가 있어야 동작합니다. 라이선스가 없거나 만료되면 해당 메뉴 실행 시 에러 다이얼로그가 뜨고 진행되지 않습니다.

3.1 라이선스 등록 방법

1. 상단 메뉴 [Help] → [License Registration] 을 선택합니다.
2. License Registration 다이얼로그가 열립니다. 상단에 현재 라이선스 상태가 표시됩니다.
3. 발급받은 라이선스 키 전문을 License Key 입력란에 그대로 붙여넣습니다. 키 일부만 복사되면 검증에 실패하므로 반드시 전체를 복사하세요.
4. Validation 버튼을 눌러 키를 검증합니다. 정상이면 `License key is valid.` 와 만료일(Expiration date)이 표시되고 Register 버튼이 활성화됩니다.
5. Register 버튼을 눌러 라이선스를 저장합니다.



▲ *License Registration — 키 검증 성공 시 만료일 표시 + Register 활성화*

3.2 라이선스 저장 위치

등록된 라이선스는 Studio 설치 폴더에 `license.bin` 파일(경로: `<NeoFlow Studio 설치 폴더>/license.bin`)로 암호화되어 저장되며, 다음 정보를 포함합니다.

항목	설명	예시
`productName`	제품명 (Studio 제품명과 일치해야 함)	`NeoFlow Studio`
`majorVersion`	제품 버전 (Studio 버전과 일치해야 함)	`1.0.0`
`macAddress`	사용 PC 의 MAC 주소 (전체 사용 시 ALL)	`AA-BB-CC-DD-EE-FF / ALL`
`expirationDate`	만료일 (yyyy-MM-dd)	`2027-12-31`

> **참고** `macAddress` 가 특정 값으로 발급된 라이선스는 해당 PC 에서만 유효합니다. PC 를 교체하면 재발급이 필요합니다(`ALL` 로 발급된 경우 제외).

3.3 자주 보는 메시지

License Registration 다이얼로그 상단에는 현재 상태와 [Validation] 결과가 다음 문구로 표시됩니다. <만료일> 자리에는 라이선스의 만료일(yyyy-MM-dd)이 들어갑니다.

상황	메시지 (화면 표시 그대로)
라이선스 미등록	`Please register license key.`
키 검증 실패(키 일부만 복사 등)	`License key validation failed. Please check that copied and pasted the whole key. If the problem persists, please contact We-Bliss Inc.`
제품명 불일치	`License key validation failed(Product). Please check that copied and pasted the whole key. If the problem persists, please contact We-Bliss Inc.`
버전 불일치	`License key validation failed(Version). Please check that copied and pasted the whole key. If the problem persists, please contact We-Bliss Inc.`
MAC 주소 확인 불가	`Check the network for MAC address information.`
MAC 주소 불일치	`The current MAC address and the MAC address registered in the license are different.`
만료됨	`License date(<만료일>) expired. Please register new license key`
키를 고친 뒤 검증 전	`License key modified. Please register after validation.`
정상	`License key is valid. Expiration date: <만료일>`

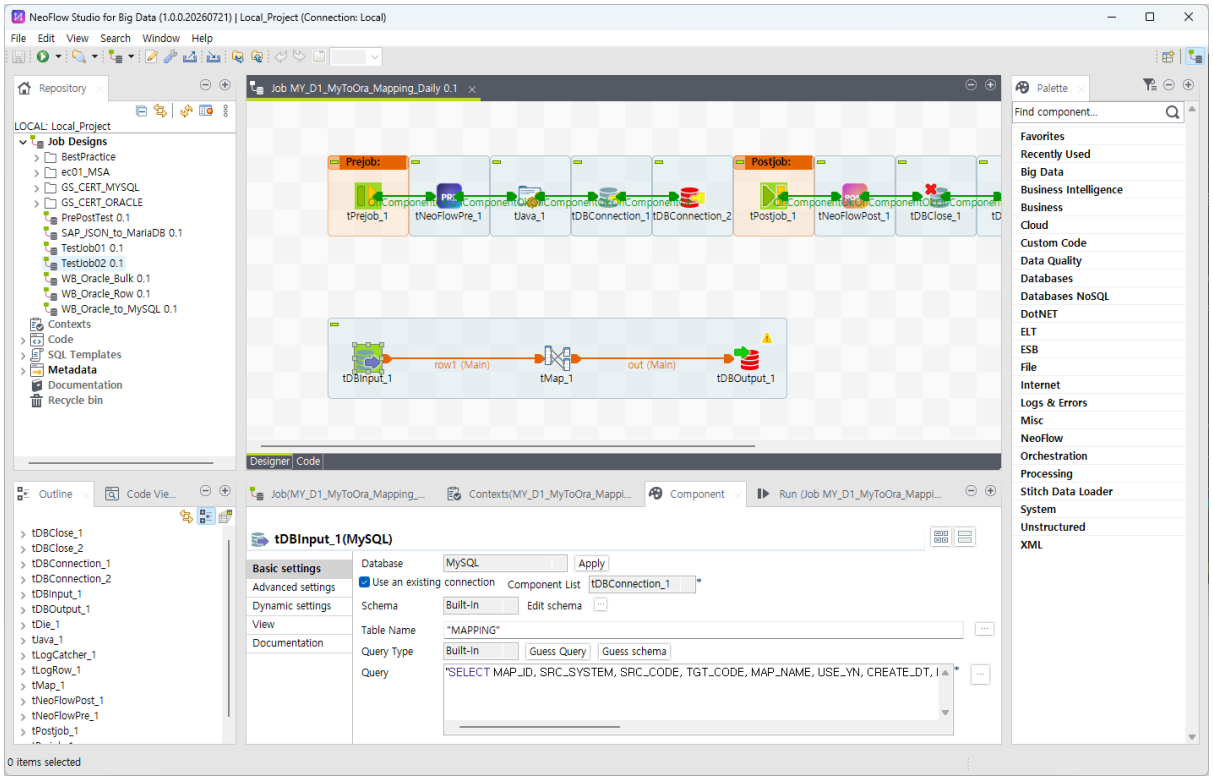
라이선스가 없거나 유효하지 않은 상태에서 [Sync to NeoFlow Server...] 나 [Build Job] 을 실행하면 제목이 `Error` 인 창에 다음 문구가 뜨고 메뉴가 진행되지 않습니다.

상황	메시지 (화면 표시 그대로)
라이선스 미등록	`Please register license key.`
키 손상·검증 실패	`License key validation failed. Please check that copied and pasted the whole key. If the problem persists, please contact We-Bliss Inc.`
만료됨	`License date expired. Please register new license key. If the problem persists, please contact We-Bliss Inc.`

> **참고** 메시지 속 `We-Bliss Inc.` 는 제조사 주식회사 위블리스의 영문 표기입니다. 문제가 지속되면 라이선스 발급처(주식회사 위블리스)에 문의하세요.

4. 화면 구성

NeoFlow Studio 의 메인 화면은 여섯 개 영역으로 나뉩니다. 이 장에서 각 영역의 역할을 먼저 익혀 두면 이후 장의 설명을 훨씬 쉽게 따라올 수 있습니다.



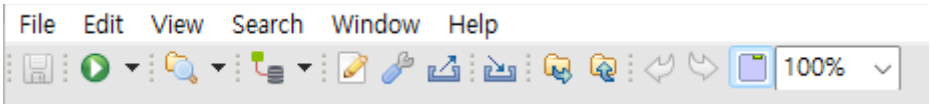
◢ *NeoFlow Studio 메인 화면 — 6 개 영역*

영역	화면 위치	역할
툴 바	최상단	저장·실행·Import/Export·프로젝트 설정 등 전역 명령
Repository	왼쪽 위	Job·Context·Metadata·Code 를 트리로 관리
Outline / Code Viewer	왼쪽 아래	열린 Job 의 컴포넌트 목록과 생성된 Java 코드
Design (Designer)	가운데	컴포넌트를 배치·연결해 Job 을 설계하는 캔버스
Configuration	가운데 아래	Job·Contexts·Component·Run 등 설정 탭 모음
Palette	오른쪽	사용할 수 있는 컴포넌트 목록

> **참고** 영역(뷰)은 탭을 끌어 위치를 바꾸거나 닫을 수 있습니다. 이 문서의 설명은 모두 기본 배치(Integration 퍼스펙티브) 기준입니다.

4.1 툴 바 영역

화면 최상단의 아이콘 모음으로, 프로젝트 전체에 적용되는 명령이 모여 있습니다. 같은 기능 대부분은 상단 메뉴(File·Edit·Window 등)에서도 실행할 수 있습니다.



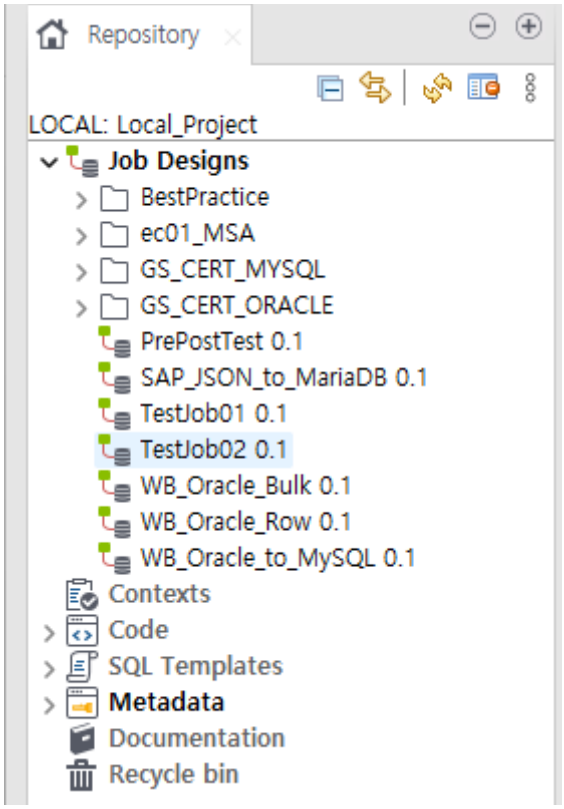
▲ 툴 바 영역

명령	기능
Save	열려 있는 Job 저장 (`Ctrl+S`)
Export items	선택한 Job·폴더를 아카이브(zip) 또는 디렉터리로 내보내기
Import items	아카이브·디렉터리에서 항목 가져오기
Find a specific job	프로젝트 내 Job 이름 검색
Run Job	Design 영역에 열린 Job 실행 (F6)
Create	Job·폴더·Context·Connection 등 새 항목 생성 모음
Project Settings	프로젝트 전체 기본 설정(통계·로그·라벨 규칙 등) 변경
Detect and update all jobs	전체 Job 에서 갱신이 필요한 항목 검색
Export projects	프로젝트 전체를 내보내기

> TIP `Project Settings` 를 Import/Export 하면 여러 프로젝트에 동일한 규칙(로그·통계·라벨)을 손쉽게 적용할 수 있습니다.

4.2 Repository 영역

화면 왼쪽 위의 트리 뷰로, 프로젝트에 속한 모든 자원이 여기에 모입니다. Job 을 만들고 여는 일도, DB 연결 정보를 등록하는 일도 모두 이 영역에서 시작합니다.



▲ *Repository 영역 — 트리 구성*

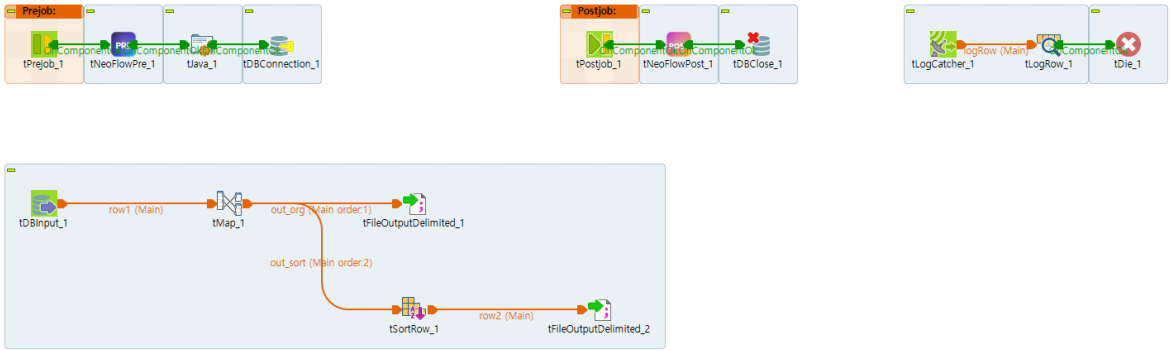
노드	역할
Job Designs	ETL Job 과 이를 분류하는 폴더. 가장 자주 사용하는 노드
Contexts	환경별로 달라지는 값(DB 접속 정보·경로·날짜)을 묶은 Context Group
Code > Routines	Job 에서 호출할 사용자 정의 Java 함수(루틴) 라이브러리
SQL Templates	표준 SQL 템플릿. 사용자 템플릿 추가 가능
Metadata	DB 연결·파일 스키마 등 재사용 가능한 메타데이터
Documentation	프로젝트에 첨부할 문서 파일 보관
Recycle bin	삭제한 항목 보관. 비우기 전까지 복구 가능

각 노드에서 마우스 우클릭하면 해당 노드에 맞는 메뉴가 나타납니다. 세부 사용법은 5 장에서 다룹니다.

> `TIP` Repository 상단의 필터 입력란에 이름 일부를 입력하면 트리가 즉시 좁혀집니다. Job 이 수십 개를 넘어가면 폴더 분류와 함께 적극 활용하세요.

4.3 Design 영역 (캔버스)

Job 을 실제로 그리는 작업 공간입니다. 팔레트에서 컴포넌트를 끌어다 놓고 선으로 연결하면 데이터 흐름이 완성됩니다. 아래는 실제로 작성된 Job 의 캔버스입니다.



▲ Design 영역 — 정렬 Job(MY_B1_Data_sorting) 캔버스

캔버스 하단에는 두 개의 탭이 있습니다.

탭	설명
Designer	컴포넌트를 배치·연결하는 그래픽 편집 화면
Code	Studio 가 생성한 Java 소스. 읽기 전용이며 디버깅 시 참고

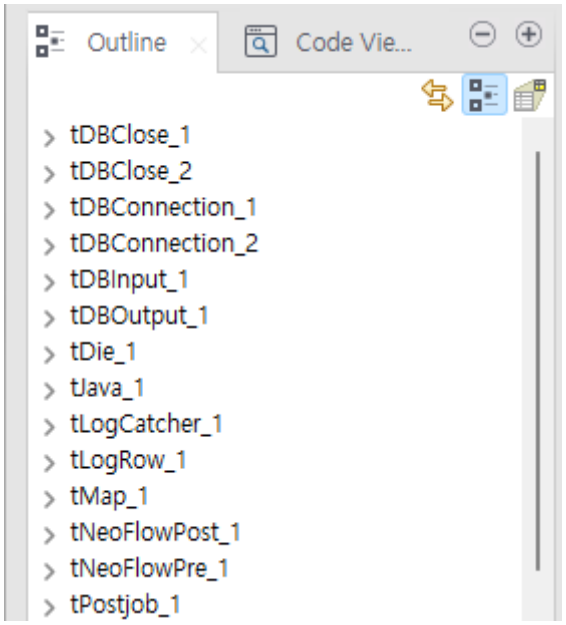
캔버스에서 회색 배경으로 묶인 덩어리 하나를 **서브잡(subjob)** 이라고 부릅니다. 위 그림에는 `tPrejob` 서브잡, `tPostjob` 서브잡, 오류 처리용 `tLogCatcher` 서브잡, 그리고 실제 데이터를 처리하는 메인 서브잡이 함께 놓여 있습니다.

> TIP 캔버스 조작 단축키 — 확대/축소는 `Ctrl`+마우스 휠, 전체 선택은 `Ctrl+A`, 컴포넌트 정렬은 우클릭 후 `[Arrange all elements]` 입니다.

4.4 Outline / Code Viewer 영역

화면 왼쪽 아래에 있으며 두 개의 탭으로 구성됩니다.

- **Outline** — 열린 Job 이 사용하는 컴포넌트 목록을 트리로 보여 줍니다. 컴포넌트가 많아 캔버스에서 찾기 어려울 때 여기서 선택하면 캔버스에서 해당 컴포넌트가 강조됩니다.
- **Code Viewer** — 캔버스에서 선택한 컴포넌트가 생성한 Java 코드 조각을 보여 줍니다. 설정값이 코드에 어떻게 반영되는지 확인할 때 유용합니다.



◢ *Outline / Code Viewer 영역*

4.5 Configuration 영역

캔버스 아래쪽의 탭 모음으로, 선택한 대상에 대한 상세 설정이 여기에 표시됩니다. 실제 작업 시간의 상당 부분을 이 영역에서 보내게 됩니다.



◢ *Configuration 영역 — Job · Contexts · Component · Run 탭*

탭	설명
Job	열린 Job 의 이름·작성자·버전 등 기본 정보와 통계·로그 설정
Contexts	이 Job 에 적용할 context 변수 관리. Repository 의 Context Group 을 가져오거나 개별 변수 생성
Component	캔버스에서 선택한 컴포넌트의 설정(Basic settings / Advanced settings 등)
Run	Job 실행과 실행 로그 확인
Job Hierarchy	선택한 Job 이 호출하는 하위 Job 구조 확인
Modules	Job 실행에 필요한 jar 모듈 목록과 추가
Problems	Job 구성 요소의 오류·경고 목록

Job 탭의 하위 항목

항목	설명
Main	Job 이름·목적·설명·버전. 회색 표시 항목은 읽기 전용이며, Job 을 닫은 상태에서 Repository 우클릭 `[Edit properties]` 로 변경
Extra	멀티 스레드 실행 여부, 프로젝트 설정 사용 여부
Stats & Logs	실행 통계·로그를 파일이나 DB 에 저장하도록 설정. 프로젝트 설정을 쓰면 개별 지정 불필요
Version	Job 버전 변경 이력

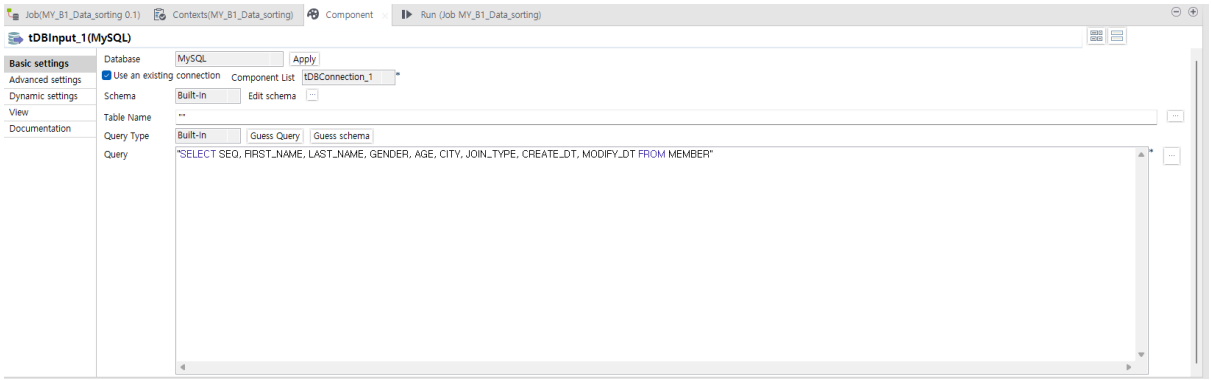
Run 탭의 하위 항목

항목	설명
Basic Run	일반 실행. 전체 데이터 흐름과 콘솔 로그를 확인
Debug Run	행 단위로 흐름을 추적하는 디버그 실행
Advanced settings	Job 별 JVM 옵션, 실행 전 저장 여부 등
Target Exec	실행 대상 지정
Memory Run	메모리 사용량을 함께 모니터링하며 실행

> **주의** `Debug Run` 도 **실제 데이터를 처리합니다**. 운영 DB 에 연결된 Job 을 디버그 실행하면 실제로 데이터가 반영되므로 반드시 테스트 환경에서 수행하세요.

컴포넌트 설정 화면 예

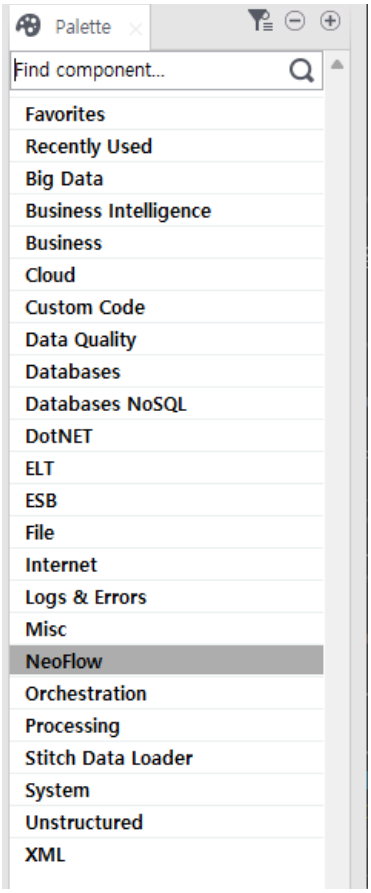
캔버스에서 컴포넌트를 클릭하면 `Component` 탭이 그 컴포넌트의 설정으로 바뀝니다. 아래는 DB 조회 컴포넌트 `tDBInput` 의 `Basic settings` 화면입니다.



▲ Component 탭 — tDBInput Basic settings

4.6 Palette 영역

화면 오른쪽의 컴포넌트 목록입니다. 기능별 폴더(패밀리)로 묶여 있고, 각 컴포넌트에 마우스를 올리면 기능 설명 툴팁이 표시됩니다. 사용할 컴포넌트를 캔버스로 끌어다 놓으면 배치됩니다.



▲ *Palette 영역 — 컴포넌트 패밀리*

- **검색** — 팔레트 상단 입력란에 이름 일부를 입력해 컴포넌트를 찾습니다.
- **Recently Used** — 최근 사용한 컴포넌트가 위쪽에 모입니다.
- **팔레트 필터** — 자주 쓰는 패밀리만 남겨 목록을 줄일 수 있습니다.
- **캔버스 직접 검색** — 캔버스를 클릭한 뒤 컴포넌트 이름을 타이핑하면 검색 상자가 열립니다. 팔레트를 뒤지는 것보다 빠릅니다.

NeoFlow 전용 컴포넌트는 `NeoFlow` 패밀리에 모여 있습니다.

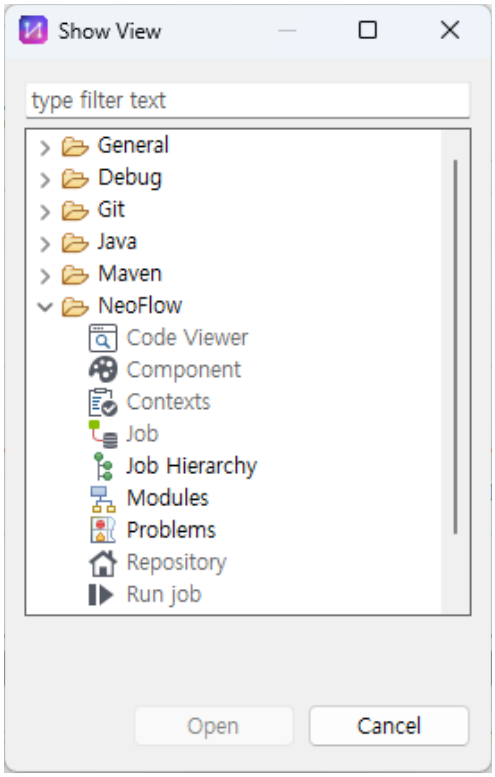


▲ *Palette — NeoFlow 전용 컴포넌트 패밀리*

> **참고** 컴포넌트 패밀리는 6 장, 표준 컴포넌트별 사용법은 8~13 장, NeoFlow 전용 `tNeoFlow*` 컴포넌트는 15 장에서 다룹니다.

4.7 뷰 추가 · 배치 되돌리기

기본 배치에 없는 뷰가 필요하면 `[Window] → [Show view...]` 에서 추가할 수 있습니다. 목록의 `NeoFlow` 폴더에 Studio 전용 뷰(Component, Contexts, Job, Job Hierarchy, Modules, Problems, Repository, Run job, Code Viewer)가 모여 있습니다.



▲ *Window > Show view... 창 — NeoFlow 카테고리*

> **TIP** 뷰를 이리저리 옮기다 배치가 엉켰다면 `[Window] → [Perspective] → [Reset Perspective...]` 한 번으로 초기 상태로 되돌아갑니다.

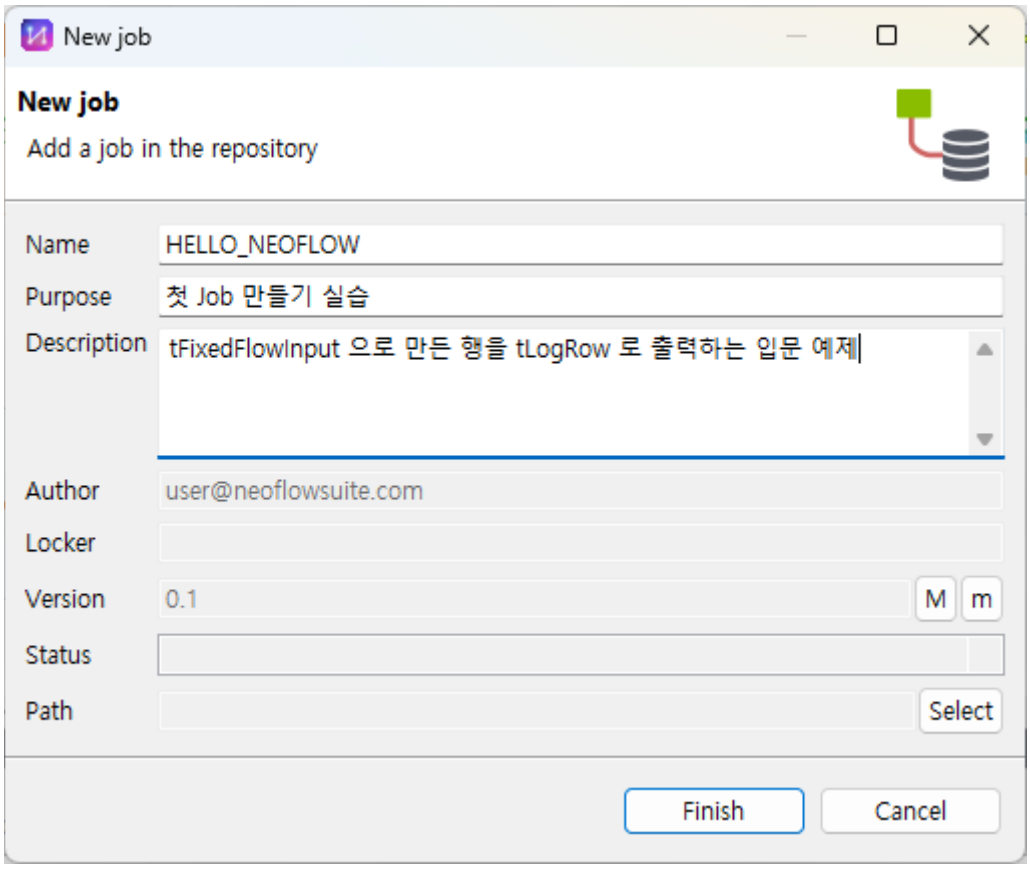
5. Repository 사용법

Repository 는 프로젝트의 모든 자원을 담는 트리입니다. 이 장에서는 노드별 우클릭 메뉴를 순서대로 살펴봅니다. 실제 손으로 따라 하는 실습은 19 장에 있으므로, 여기서는 "어떤 메뉴가 무슨 일을 하는지" 파악하는 정도로 읽으면 됩니다.

5.1 Job Designs — Job 과 폴더 만들기

Create job

`Job Designs` 노드에서 우클릭 → `[Create job]` 을 선택하면 Job 생성 마법사(창 제목은 `New job`)가 열립니다. 입력한 이름으로 Repository 에 Job 이 추가되고, 동시에 Design 영역에 빈 캔버스가 열립니다.



▲ *Create job 마법사*

항목	설명
Name	Job 이름. 영문·숫자·언더스코어만 사용하며 공백·한글은 사용할 수 없습니다
Purpose	Job 의 목적(한 줄 요약)
Description	상세 설명
Author	작성자. 로그인 정보에서 자동 입력
Version	기본 `0.1` 로 생성됩니다
Status	개발 상태(개발/테스트/운영 등). 프로젝트 설정에서 항목을 정의
Path	저장할 폴더 경로

> TIP Job 이름에 규칙을 두면 수가 늘어나도 관리가 쉽습니다. 예를 들어 «업무구분_대상_동작» 형식(`MY_B1_Data_sorting`)처럼 접두어를 통일해 두면 Repository 필터 검색으로 한 번에 묶어 볼 수 있습니다.

> 주의 Job 이름에 **한글이나 공백**을 쓰면 Java 클래스명 생성 규칙에 어긋나 빌드가 실패합니다. Purpose·Description 에는 한글을 자유롭게 써도 됩니다.

Create folder

`Job Designs` 우클릭 → `[Create folder]` 로 분류용 폴더를 만듭니다. 폴더 안에 폴더를 만드는 다단 구성도 가능하며, 만든 뒤에는 Job 을 드래그해 옮길 수 있습니다.

5.2 Job 열기와 버전 관리

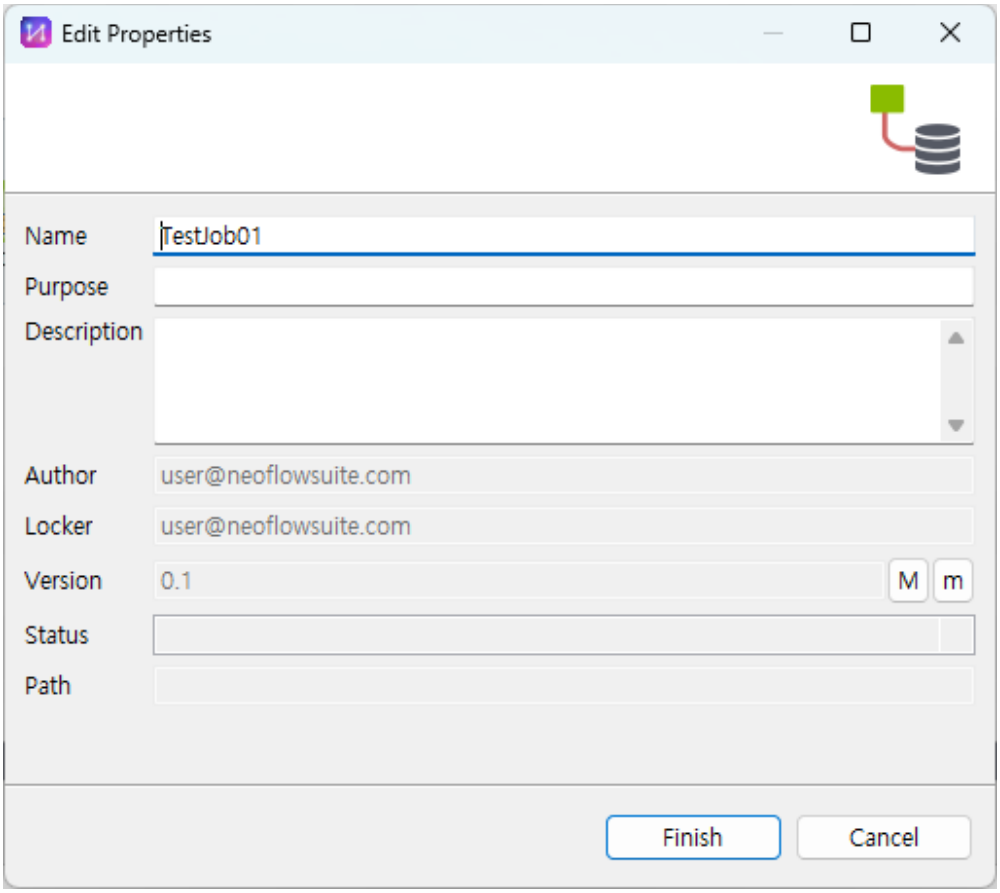
Repository 에서 Job 을 우클릭하면 아래 메뉴가 나타납니다. Job 을 더블 클릭하면 `Edit job` 과 동일하게 편집 모드로 열립니다.

메뉴	설명
Edit job	편집 모드로 열기. 속성(properties)을 제외한 모든 수정 가능
Read job	읽기 전용으로 열기. 캔버스 배경이 회색으로 표시됩니다
Open another version	버전을 올린 Job 의 이전 버전을 읽기 전용으로 열기
Open Job Hierarchy	이 Job 이 호출하는 하위 Job 구조를 트리로 확인
Edit properties	이름·목적·설명·버전·상태 변경. **Job 을 닫은 상태에서만** 가능
Duplicate	이름을 지정해 즉시 복제. 다중 버전 Job 은 복제할 버전을 선택
Copy / Paste	복사 후 붙여넣기. 기본 이름은 «원래이름_Copy»
Delete	Recycle bin 으로 이동. 비우기 전까지 복구 가능
Generate Doc As HTML	Job 구성을 HTML 문서로 생성해 zip 으로 저장

> 참고 NeoFlow Server 에 동기화한 Job 을 `Duplicate` 하거나 `Copy` → `Paste` 하면, 사본의 NeoFlow 연결 값(`NEOFLOW_*` context)이 비워집니다. 사본을 서버에 올리려면 사본을 다시 Sync 합니다(17.4).

버전 올리기 (m / M)

`Edit properties` 창에서 버전을 올릴 수 있습니다. 소문자 `m` 은 소수점 단위(0.1 → 0.2), 대문자 `M` 은 정수 단위(0.1 → 1.0)로 올립니다. **한번 올린 버전은 되돌릴 수 없습니다.** 이전 버전은 Repository 에 그대로 남아 `Open another version` 으로 열람하거나 Build 시 선택할 수 있습니다.



▲ *Edit properties — 버전 변경(m / M)*

> **참고** Job 우클릭 메뉴의 `Setup Codes Dependencies` 는 특정 Job 의 루틴 의존성을 따로 손볼 때 씁니다. 사용자 정의 루틴은 기본적으로 모든 Job 에 **자동 임포트**되므로 평소에는 쓸 일이 없습니다(5.6 참조).

5.3 내보내기 · 가져오기 · 빌드

Job 을 다른 환경으로 옮기거나 외부에서 실행할 수 있는 형태로 만드는 세 가지 메뉴입니다. 목적이 서로 다르므로 구분해서 사용하세요.

메뉴	결과물	용도
Export items	Job 소스(.item/.properties) 아카이브	다른 Studio·프로젝트로 Job 이관
Import items	—	내보낸 아카이브·디렉터리를 프로젝트로 반입
Build Job	실행 가능한 zip (스크립트·jar·context)	Studio 없이 서버·배치에서 실행

Export items

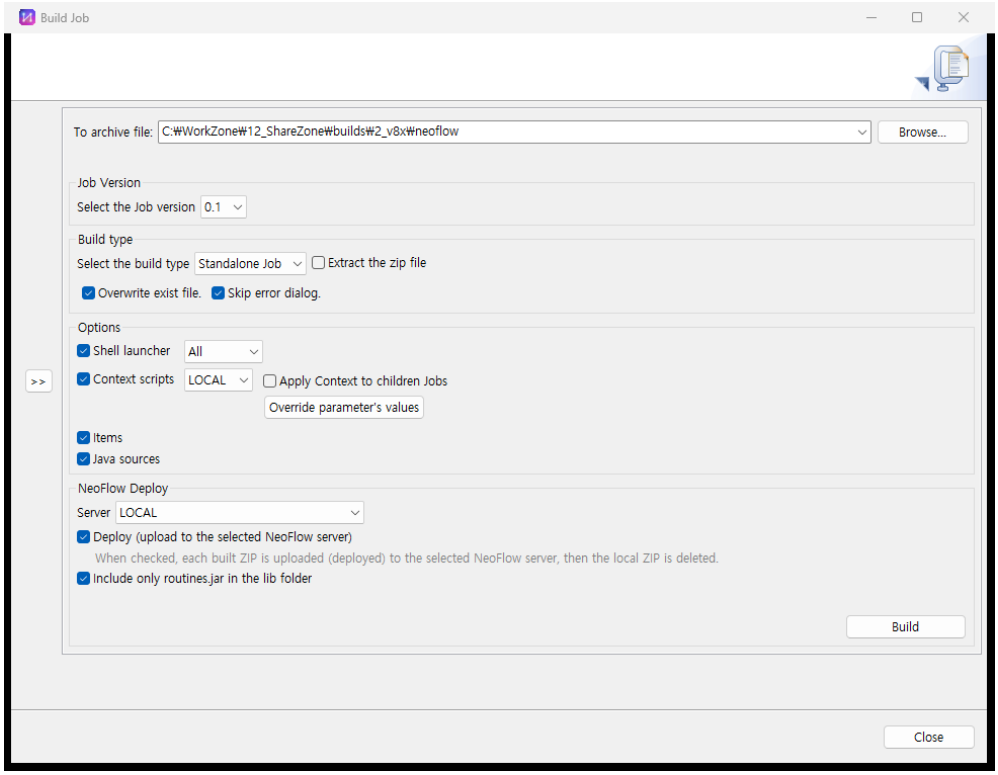
- 대상 Job 또는 폴더를 선택하고 아카이브 파일명을 지정한 뒤 [Finish] 를 누릅니다.
- 디렉터리를 선택하면 zip 대신 «프로젝트명» 폴더 형태로 저장됩니다.
- 같은 이름의 파일이 있으면 덮어쓸지 확인합니다.
- **Job 내부 오류 여부는 검사하지 않습니다** — 오류가 있는 Job 도 그대로 내보내집니다.

Import items

- `Select archive file` 로 내보낸 zip 을 고르거나 `Select root directory` 로 폴더를 고른 뒤, 가져올 항목을 체크합니다.
- 프로젝트에 같은 항목이 이미 있으면 `Overwrite existing items` 를 체크해야 덮어씁니다.
- `Internal id` 는 기본값 `Keep original internal id` 를 유지하세요. NeoFlow Server 는 Job 의 내부 ID(Talend Job Id)로 서버 Job 을 찾으므로, `Always regenerate ID at import` 를 고르면 이미 등록된 서버 Job 과 연결이 끊깁니다.

Build Job

Design 영역에 구성한 Job 을 실행 가능한 아카이브로 저장합니다. NeoFlow Studio 의 Build Job 창에는 서버 배포용 NeoFlow Deploy 영역이 있고 Deploy 가 기본으로 체크되어 있으므로, 로컬 zip 만 만들려면 Deploy 를 해제합니다(18 장). context 를 사용하는 Job 은 반드시 Context 옵션에서 사용할 context 를 포함시켜야 합니다.



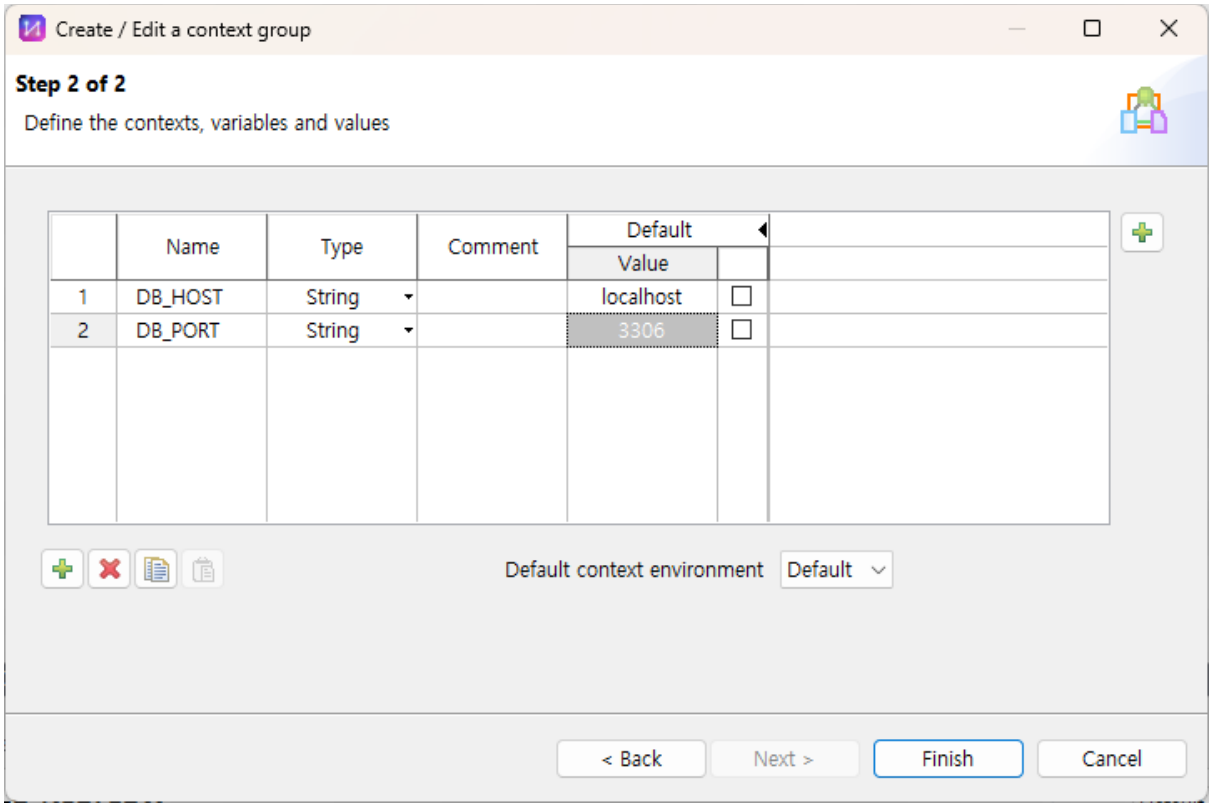
▲ *Build Job 창*

> **주의** **오류가 있는 Job 은 Build 할 수 없습니다.** 캔버스에 빨간 오류 표시가 있으면 먼저 해결한 뒤 다시 시도하세요. 반면 `Export items` 는 오류를 검사하지 않으므로, "내보내기는 됐는데 서버에서 안 돈다"면 Build 로 오류를 먼저 확인해 보는 것이 좋습니다.

> **참고** NeoFlow Server 로 Job 을 올릴 때는 Build 로 만든 zip 을 수동 업로드하는 대신 **Sync to NeoFlow Server** 를 사용하는 것이 표준 절차입니다(17·18 장, 19.10 실습 9).

5.4 Contexts — 환경별 값 관리

DB 호스트, 파일 경로, 기준일자처럼 **환경(개발·검증·운영)마다 달라지는 값**은 컴포넌트에 직접 쓰지 않고 context 변수로 뺍니다. 그러면 Job 을 고치지 않고 실행 시점의 context 만 바꿔 환경을 이전할 수 있습니다.



▲ *Create / Edit a context group 편집기*

1. `Contexts` 노드 우클릭 → `[Create context group]` 을 선택하고 이름을 입력합니다.
2. 다음 단계에서 `[+]` 버튼으로 변수를 추가하고 Name·Type·Value 를 지정합니다.
3. 환경을 나누려면 상단에서 context 환경(Default·Dev·Prod 등)을 추가하고, 같은 변수에 환경별로 다른 값을 넣습니다.
4. Job 의 Configuration 영역 `Contexts` 탭에서 이 그룹을 가져와 Job 에 적용합니다.
5. 컴포넌트 입력란에서는 `context.변수명` 으로 참조합니다.

```
"jdbc:mysql://" + context.DB_HOST + ":" + context.DB_PORT + "/" + context.DB_NAME
```

메뉴	설명
Create context group	새 Context Group 생성
Edit context group	변수·값·환경 편집
Read context group	읽기 전용으로 확인
Detect dependencies	이 Context 를 참조하는 Job 을 찾아 갱신

> **TIP** Job 안에서 context 값을 더블 클릭하면 Context Group 편집기로 바로 연결됩니다. 이때 바꾼 값을 다른 Job 에도 반영할지 물어보므로, 공통 값이면 함께 적용하세요.

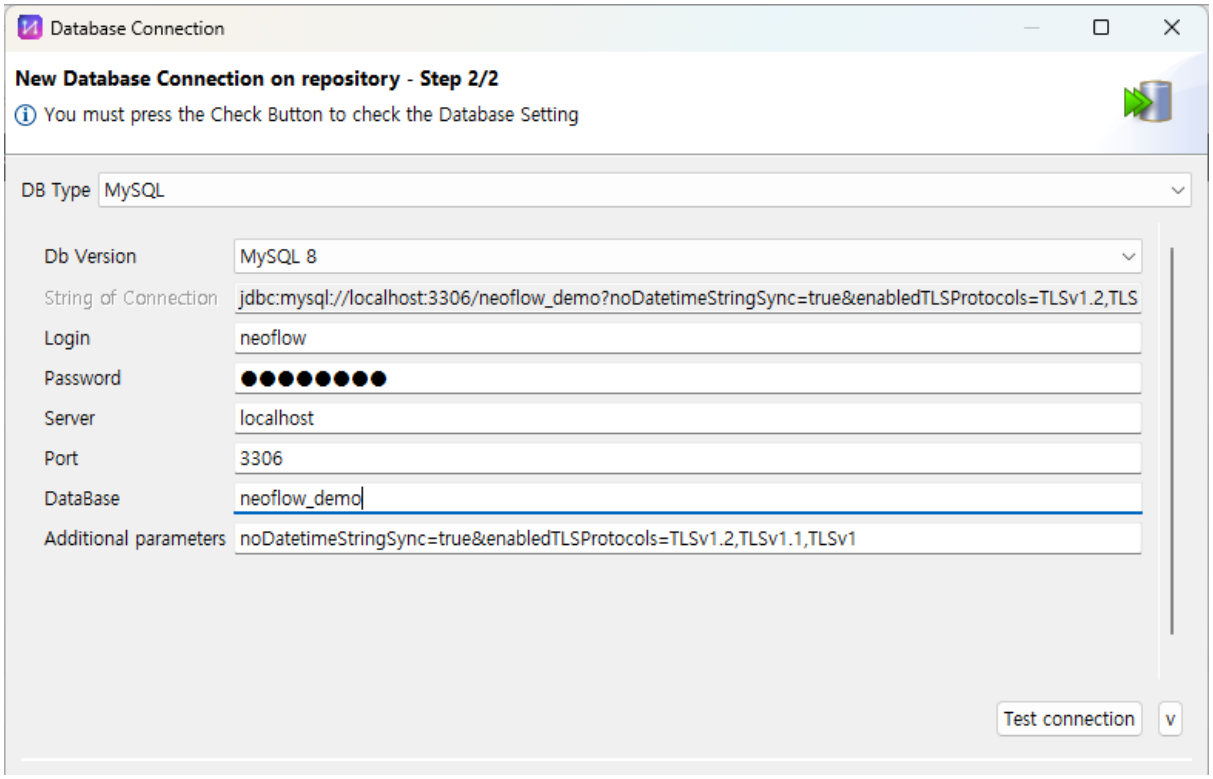
> **참고** NeoFlow Server 와 연동하면 `context.NEOFLOW_*` 서버 연결 변수가 자동 주입되고, 서버에서 넘긴 실행 파라미터도 context 로 받을 수 있습니다. 자세한 내용은 14·16 장을 참조하세요.

5.5 Metadata — DB 연결과 스키마

같은 DB 접속 정보와 테이블 구조를 Job 마다 다시 입력하는 대신, Repository 의 `Metadata` 에 한 번 등록해 두고 여러 Job 에서 참조합니다.

Create connection — DB 연결 등록

1. `Metadata > Db Connections` 우클릭 → `[Create connection]` 을 선택하고 연결 이름을 입력합니다.
2. 다음 단계에서 DB Type 을 고르면 해당 DB 의 버전 목록이 자동으로 연동됩니다. 버전을 선택하고 Login·Password·Server·Port·Database 를 입력합니다.
3. [Test connection] 을 눌러 연결을 확인합니다. 드라이버(jar)가 없으면 필요한 모듈 목록이 표시되며, 내려받으면 Configuration 영역 `Modules` 탭에 자동 추가됩니다.
4. 연결 정보를 context 로 관리하려면 [Export as context] 를 사용합니다. 입력한 이름으로 Context Group 이 자동 생성됩니다.
5. [Finish] 를 누르면 `Db Connections` 하위에 연결이 추가됩니다.



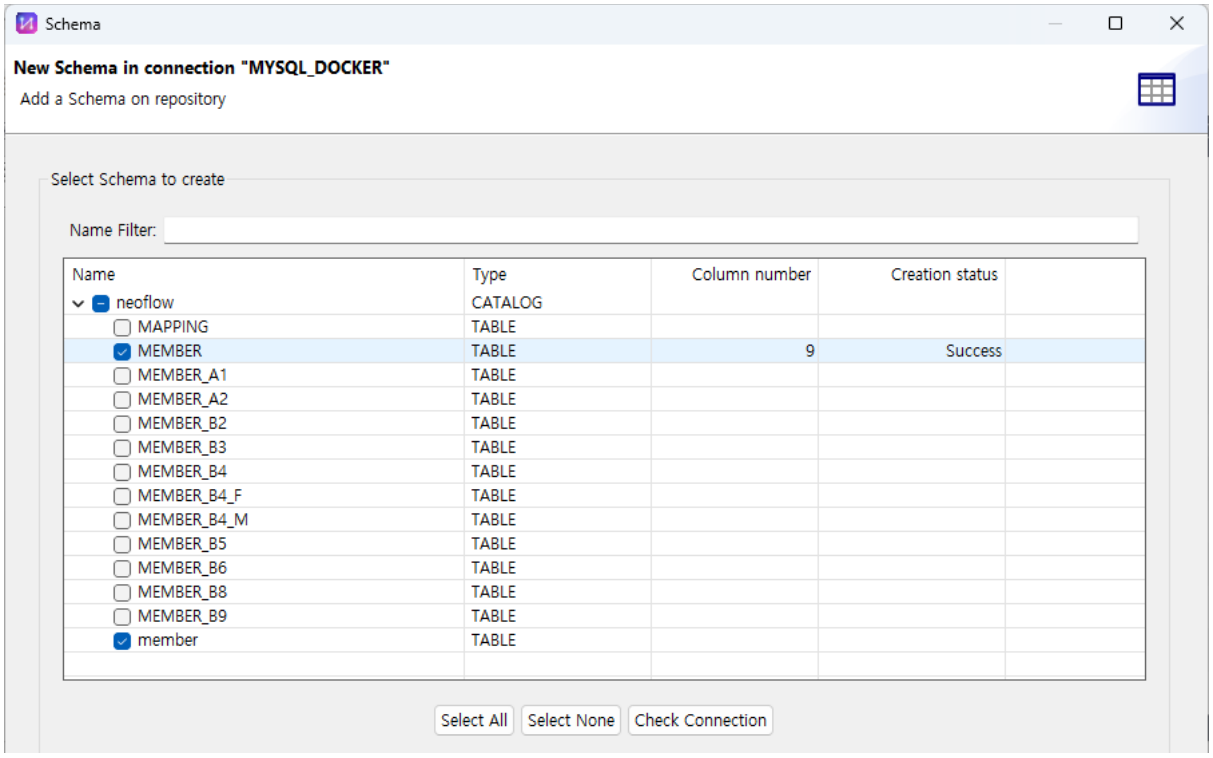
▲ *Create connection — DB 접속 정보 입력*

> **주의** 연결 확인이 실패하면 오류 팝업에 원인이 표시됩니다. 방화벽·계정 권한·드라이버 누락이 가장 흔한 원인이며, 드라이버는 라이선스 문제로 Studio 에 기본 포함되지 않는 경우가 있습니다.

Retrieve Schema — 테이블 스키마 가져오기

등록한 연결에서 우클릭 → `[Retrieve Schema]` 를 선택하면 DB 의 테이블 구조를 Studio 로 읽어 옵니다. 필터 조건에 `%` 를 쓰면 전체를 조회합니다.

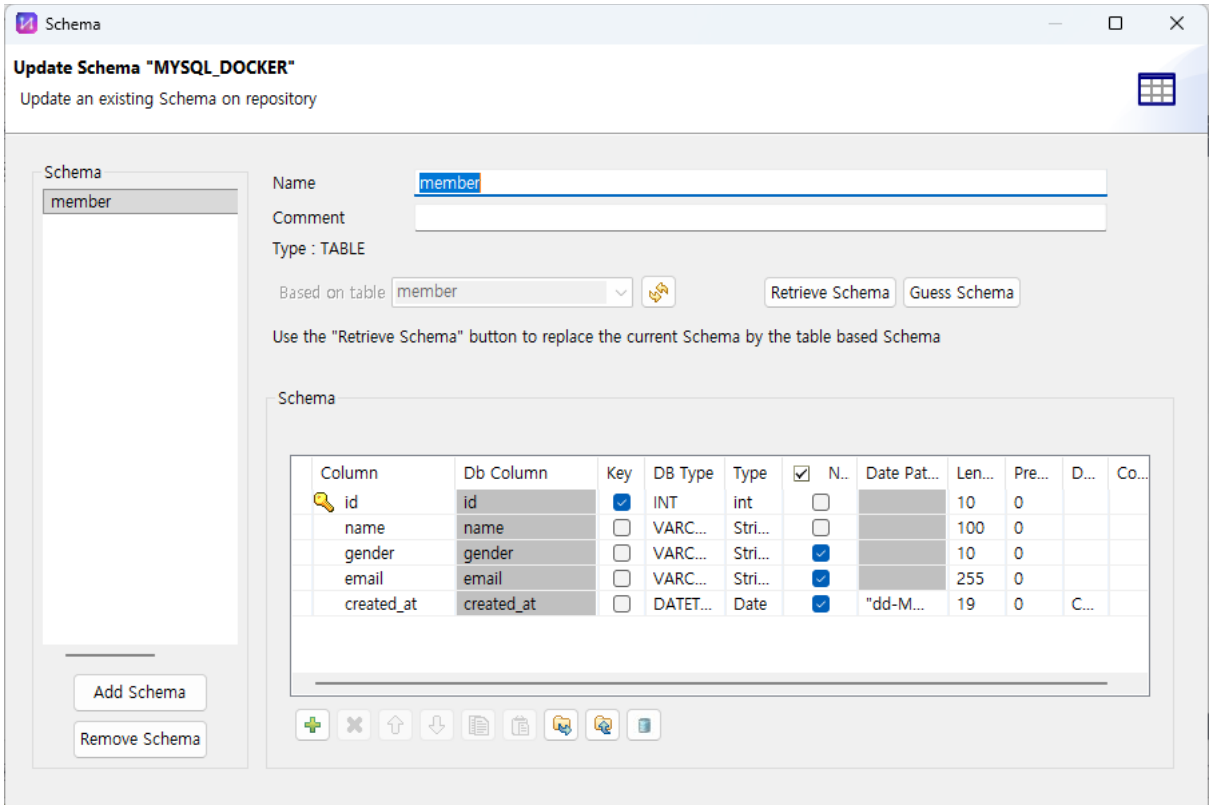
1. 필터 조건(스키마·테이블명 패턴)과 조회 대상 타입(Table·View 등)을 지정합니다.
2. 조회된 목록에서 가져올 테이블을 체크합니다.
3. Creation status 가 모두 `Success` 로 바뀌면 [Finish] 를 누릅니다.



◢ *Retrieve Schema — 테이블 선택*

Edit Schema — 스키마 수정

가져온 테이블에서 우클릭 → `[Edit Schema]` 로 컬럼 이름·타입·길이·nullable 등을 수정합니다. **이 수정은 Studio 내부의 메타데이터만 바꿉니다 — 실제 DB 는 변경되지 않습니다.**

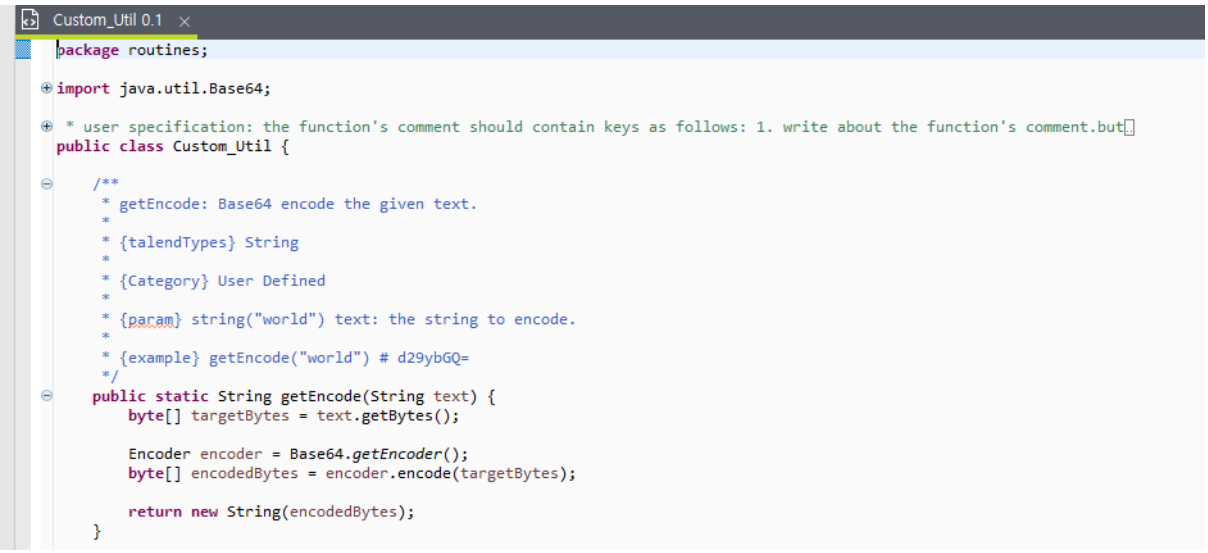


◢ *Edit Schema — 컬럼 속성 편집*

> **참고** DB 쪽 테이블 구조가 바뀌었다면 `Retrieve Schema` 를 다시 실행해 재인식시킵니다. Job 안의 컴포넌트 스키마가 어긋나면 컴포넌트 우클릭 → `[Sync columns]` 로 맞춥니다.

5.6 Code > Routines — 사용자 정의 함수

컴포넌트만으로 표현하기 어려운 로직은 Java 함수(루틴)로 만들어 두고 `tMap` 식이나 `tJava` 코드에서 호출합니다. `Code > Routines` 우클릭 → `[Create routine]` 으로 생성합니다.



▲ Code > Routines — 사용자 정의 루틴 편집 화면

작성한 루틴은 **모든 Job 에 자동으로 임포트**되므로, 패키지 경로 없이 «클래스명.메서드명» 형태로 바로 호출할 수 있습니다.

```
Custom_Util.getEncode(row1.FIRST_NAME)
```

> **참고** NeoFlow Server 가 넘긴 실행 파라미터를 context 변수에 반영하는 `NeoFlowContext` 는 Studio 에 기본 포함된 시스템 routine 으로 `Code > Routines > system` 에 있습니다. 따로 등록할 필요가 없으며, 사용법은 16 장에서 다룹니다.

> **주의** 워크스페이스의 `code/routines/` 폴더에 `.java` 파일을 직접 복사해 넣으면 Repository 메타데이터가 없어 Studio 가 인식하지 못합니다. 반드시 `[Create routine]` 으로 만든 뒤 내용을 붙여넣으세요.

5.7 Recycle bin 과 그 밖의 노드

노드 / 메뉴	설명
Recycle bin	삭제한 항목 보관소. 우클릭 `[Restore]` 로 복구
Empty recycle bin	휴지통 비우기. 실행하면 영구 삭제되어 복구할 수 없습니다
SQL Templates	표준 SQL 템플릿 제공. 사용자 템플릿 추가 가능
Documentation	프로젝트에 필요한 참고 문서를 첨부해 보관

6. 컴포넌트 기본 사용법

컴포넌트는 ETL Job 을 구성하는 기본 블록입니다. 파일·데이터베이스에서 데이터를 읽고, 변환·정제하고, 다른 저장소에 쓰는 각 작업이 하나의 컴포넌트로 표현됩니다. 이 장은 모든 컴포넌트에 공통인 조작과 개념을 다루고, 컴포넌트별 사용법은 8~13 장에서 다룹니다.

6.1 컴포넌트 기본 조작

1. 팔레트에서 컴포넌트를 찾아 캔버스로 드래그하거나, 캔버스를 클릭한 뒤 이름 일부를 타이핑해 검색·배치합니다.
2. 컴포넌트를 클릭하면 Configuration 영역 `Component` 탭에 설정이 표시됩니다.
3. `Basic settings` 에서 핵심 설정을, 필요할 때만 `Advanced settings` 에서 세부 옵션을 조정합니다.
4. 다음 컴포넌트로 데이터를 넘기려면 출력 쪽 컴포넌트에서 우클릭 → `[Row] → [Main]` 을 선택한 뒤 대상 컴포넌트를 클릭합니다.
5. `Ctrl+S` 로 저장하고 `F6` 으로 실행합니다.

탭	설명
Basic settings	스키마·연결·경로 등 핵심 설정. 대부분 여기서 끝납니다
Advanced settings	인코딩·배치 크기·통계 수집 등 고급 옵션
Dynamic settings	파라미터를 context 변수나 코드로 런타임에 결정
View	캔버스에 표시할 라벨·색 등 외관
Documentation	컴포넌트에 남기는 메모

> `TIP` 이미 연결된 두 컴포넌트 사이에 새 컴포넌트를 끼워 넣으려면, 새 컴포넌트를 **기존 연결선 위로 드래그**하면 자동으로 중간에 삽입되고 연결이 다시 이어집니다.

6.2 주요 컴포넌트 패밀리

팔레트의 컴포넌트는 기능별 패밀리로 묶여 있습니다. 표 왼쪽은 각 패밀리 대표 컴포넌트의 실제 팔레트 아이콘입니다.

아이콘	패밀리	역할	대표 컴포넌트
	Databases	DB 연결·조회·적재·SQL 실행	`tDBInput`, `tDBOutput`, `tDBRow`
	File	파일 읽기·쓰기·목록·복사·삭제	`tFileInputDelimited`, `tFileList`
	Processing	매핑·변환·집계·정렬·필터·중복 제거	`tMap`, `tAggregateRow`, `tSortRow`
	Orchestration	Job 호출·순서 제어·반복·대기	`tRunJob`, `tPrejob`, `tLoop`
	Logs & Errors	콘솔 출력·통계·에러 수집·중단	`tLogRow`, `tDie`, `tLogCatcher`
	Custom Code	Java 코드 직접 작성	`tJava`, `tJavaRow`,

아이콘	패밀리	역할	대표 컴포넌트
			`tJavaFlex`
	Misc	테스트 데이터 생성·버퍼·전역 변수	`tFixedFlowInput`, `tRowGenerator`

> TIP 컴포넌트 이름은 모두 소문자 `t` 로 시작합니다(예: `tMap`, `tDBInput`). 캔버스를 클릭하고 이름 일부를 입력하면 팔레트에서 빠르게 찾을 수 있습니다.

6.3 처음 익혀 두면 좋은 컴포넌트

수백 종의 컴포넌트가 있지만, 실제 ETL Job 의 대부분은 아래 열 몇 개의 조합으로 만들어집니다. 19 장 실습에서도 이들을 사용합니다.

아이콘	컴포넌트	하는 일
	`tDBConnection`	DB 연결을 한 번 맺고 Job 전체에서 공유
	`tDBInput`	SQL 로 DB 에서 데이터를 읽어 흐름에 흘려보냄
	`tDBOutput`	흐름의 데이터를 테이블에 INSERT/UPDATE 로 적재
	`tDBRow`	DDL·UPDATE 등 임의의 SQL 을 실행
	`tMap`	컬럼 매핑·변환·조인·조건 분기의 중심 컴포넌트
	`tFilterRow`	조건에 맞는 행만 통과시킴
	`tSortRow`	지정 컬럼 기준으로 정렬
	`tAggregateRow`	그룹별 집계(count·sum·min·max 등)
	`tFileOutputDelimited`	구분자 텍스트 파일로 저장
	`tLogRow`	데이터를 Run 콘솔에 출력 — 디버깅의 기본
	`tRunJob`	다른 Job 을 자식으로 호출
	`tDie`	Job 을 즉시 중단하고 종료 코드 반환

> 참고 `tDBInput` 처럼 `tDB` 로 시작하는 컴포넌트는 **DB 종류를 파라미터로 고르는 범용 컴포넌트**입니다. `tOracleInput`·`tMysqlInput` 처럼 DB 별로 나뉜 컴포넌트도 있지만, 신규 개발은 범용 `tDB*` 사용을 권장합니다. DB 를 바꿀 때 컴포넌트를 교체하지 않아도 되기 때문입니다.

6.4 스키마(Schema) 정의

스키마는 컴포넌트를 통과하는 데이터의 구조(컬럼 이름·타입·길이 등)를 정의합니다. 입력 컴포넌트에서 스키마를 정의하면, 연결을 따라 다음 컴포넌트로 전파(propagate)됩니다. [Edit schema] 버튼으로 스키마 편집기를 엽니다.

컬럼 속성	설명
Column	컬럼 이름(Java 식별자 규칙)
Type	데이터 타입(String, Integer, Date, BigDecimal, Boolean 등)

컬럼 속성	설명
Key	키 컬럼 여부(체크 시 키로 사용)
Nullable	null 허용 여부
Length / Precision	길이 / 소수점 자릿수(BigDecimal·Date 포맷 등)
Date pattern	Date 타입의 포맷(예: yyyy-MM-dd HH:mm:ss)

참고 스키마는 Built-In(컴포넌트에 직접 저장)과 Repository(메타데이터로 공유) 두 가지가 있습니다. 여러 Job 에서 같은 구조를 재사용한다면 Repository 메타데이터로 만들어 두고 참조하세요. Built-In 은 해당 컴포넌트에만 적용됩니다.

주의 연결 후 입력 스키마를 바꾸면 다운스트림 컴포넌트와 어긋나 빨간 느낌표(동기화 필요)가 표시됩니다. 컴포넌트를 우클릭 → [Sync columns] 로 다시 맞추세요.

6.5 context 변수

환경(개발/운영)마다 달라지는 값(DB 호스트·파일 경로·날짜 등)은 컴포넌트에 직접 쓰지 말고 context 변수로 빼냅니다. 파라미터 입력란에 `context.변수명` 으로 참조하며, 실행 시 선택한 context 그룹의 값이 주입됩니다.

```
// 컴포넌트 파라미터 입력란에서 context 변수 참조
"jdbc:mysql://" + context.DB_HOST + ":" + context.DB_PORT + "/" + context.DB_NAME

// 코드/조건식에서 context 참조
context.LOAD_DATE
```

TIP NeoFlow Server 로 Sync/Deploy 하면 `context.NEOFLOW_BASE_URL` 등 서버 연결 context 가 자동 주입됩니다(14.6). 일반 ETL 파라미터도 같은 방식으로 context 로 관리하면 환경 이전이 쉬워집니다.

참고 여러 Job 에서 함께 쓰는 context 변수는 Repository 의 Context Group 으로 관리합니다(5.4).

6.6 NeoFlow 전용 컴포넌트

팔레트의 `NeoFlow` 패밀리에는 NeoFlow Server 와 연동하기 위한 전용 컴포넌트가 있습니다. 표준 ETL 만 작성한다면 당장 쓰지 않아도 되지만, Job 을 서버 운영에 올릴 때 필요합니다.

컴포넌트	역할
`tNeoFlowPre`	실행 시작 시 서버에서 업무기준일(BizDate) 구간을 가져와 globalMap 에 채움
`tNeoFlowPost`	정상 종료 시 BizDate 워터마크를 서버에 커밋
`tNeoFlowThread`	서브잡을 시차를 두고 병렬로 기동

참고 세 컴포넌트의 파라미터·반환값·배치 규칙은 15 장에서 다룹니다. `tNeoFlowPre` 는 `tPrejob` 뒤, `tNeoFlowPost` 는 `tPostjob` 뒤에 두는 것이 배치 규칙입니다.

7. 연결(Connection)

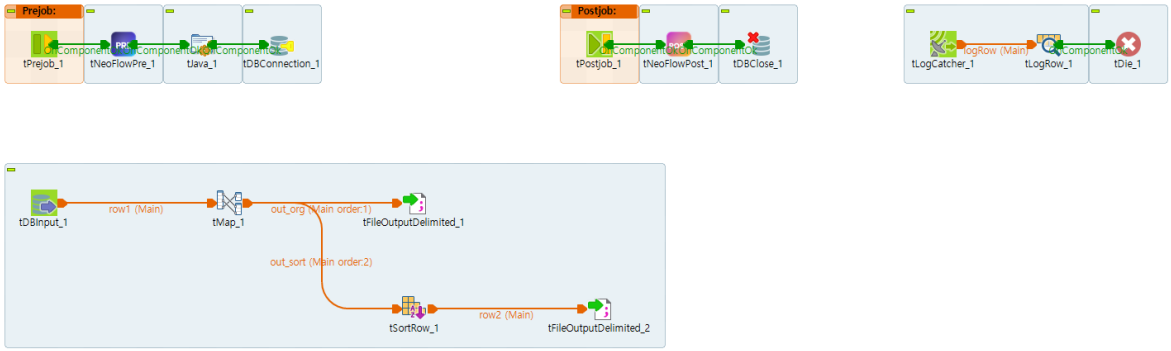
컴포넌트를 잇는 선은 단순한 화살표가 아니라 **무엇을 전달하는가**를 정의합니다. 데이터를 흘려보내는 Row, 행 단위로 반복시키는 Iterate, 실행 순서만 제어하는 Trigger 로 나뉘며, 연결 타입을 잘못 고르면 Job 이 의도와 다르게 동작합니다.

연결을 만들려면 출발 컴포넌트에서 우클릭한 뒤 원하는 연결 타입을 고르고, 도착 컴포넌트를 클릭합니다.

7.1 Row — 데이터 흐름

행(row) 단위 데이터를 다음 컴포넌트로 전달하는 연결입니다. 가장 많이 쓰입니다.

종류	설명
Main	기본 데이터 흐름. 대부분의 연결이 여기 해당합니다
Lookup	조인 시 참조(보조) 쪽 흐름. 다중 입력을 받는 컴포넌트에서만 사용
Filter	`tFilterRow` 에서 조건에 맞는 데이터를 내보내는 연결
Reject	조건 불일치·타입 오류 등으로 처리하지 못한 데이터를 모으는 연결
ErrorReject	`tMap` 의 Die on error 를 끈 상태에서 오류 행을 모으는 연결
Output	`tMap` 에서 정의한 출력 테이블별 흐름

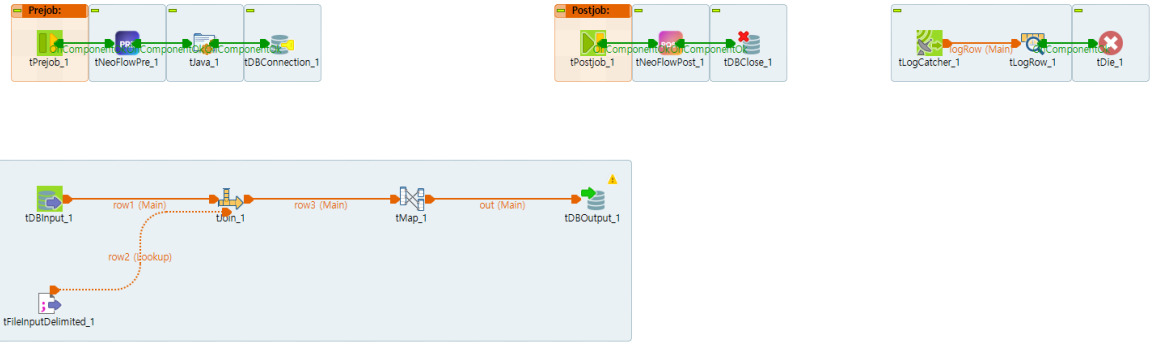


▲ Row 연결 — tDBInput → tMap → 두 갈래 출력(out_org / out_sort)

위 캔버스에서 `row1 (Main)` 은 `tDBInput_1` 이 읽은 데이터를 `tMap_1` 로 넘기는 Main 연결이고, `out_org` / `out_sort` 는 `tMap_1` 이 두 갈래로 내보내는 출력 흐름입니다. 연결선 위의 이름은 사용자가 지정한 것으로, 캔버스에서 흐름을 구분하는 데 도움이 됩니다.

Main 과 Lookup

두 데이터를 조인할 때는 **결과를 끝까지 끌고 갈 쪽을 먼저 연결**합니다. 먼저 연결한 흐름이 `Main`, 나중에 연결한 흐름이 `Lookup` 이 됩니다.



◢ *Main / Lookup — tJoin 으로 두 소스를 결합*

> **TIP** Main 과 Lookup 을 반대로 연결했다면, 바꾸려는 연결선을 우클릭해 `[Set this connection as Main]` 을 선택하면 됩니다. 지우고 다시 그릴 필요가 없습니다.

> **주의** Lookup 쪽 데이터는 보통 **메모리에 먼저 적재**된 뒤 Main 흐름과 대조됩니다. 건수가 큰 쪽을 Lookup 으로 두면 메모리 부족이 발생할 수 있으니, 작은 쪽을 Lookup 으로 연결하세요.

Reject — 실패한 데이터 살리기

데이터 타입 불일치, 제약 조건 위반, 필터 미통과 등으로 처리되지 못한 행을 버리지 않고 따로 모으는 연결입니다. 대상 컴포넌트의 `Die on error` 옵션을 **해제한 상태**에서만 사용할 수 있습니다.

```
tDBInput   --(Main)---> tMap   --(Main)---> tDBOutput
                          |
                          +--(Reject)-----> tFileOutputDelimited
```

위 구조에서 정상 행은 `tDBOutput` 으로, 처리하지 못한 행은 `tFileOutputDelimited` 로 흘러 별도 파일에 남습니다.

> **TIP** 운영 Job 에는 Reject 흐름을 함께 두는 것이 좋습니다. 오류 행 몇 건 때문에 Job 전체가 죽는 대신, 정상 데이터는 적재하고 오류 행은 별도 파일·테이블에 남겨 나중에 확인할 수 있습니다.

7.2 Iterate — 반복

데이터를 흘려보내는 대신, 뒤에 연결된 컴포넌트를 **여러 번 반복 실행**시키는 연결입니다. 폴더의 파일 목록을 하나씩 처리하거나(`tFileList`), 행 값을 순회하며 하위 Job 을 호출할 때(`tFlowToIterate` + `tRunJob`) 사용합니다.

```
tFileList  --(Iterate)--> tFileInputDelimited  --(Main)---> tDBOutput
```

위 구조에서는 폴더 안의 파일 수만큼 뒤쪽 흐름 전체가 반복 실행됩니다. 파일 하나를 처리할 때마다 `tFileList` 가 다음 파일로 넘어갑니다.

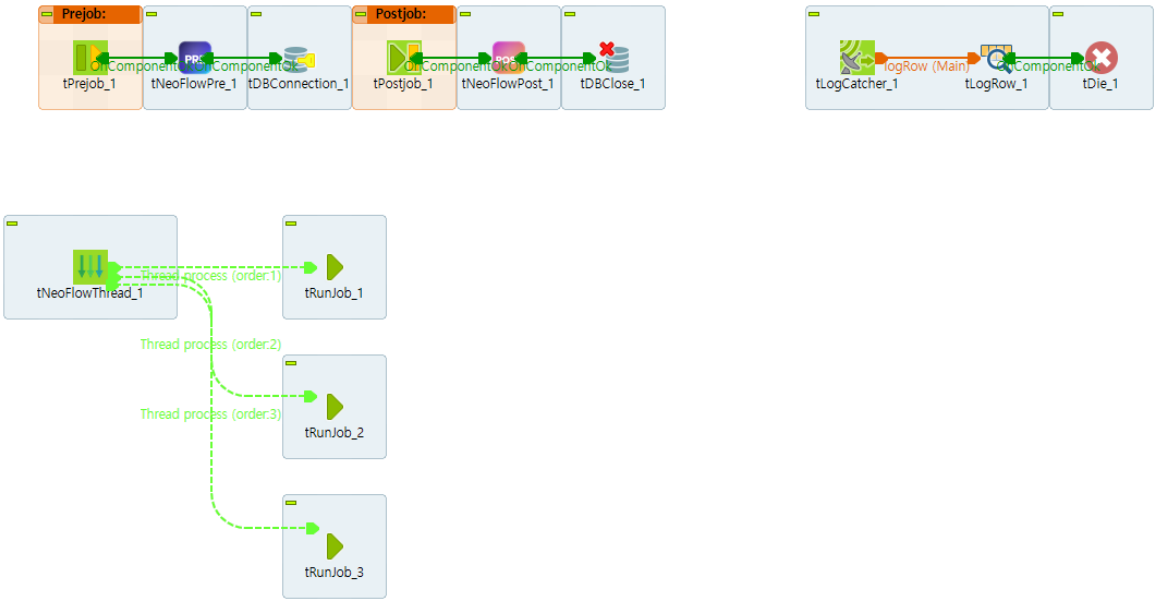
Iterate 연결에는 스키마가 없습니다. 반복 중 현재 값(파일 경로·행 값 등)은 `globalMap` 에서 꺼냅니다(20.3).

7.3 Trigger — 실행 순서 제어

데이터를 전달하지 않고 **실행 순서와 조건**만 정의하는 연결입니다. 서브잡 사이의 선후 관계를 만들 때 사용합니다.

트리거	의미
OnSubjobOk	앞 서브잡이 오류 없이 끝나면 다음 서브잡 실행
OnSubjobError	앞 서브잡이 오류로 끝났을 때 다음 서브잡 실행
OnComponentOk	앞 **컴포넌트**가 오류 없이 끝나면 다음 실행
OnComponentError	앞 컴포넌트에서 오류가 나면 즉시 다음 실행
Run if	지정한 Java 조건식이 참일 때만 다음 실행

`OnSubjob*` 은 **서브잡 전체**가 끝난 뒤 판정하고, `OnComponent*` 는 **해당 컴포넌트 하나**가 끝난 시점에 판정합니다. 서브잡 안의 다음 단계로 넘어가려면 `OnComponentOk`, 별개의 처리 덩어리를 이어 붙이려면 `OnSubjobOk` 를 쓰는 것이 일반적입니다.



▲ *Trigger 연결 — tRunJob 으로 자식 Job 3 개를 병렬 기동*

> **TIP** Run if 조건식에는 Java 식을 그대로 씁니다. 예를 들어 앞 컴포넌트가 처리한 행 수가 0 보다 클 때만 진행하려면 `((Integer)globalMap.get("tDBInput_1_NB_LINE")) > 0` 처럼 작성합니다.

7.4 서브잡(Subjob) 이해하기

캔버스에서 옅은 회색 배경으로 묶여 보이는 덩어리가 서브잡입니다. **Row 연결로 이어진 컴포넌트 묶음 하나가 하나의 서브잡**이며, Trigger 연결은 서브잡과 서브잡 사이를 잇습니다. 하나의 Job 안에 여러 서브잡을 두면 순차 처리·병렬 처리·오류 처리를 구조적으로 나눌 수 있습니다.

```
SUBJOB 1   tPrejob --(OnComponentOk)--> tDBConnection
                |
          (OnSubjobOk)
```

```
                v
SUBJOB 2   tDBInput --(Main)──> tMap --(Main)──> tDBOutput
                |
          (OnSubjobOk)
                v
SUBJOB 3   tPostjob --(OnComponentOk)──> tDBClose
```

참고 `tPrejob` 서브잡은 Job 의 다른 무엇보다 **먼저**, `tPostjob` 서브잡은 **마지막**에 실행됩니다. DB 연결 열기·닫기처럼 Job 전체를 감싸는 처리는 이 두 서브잡에 둡니다.

TIP 서브잡을 접거나 펼치려면 서브잡 왼쪽 위의 작은 사각형을 클릭합니다. 컴포넌트가 많은 Job 에서 화면을 정리할 때 유용합니다.

8. 데이터베이스 컴포넌트

Databases 패밀리는 관계형 DB 에 연결해 데이터를 조회·적재하고 SQL 을 실행합니다. `tDBInput`·`tDBOutput` 처럼 `tDB` 로 시작하는 컴포넌트는 DB 종류를 파라미터로 고르는 범용 컴포넌트이고, `tMysqlInput`·`tOracleInput` 처럼 DB 별로 나뉜 컴포넌트도 있습니다. 동작·파라미터 구조는 같습니다.

아이콘	컴포넌트	용도
	`tDBConnection`	DB 연결을 한 번 열고 여러 컴포넌트가 공유
	`tDBInput`	SQL SELECT 로 행을 조회해 Main 흐름으로 전달
	`tDBOutput`	흐름의 행을 테이블에 Insert/Update/Delete
	`tDBRow`	결과 흐름 없이 임의 SQL(DDL·DML·프로시저) 실행
	`tDBCommit`	트랜잭션 커밋
	`tDBRollback`	트랜잭션 롤백
	`tDBClose`	공유 연결 닫기

> **참고** 8~13 장은 실무 ETL 에서 자주 쓰는 표준 컴포넌트를 골라 설명합니다. 컴포넌트별 전체 파라미터와 시나리오는 Talend Open Studio 공식 문서(Talend Studio User Guide 8.0, Talend Open Studio Components Reference Guide)를 함께 참고하세요(20.7).

8.1 tDBInput — 조회

tDBInput 은 정의한 스키마 순서와 정확히 일치하는 SELECT 쿼리를 실행하고, 결과 행을 Main 흐름으로 다음 컴포넌트에 전달합니다.

- 핵심 파라미터 — DB 연결 정보(또는 Use an existing connection), 스키마, Query.
- Query 는 스키마 컬럼 순서·이름과 맞아야 합니다. `Guess Query` 버튼으로 스키마 기반 쿼리를 생성할 수 있습니다.
- 증분 적재 시 WHERE 절에 context/globalMap 값을 넣어 기간을 한정합니다.

```
"SELECT id, name, updated_at FROM sales " +
"WHERE updated_at >= '" + context.START_DATE + "'"
```

8.2 tDBOutput — 적재

tDBOutput 은 입력 흐름의 행을 대상 테이블에 기록합니다. Action on data 로 동작을 정합니다.

Action on data	동작
Insert	신규 행 삽입(중복 시 에러 가능)
Update	키 기준으로 기존 행 수정
Insert or update	있으면 수정, 없으면 삽입
Update or insert	수정 시도 후 대상 없으면 삽입
Delete	키 기준으로 행 삭제

> TIP Advanced settings 의 Batch size 로 일괄 처리 건수를 키우면 대량 적재 성능이 크게 향상됩니다. 실패 행만 따로 처리하려면 tDBOutput 에 [Row] → [Reject] 흐름을 연결하세요(Die on error 해제 필요).

8.3 연결 공유 — tDBConnection 패턴

같은 DB 를 여러 컴포넌트가 쓴다면, tDBConnection 으로 연결을 한 번만 열고 각 컴포넌트에서 [Use an existing connection]을 선택해 공유합니다. 매 컴포넌트가 연결을 새로 여는 비용을 줄이고, 하나의 트랜잭션으로 묶을 수 있습니다.

```
tDBConnection --(OnSubjobOk)──▶ tDBInput --(Main)──▶ tMap ──▶ tDBOutput
                                                                  |
                                                              (OnSubjobOk)
                                                                  v
                                                            tDBCommit ──▶ tDBClose
```

8.4 tDBRow — 임의 SQL

tDBRow 는 결과 흐름 없이 SQL 문을 그대로 실행합니다. 테이블 TRUNCATE, 인덱스 생성, 저장 프로시저 호출, 적재 전후 정리 작업 등에 씁니다. 입력 흐름을 받으면 행마다 SQL 을 실행할 수도 있습니다.

> 주의 tDBRow 로 DDL(TRUNCATE/DROP 등)을 실행할 때는 대상 테이블·스키마를 반드시 context 로 분리해 운영 DB 에서 오작동하지 않도록 하세요.

9. 파일 컴포넌트

File 패밀리는 구분자 텍스트(CSV/TSV)·Excel·JSON·XML 파일을 읽고 쓰며, 파일 목록 순회·복사·삭제 등 관리 작업을 제공합니다.

아이콘	컴포넌트	용도
	tFileInputDelimited	구분자 텍스트 파일을 행→필드로 읽기
	tFileOutputDelimited	흐름을 구분자 텍스트로 쓰기
	tFileInputExcel	Excel(xls/xlsx) 읽기
	tFileOutputExcel	Excel(xls/xlsx) 쓰기
	tFileInputJSON	JSON 파일 파싱(JSONPath)
	tFileInputXML	XML 파일 파싱(XPath)
	tFileList	디렉터리의 파일 목록을 Iterate 로 순회
	tFileCopy	파일 복사·이동·이름변경
	tFileDelete	파일 삭제
	tFileArchive	여러 파일을 zip 으로 압축

9.1 tFileInputDelimited — 구분자 파일 읽기

tFileInputDelimited 는 텍스트 파일을 한 줄씩 읽어 필드 구분자로 나눈 뒤 스키마에 맞춰 다음 컴포넌트로 전달합니다.

파라미터	설명
File name/Stream	읽을 파일 경로(보통 context 로 지정)
Row Separator	행 구분자(기본 \n)
Field Separator	필드 구분자(예: 쉼표, 탭 \t)
Header / Footer	건너뛸 머리/꼬리 행 수(헤더 1 이면 첫 줄 제외)
Limit	읽을 최대 행 수(0=제한 없음)

9.2 tFileOutputDelimited — 구분자 파일 쓰기

tFileOutputDelimited 는 입력 흐름을 텍스트 파일로 출력합니다. Append(이어쓰기), Include header(헤더 포함), 인코딩(Advanced) 등을 설정합니다. 같은 경로에 누적 적재할 때는 Append 를 켜고, 매 실행 새로 만들 때는 끕니다.

9.3 tFileList — 파일 목록 순회

tFileList 는 디렉터리에서 마스크(예: `*.csv`)에 맞는 파일 목록을 만들고, Iterate 연결로 후속 서브잡을 파일 수만큼 반복 실행합니다. 현재 반복의 파일 정보는 globalMap 에서 꺼냅니다.

```
tFileList --(Iterate)⟶ tFileInputDelimited --(Main)⟶ tDBOutput
```

```
// 현재 반복의 파일 정보(tFileInputDelimited 의 File name 란에 사용)
((String) globalMap.get("tFileList_1_CURRENT_FILEPATH"))   // 전체 경로
((String) globalMap.get("tFileList_1_CURRENT_FILE"))       // 파일명
((Integer) globalMap.get("tFileList_1_NB_FILE"))           // 처리 순번
```

10. 변환·처리 컴포넌트

Processing 패밀리는 데이터를 매핑·정제·집계·정렬·필터링합니다. 그중 tMap 은 가장 강력하고 자주 쓰이는 핵심 컴포넌트입니다.

아이콘	컴포넌트	용도
	tMap	다중 입력→다중 출력 매핑·변환·조인·필터(올인원)
	tFilterRow	조건으로 행을 통과/거부(Filter/Reject)로 분기
	tAggregateRow	그룹별 집계(합계·평균·건수 등)
	tSortRow	하나 이상 컬럼 기준 정렬
	tUniqRow	중복 제거(unique/duplicates 분리)
	tReplicate	하나의 흐름을 동일 스키마로 복제
	tUnite	같은 스키마 여러 흐름을 병합(UNION)
	tConvertType	컬럼 데이터 타입 변환
	tReplace	문자열 값 검색·치환

10.1 tMap — 개요와 Map Editor

tMap 은 하나 이상의 입력 흐름을 받아 변환·조인하고 하나 이상의 출력 흐름으로 라우팅합니다. [...] 버튼으로 Map Editor 를 열면 좌측은 입력(메인+lookup), 가운데는 변수(Var), 우측은 출력 영역입니다. 입력 컬럼을 출력으로 드래그해 매핑하고, 표현식 칸에 Java 식을 써서 가공합니다.

```
              tDBInput (참조 테이블)
                   |
                   | Row > Lookup
                   v
tFileInput --(Row > Main)──▶ [ tMap ] --(out: 매핑된 결과)──▶ tDBOutput
                                 |
                                 '----(reject)----> tLogRow
```

10.2 tMap — Lookup(조인)

메인 흐름 외에 참조 테이블을 Lookup 으로 연결하면 조인할 수 있습니다. Map Editor 에서 lookup 테이블의 키와 메인 키를 연결하고, 조인·매칭 방식을 정합니다.

설정	값과 의미
Join Model	Left Outer Join(기본, 매칭 없어도 메인 통과) / Inner Join(매칭만 통과)
Match Model	Unique match(마지막 1 건) / First match(첫 1 건) / All matches(여러 건)
Lookup Model	Load once(시작 시 전체 적재) / Reload at each row(행마다 재조회)
inner join reject	Inner Join 에서 매칭 실패한 메인 행을 별도 출력으로 분기

> TIP 대용량 참조는 Load once 가 메모리에 한 번만 올려 빠릅니다. 참조 데이터가 실행 중 바뀌거나 메인 키별로만 조회하면 충분할 때는 Reload at each row(요청 시 키로 조회)가 메모리에 유리합니다.

10.3 tMap — 출력·필터·Reject

출력 테이블마다 표현식 필터(Expression filter)를 걸어 조건에 맞는 행만 해당 출력으로 보낼 수 있어, 한 tMap 에서 여러 갈래로 라우팅이 가능합니다. 어떤 출력에도 못 들어간 행은 "Catch output reject"를 켠 reject 출력으로 모읍니다.

```
// 출력 표현식 예 (입력 row1 을 가공)
row1.firstName + " " + row1.lastName         // 컬럼 결합
row1.amount == null ? 0 : row1.amount         // null 처리
Relational.ISNULL(row1.email) ? "N/A" : row1.email

// 출력 필터 예 (이 출력으로 보낼 행 조건)
row1.amount > 1000
```

> 참고 tMap 의 Var(변수) 영역에 중간 계산을 정의해 여러 출력에서 재사용할 수 있습니다. 변수는 위에서 아래로 평가되며 `Var.변수명` 으로 참조합니다.

10.4 tFilterRow / tAggregateRow / tSortRow

tFilterRow 는 조건(컬럼 비교 또는 고급 Java 식)을 만족하는 행을 Filter 흐름으로, 나머지를 Reject 흐름으로 보냅니다. tAggregateRow 는 group by 컬럼으로 묶어 집계 함수를 적용합니다. tSortRow 는 하나 이상 컬럼을 기준으로 오름/내림차순 정렬합니다.

tAggregateRow 함수	설명
count	그룹 내 행 수
sum / avg	합계 / 평균
min / max	최소 / 최대
first / last	그룹의 첫 / 마지막 값
list / count(distinct)	값 목록 / 고유 값 개수

> 주의 tAggregateRow 와 tSortRow 는 입력을 메모리에 모으는 블로킹 컴포넌트입니다. 초대용량은 tSortRow 의 디스크 임시파일 옵션(Advanced)이나 DB 측 정렬/집계(GROUP BY)를 고려하세요.

10.5 tUniqRow / tReplicate / tUnite

- `tUniqRow` — 지정 키 기준 중복 제거. 고유 행은 unique 흐름, 중복 행은 duplicates 흐름으로 분리.
- `tReplicate` — 하나의 흐름을 동일 스키마로 복제해 두 갈래로 동시 처리.
- `tUnite` — 스키마가 같은 여러 흐름을 하나로 병합(UNION). 입력 순서대로 이어 붙임.

11. 오케스트레이션 컴포넌트

Orchestration 패밀리는 데이터를 직접 다루지 않고 Job 의 실행 순서·반복·하위 Job 호출을 제어합니다.

아이콘	컴포넌트	용도
	tPrejob	Job 본체보다 먼저 실행(연결 준비 등)
	tPostjob	Job 본체 후 항상 실행(정리 등)
	tRunJob	다른 Job(자식 Job) 호출, context 전달
	tFlowToIterate	데이터 흐름의 각 행을 Iterate(반복)로 변환
	tIterateToFlow	Iterate 를 다시 데이터 흐름(행)으로 변환
	tLoop	지정 횟수/조건만큼 반복(for/while)
	tSleep	지정 시간 동안 대기
	tWaitForFile	파일 생성/도착을 대기

11.1 tPrejob / tPostjob

tPrejob 에 연결된 작업은 Job 의 다른 어떤 작업보다 먼저, tPostjob 에 연결된 작업은 가장 마지막에(실패해도) 실행됩니다. 보통 연결 열기·정리, 시작/종료 로그에 씁니다.

```
tPrejob  --(OnComponentOk)──▶ tDBConnection
[메인 ETL 서브잡들 ...]
tPostjob --(OnComponentOk)──▶ tDBClose
```

> **참고** NeoFlow BizDate 컴포넌트를 쓰는 Job 은 `tPrejob` 뒤에 `tNeoFlowPre`, `tPostjob` 뒤에 `tNeoFlowPost` 를 `OnComponentOk` 로 연결합니다(15 장).

11.2 tRunJob — 자식 Job 호출

tRunJob 은 다른 Job 을 하위 작업으로 실행합니다. 큰 워크플로우를 재사용 가능한 작은 Job 으로 쪼갤 때 씁니다. context 파라미터를 넘기거나 전체 context 를 전달할 수 있습니다.

- Job — 호출할 자식 Job 과 버전 선택. `Use dynamic job` 으로 이름을 런타임에 지정할 수도 있음.
- Context / Transmit whole context — 부모의 context 를 자식에 전달하거나, 개별 파라미터만 매핑.
- Die on child error — 자식 실패 시 부모도 중단할지 여부.

```
// 자식 Job 종료 코드/예외 (tRunJob 이후)
((Integer) globalMap.get("tRunJob_1_CHILD_RETURN_CODE"))
((String)  globalMap.get("tRunJob_1_CHILD_EXCEPTION_STACKTRACE"))
```

11.3 tFlowToIterate / tIterateToFlow

tFlowToIterate 는 데이터 흐름의 각 행을 하나의 반복(Iteration)으로 바꿔, 행마다 후속 서브잡을 실행하게 합니다(예: 행에 담긴 파일명마다 다른 Job 호출). 각 컬럼 값은 globalMap 에 `<연결명>.<컬럼>` 키로 저장됩니다. 반대로 tIterateToFlow 는 Iterate 를 다시 행 흐름으로 되돌립니다.

```
// tFlowToIterate 이후, 현재 반복 행의 컬럼 값 참조
((String) globalMap.get("row1.fileName"))
```

11.4 tLoop

tLoop 은 후속 서브잡을 정해진 횟수(For) 또는 조건(While)만큼 반복 실행합니다. 현재 반복 인덱스는 globalMap 의 `tLoop_1_CURRENT_VALUE` / `tLoop_1_CURRENT_ITERATION` 으로 꺼냅니다. 페이지네이션 호출, 재시도 루프 등에 활용합니다.

12. 로그·에러 처리 컴포넌트

Logs & Errors 패밀리는 실행 로그·통계를 출력·수집하고, 오류 시 Job 을 경고·중단합니다. 운영 ETL 의 관측성과 안정성을 책임지는 컴포넌트입니다.

아이콘	컴포넌트	용도
	tLogRow	흐름의 행을 콘솔에 출력(디버깅·확인)
	tDie	Job 을 즉시 중단하고 종료 코드 반환
	tWarn	경고 메시지 발생(tLogCatcher 가 수집)
	tLogCatcher	tDie/tWarn/Java 예외를 행 흐름으로 수집
	tStatCatcher	컴포넌트·Job 실행 통계 수집
	tFlowMeterCatcher	tFlowMeter 가 측정한 흐름 건수 수집
	tAssert	검증 조건 평가(테스트)
	tAssertCatcher	tAssert 결과를 행 흐름으로 수집

12.1 tLogRow — 콘솔 출력

tLogRow 는 통과하는 행을 콘솔에 그대로 출력합니다. 흐름 중간에 끼워 데이터를 눈으로 확인할 때 유용합니다. 출력 모드는 Basic(구분자), Table(표), Vertical(키:값 세로 나열)이 있습니다.

> TIP 디버깅이 끝난 tLogRow 는 우클릭 → [Deactivate]로 비활성화하면(회색 처리) 삭제하지 않고도 출력만 끌 수 있습니다.

12.2 tDie / tWarn

tDie 는 치명적 오류 시 Job 을 즉시 중단하고 지정한 종료 코드·메시지를 남깁니다. tWarn 은 중단 없이 경고만 발생시킵니다. 두 컴포넌트의 메시지는 tLogCatcher 로 모아 한곳에서 기록할 수 있습니다.

12.3 tLogCatcher — 에러 수집 패턴

tLogCatcher 는 같은 Job 의 tDie/tWarn 호출과 Java 예외(Use Runtime exceptions 체크 시)를 행 흐름으로 받아, 메시지·코드·우선순위·발생 위치 등을 컬럼으로 제공합니다. 이를 tLogRow 로 출력하거나 tDBOutput 으로 로그 테이블에 적재해 중앙 집계합니다.

```
tDie / tWarn / (Java 예외)
        |
        v
   tLogCatcher --(Main)──> tDBOutput (로그 테이블)
                       '──> tLogRow  (콘솔)
```

> 참고 tLogCatcher 흐름의 주요 컬럼: `Moment`(시각), `pid`, `Type`, `code`, `priority`, `origin`(발생 컴포넌트), `message`. 운영에서는 이 흐름을 DB·파일로 적재해 실패 원인을 추적합니다.

12.4 tStatCatcher — 실행 통계

tStatCatcher 는 컴포넌트·서브잡·Job 의 시작/종료/소요시간 같은 통계를 수집합니다. 각 컴포넌트 Advanced settings 의 `tStatCatcher Statistics` 를 체크하면 해당 컴포넌트의 통계가 잡혀 tStatCatcher 흐름으로 흘러들어옵니다. 처리 시간·건수를 모니터링 테이블에 적재할 때 사용합니다.

13. 코드·기타 컴포넌트

Misc·Custom Code 패밀리는 임의 Java 코드 실행, 테스트 데이터 생성, context 로딩, 전역 변수 설정 등 표준 컴포넌트로 표현하기 어려운 작업을 처리합니다.

아이콘	컴포넌트	용도
	tJava	흐름과 무관하게 Java 코드 1 회 실행
	tJavaRow	입력 흐름의 행마다 Java 코드 실행
	tJavaFlex	시작/행/종료 3 구간 Java 코드 작성
	tFixedFlowInput	고정 값 행을 즉석 생성(테스트·상수 흐름)
	tRowGenerator	규칙 기반 더미 데이터 대량 생성(테스트)
	tContextLoad	key-value 흐름으로 context 변수를 런타임 로딩
	tSetGlobalVar	globalMap 전역 변수 설정
	tBufferOutput	결과를 버퍼에 저장(부모 Job 반환 등)

13.1 tJava / tJavaRow / tJavaFlex

컴포넌트	차이
tJava	흐름과 무관하게 코드 1 회 실행(초기화·로그·계산)
tJavaRow	입력 흐름의 행마다 코드 실행(input_row 로 컬럼 접근)
tJavaFlex	시작 코드 / 행 코드 / 종료 코드 3 구간을 모두 직접 작성

```
// tJava — 전역 변수 설정/로그
System.out.println("load date = " + context.LOAD_DATE);
globalMap.put("runId", java.util.UUID.randomUUID().toString());

// tJavaRow — 입력 행(input_row)을 출력 행(output_row)으로 가공
output_row.id    = input_row.id;
output_row.name  = input_row.name.trim().toUpperCase();
```

13.2 tFixedFlowInput / tRowGenerator

tFixedFlowInput 은 직접 입력한 소수의 고정 행을 흐름으로 만들어, 매핑·로직을 빠르게 테스트하거나 상수 행을 주입할 때 씁니다. tRowGenerator 는 컬럼별 생성 규칙(랜덤·순번·패턴)으로 대량의 더미 데이터를 만들어 성능 테스트에 활용합니다.

13.3 tContextLoad — context 동적 로딩

tContextLoad 는 두 컬럼(key, value) 흐름을 받아 그 값으로 context 변수를 런타임에 덮어씁니다. 설정을 DB·파일에서 읽어와 환경값을 주입할 때 유용합니다. 정의되지 않은 키나 누락 값은 경고/에러로 알려줍니다.

```
tDBInput (key, value) --(Main)⟶ tContextLoad   // context 를 DB 값으로 갱신
```

13.4 tSetGlobalVar / globalMap

tSetGlobalVar 는 key-value 를 globalMap 에 넣어 Job 전역에서 공유합니다. globalMap 은 컴포넌트가 만든 반환값·전역 변수를 담는 런타임 맵으로, 어디서든 `globalMap.get("키")` 로 꺼냅니다(형 변환 필요). 부록 20.3 에 자주 쓰는 키를 정리했습니다.

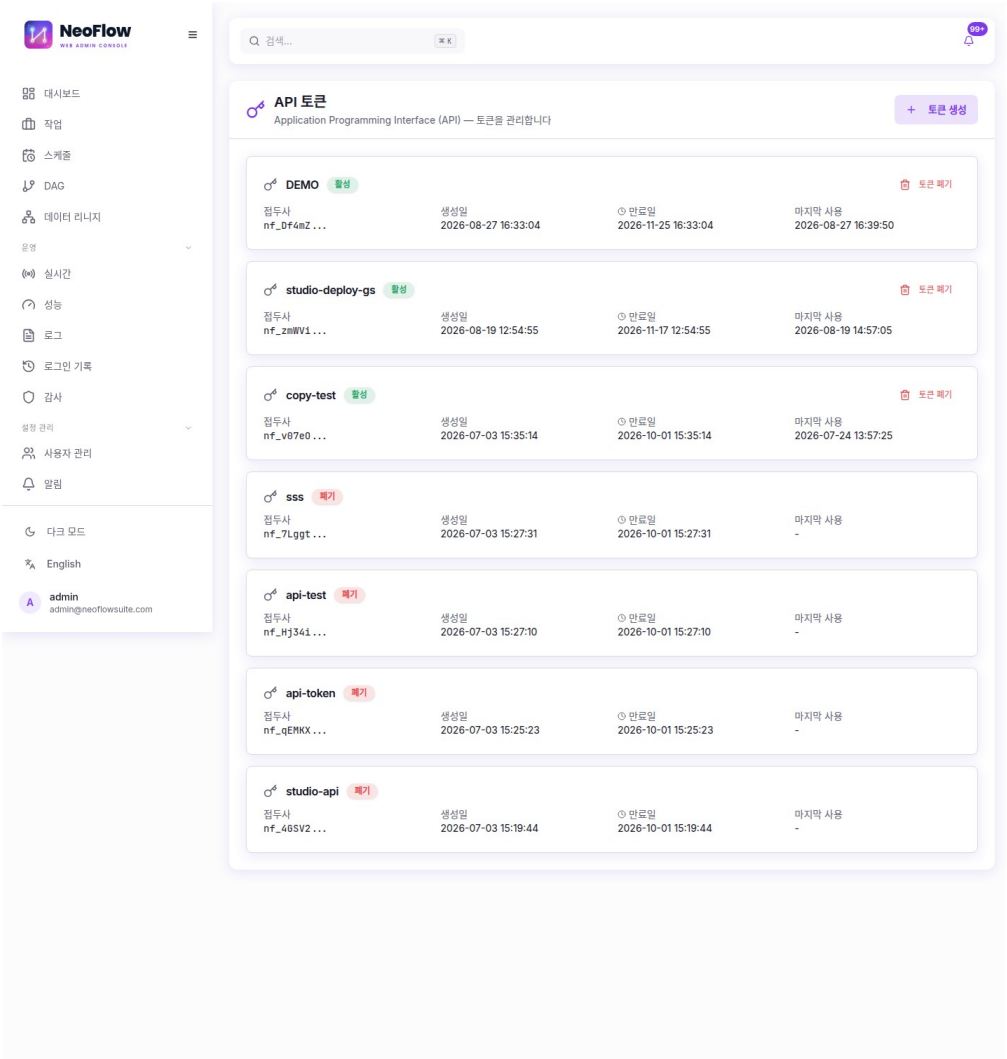
14. NeoFlow Server 연결 정보 등록

Sync 기능이 Job 을 빌드해서 보낼 NeoFlow 서버 프로파일을 먼저 등록합니다. 프로파일은 여러 개(예: `LOCAL`, `DEV`, `PROD`) 만들 수 있고, 그중 하나를 기본값으로 지정합니다. 프로파일에 넣을 API token 은 NeoFlow Server 관리 콘솔에서 먼저 발급받아야 하며(14.1), Studio 에 저장될 때 암호화됩니다.

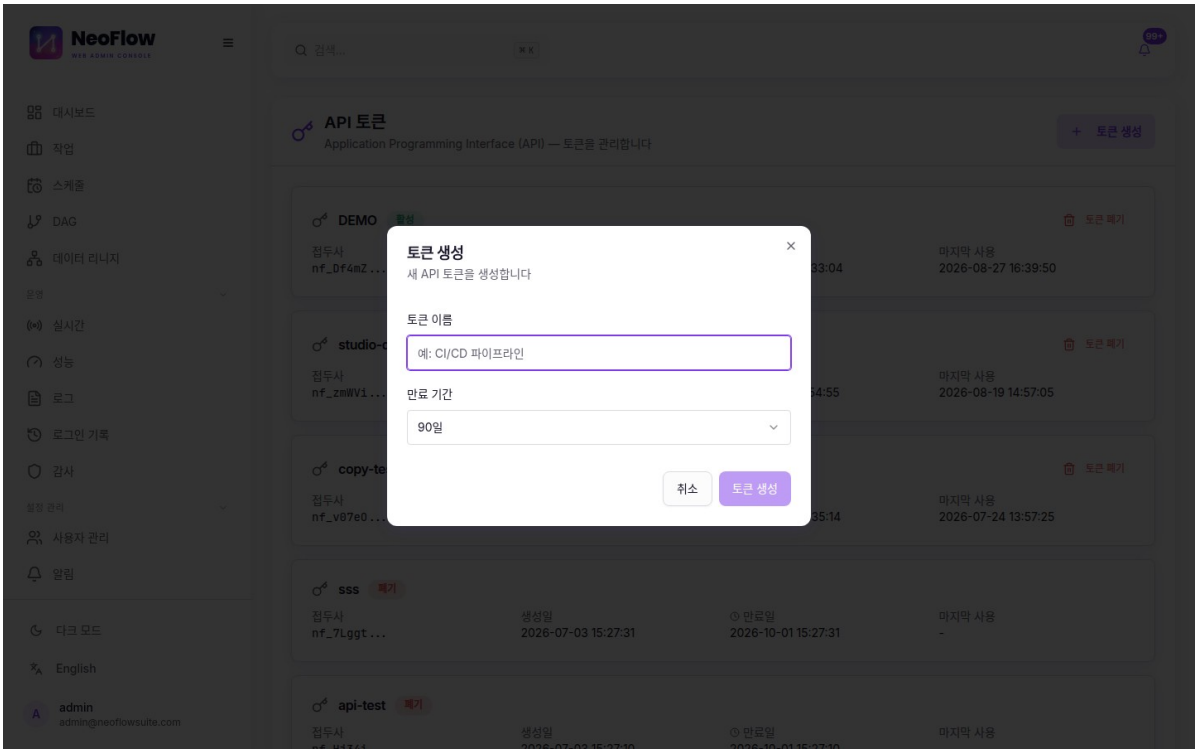
14.1 API token 발급 (NeoFlow Server)

프로파일에 입력할 API token 은 NeoFlow Server 웹 관리 콘솔에서 발급합니다. [API 토큰] 화면은 모든 사용자가 쓸 수 있고 각자 본인 토큰만 봅니다. 토큰은 발급한 사용자의 현재 역할로 동작하며, Sync 가 호출하는 동기화 API 는 ADMIN·매니저(JOB_MANAGER)만 허용하므로 Sync 용 토큰은 이 두 역할의 계정으로 발급해야 합니다.

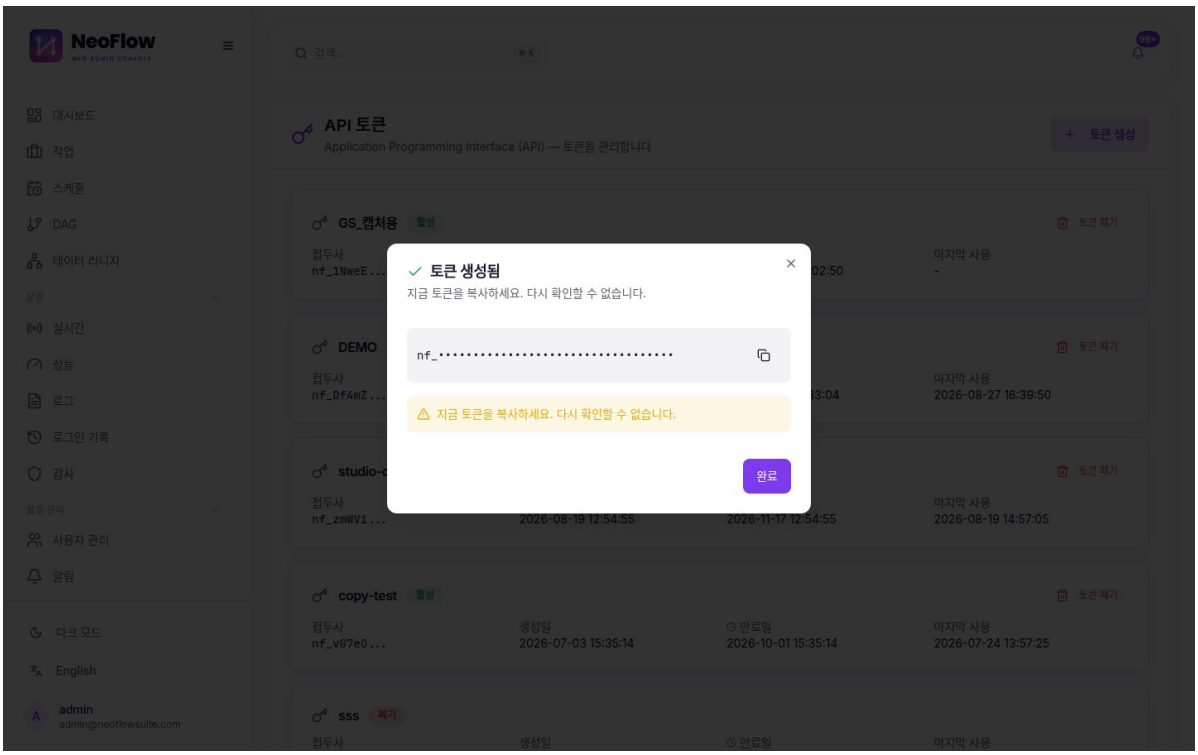
1. NeoFlow Server 웹 관리 콘솔에 ADMIN 또는 매니저 계정으로 로그인합니다. 초기 비밀번호로 처음 로그인하면 비밀번호 변경 화면이 먼저 나타나며, 비밀번호를 바꾸기 전에는 다른 화면을 쓸 수 없습니다.
2. 좌측 사이드바 설정 관리 영역의 [설정] → [API 토큰] 으로 이동합니다.
3. 화면 우측 상단의 [토큰 생성] 버튼을 누릅니다.
4. 토큰 이름(예: `NeoFlow_Studio`)과 만료 기간(30 일 / 60 일 / 90 일 / 만료 없음, 기본 90 일)을 지정하고 [토큰 생성] 을 누릅니다.
5. 토큰 생성됨 모달에 평문 토큰이 표시됩니다. 복사 버튼으로 즉시 복사한 뒤 [완료] 로 닫습니다.



◢ *NeoFlow Server 관리 콘솔 — 설정 › API 토큰(토큰 카드 목록 · 토큰 생성)*



▲ *토큰 생성 — 토큰 이름 · 만료 기간(30 일 / 60 일 / 90 일 / 만료 없음) 지정*



▲ *토큰 생성됨 — 평문 토큰은 이때 한 번만 표시됩니다(문서에서는 토큰 값 마스킹)*

> **주의** 평문 토큰은 발급 직후 한 번만 표시되며 모달을 닫으면 다시 볼 수 없습니다. 복사하지 못했다면 해당 토큰을 [토큰 폐기] 하고 새로 발급하세요.

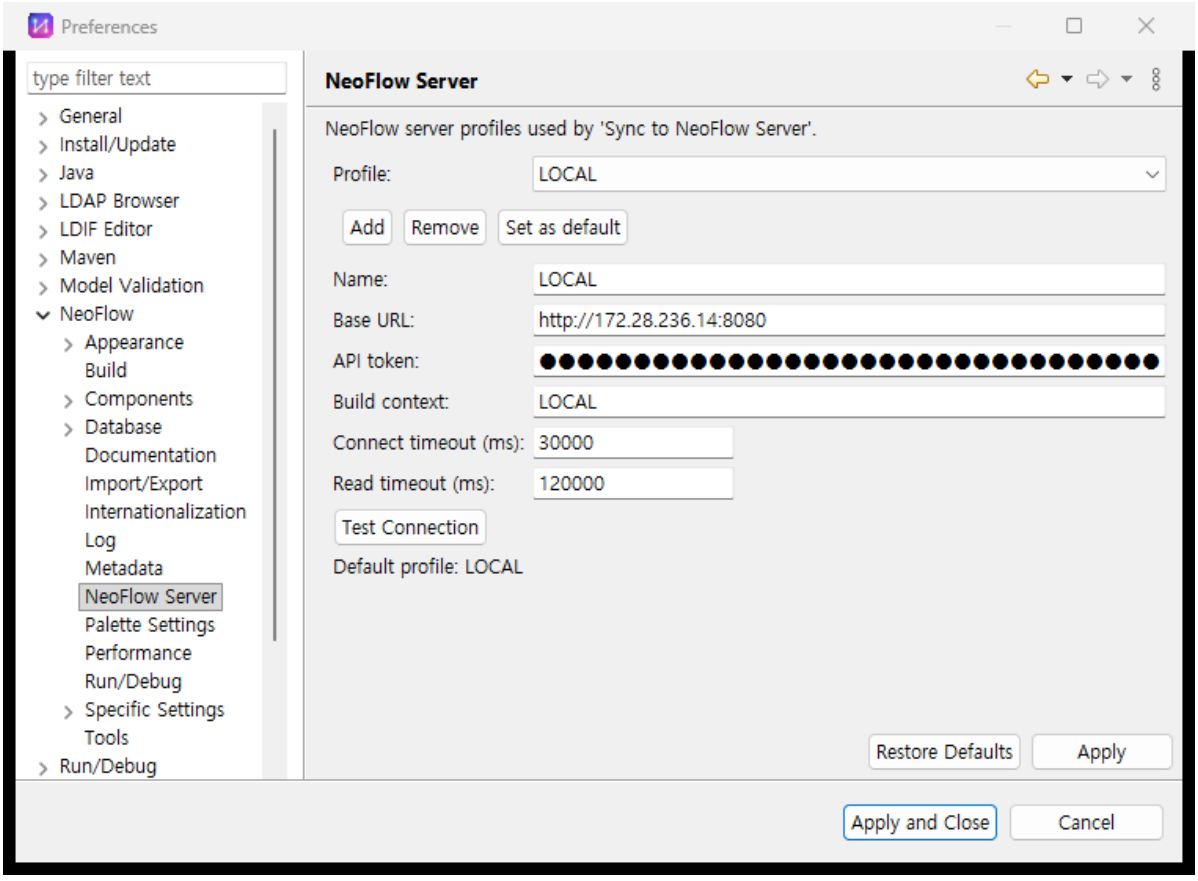
목록 화면의 토큰 카드에는 다음 정보가 표시됩니다.

항목	설명
이름	토큰 식별용 이름
상태	활성 / 만료 / 폐기
접두사	토큰 앞부분(nf_ + 일부). 전체 토큰 값은 표시되지 않습니다.
생성일 / 만료일	발급 일시 및 만료 일시
마지막 사용	해당 토큰으로 마지막 인증한 일시(최대 5 분 늦게 반영, 미사용 시 - 로 표시)

> **참고** 발급한 토큰 값은 14.4 프로파일별 입력 항목의 `API token` 란에 붙여넣습니다.

14.2 Preferences 페이지 열기

상단 메뉴 [Window] → [Preferences] 를 열고, 왼쪽 트리에서 [NeoFlow] → [NeoFlow Server] 를 선택합니다.



▲ *Preferences → NeoFlow → NeoFlow Server — 프로파일 관리 · 입력 항목 · 전역 타임아웃 · Test Connection*

14.3 프로파일 관리

페이지 상단에서 프로파일을 추가·삭제하고 기본값을 지정합니다. 하단에는 `Default profile: <이름>` 으로 현재 기본 프로파일이 표시됩니다.

컨트롤	설명
Profile (드롭다운)	등록된 프로파일 목록에서 선택

컨트롤	설명
Add	새 프로파일 추가 (PROFILE_1, PROFILE_2... 자동 명명)
Remove	선택한 프로파일 삭제
Set as default	선택한 프로파일을 기본값으로 지정 (Sync 마법사가 기본 선택)

14.4 프로파일별 입력 항목

각 프로파일에는 서버 주소와 인증 정보, 빌드 환경을 입력합니다.

필드	설명	예시
Name	프로파일 이름	`PROD`
Base URL	NeoFlow 서버 주소 (스킴+호스트, 기본 포트가 아니면 포트 포함)	`https://neoflow.example.com`
API token	서버 인증용 Bearer 토큰 (마스킹·암호화 저장)	(서버에서 발급)
Build context	빌드 시 사용할 context 이름. 비우면 프로파일 이름 사용 (아래 참고)	`Default`

타임아웃은 모든 프로파일에 공통 적용되는 전역 설정입니다.

필드	기본값	설명
Connect timeout (ms)	`5000`	서버 연결 타임아웃
Read timeout (ms)	`30000`	응답 읽기 타임아웃

값을 입력·수정한 뒤에는 [Apply] 또는 [Apply and Close] 를 눌러야 저장됩니다.

> **참고** Build context 는 Sync 할 Job 에 이미 있는 context 이름으로 지정하세요. Job 에 없는 이름이면 Sync 가 그 Job 의 첫 번째 context 를 복사해 같은 이름의 환경을 새로 만들고 기본 context 로 지정합니다. 그 결과 PC 경로처럼 서버에서 쓸 수 없는 값이 서버 실행에 그대로 들어갈 수 있습니다.

14.5 연결 테스트

Base URL 과 API token 을 모두 입력한 뒤 Test Connection 버튼을 누르면 서버 도달 여부와 토큰 권한을 확인합니다.

상황	메시지 (제목 Test Connection 창, 화면 표시 그대로)
성공	`Success: server reachable and token authorized.`
Base URL·API token 미입력	`Enter base URL and API token first.`
토큰 만료·폐기·오입력, 발급 계정 비활성	`Invalid or expired API token`
토큰 발급 계정이 ADMIN·매니저가 아님	`Token lacks the ADMIN/JOB_MANAGER role`
서버에 연결할 수 없음(주소·방화벽·	`Cannot reach server: <원인>`

상황	메시지 (제목 Test Connection 창, 화면 표시 그대로)
인증서)	

> **주의** 서버가 자체 서명 인증서(self-signed)를 쓰면 `Cannot reach server:` 뒤에 `PKIX path building failed` 가 표시됩니다. Studio 는 실행에 쓰는 JDK 의 인증서 저장소(`cacerts`)로만 서버 인증서를 확인하므로, 서버 관리자에게 받은 인증서 파일(예: `tls.crt`)을 이 저장소에 등록한 뒤 Studio 를 다시 시작해야 합니다. Sync·Deploy 도 같은 이유로 실패합니다.

관리자 권한 명령 프롬프트에서 아래처럼 등록합니다. `JAVA_HOME` 은 Studio ini 의 `-vm` 에 지정한 JDK 폴더로, 인증서 경로는 실제 위치로 바꿉니다. 첫 명령이 보여 주는 SHA-256 지문을 서버 관리자에게 받은 값과 대조한 뒤 등록하세요.

```
set JAVA_HOME=C:\Program Files\Java\jdk-11
"%JAVA_HOME%\bin\keytool" -printcert -file C:\neoflow\tls.crt
"%JAVA_HOME%\bin\keytool" -importcert -alias neoflow-server ^
    -file C:\neoflow\tls.crt ^
    -keystore "%JAVA_HOME%\lib\security\cacerts" -storepass changeit -noprompt
```

14.6 저장 및 Job 으로의 주입 원리

프로파일은 NeoFlow Studio 환경설정(Preferences)에 저장되며 API token 은 암호화되어 보관됩니다. Sync 실행 시 선택한 프로파일의 정보가 Job 의 context 변수로 주입됩니다(프로파일의 `Build context` 환경에 기록).

context 변수	값	Job 에서 참조
`NEOFLOW_BASE_URL`	서버 Base URL	`context.NEOFLOW_BASE_URL`
`NEOFLOW_API_TOKEN`	API 토큰	`context.NEOFLOW_API_TOKEN`
`NEOFLOW_JOB_ID`	Sync 로 발급된 NeoFlow Job ID	`context.NEOFLOW_JOB_ID`

> **참고** 이 변수들은 `tNeoFlowPre` 의 Advanced settings(`BASE_URL`·`API_TOKEN`·`JOB_ID`)에 연결되며, Studio 에서 Job 을 직접 실행(테스트 실행)할 때만 쓰입니다. 사용자가 직접 입력할 필요는 없으며, Sync 가 자동으로 채워 넣습니다. NeoFlow Server 에서 실행될 때는 서버가 환경변수로 넣어 준 값이 우선하므로 이 context 값은 쓰이지 않습니다.

> **참고** `tNeoFlowPost` 에는 `BASE_URL`·`API_TOKEN`·`JOB_ID` 설정 항목이 없습니다. NeoFlow Server 에서 실행될 때는 서버가 실행마다 발급해 환경변수로 넣어 주는 실행 토큰(`NEOFLOW_TOKEN`)과 실행 ID(`EXECUTION_ID`)로 워터마크를 커밋하고, Studio 테스트 실행에서는 같은 실행에서 `tNeoFlowPre` 가 연 세션을 이어 씁니다.

주의 Studio 의 테스트 실행(로컬 실행)에서 서버 값을 받는 것은 개발·검증 서버에서만 됩니다. 로컬 실행 때 `tNeoFlowPre` 가 서버에 개발용 실행 토큰(`/api/executions/dev-token`)을 요청하는데, 운영(prod) 프로파일로 설치한 서버는 이 요청을 403 으로 거부합니다. 이때 Job 자체는 실행되지만 `tNeoFlowPre` 는 BizDate 값을 채우지 않고 통과하고, `tNeoFlowPost` 는 커밋을 건너뛰며, `NeoFlowJobApi` 를 호출하는 Job 은 실패합니다. `NeoFlowContext` 만 쓰는 Job 은 Context 탭 기본값으로 실행됩니다. 운영 서버에서는 Job 을 Sync·활성화한 뒤 NeoFlow Server 웹 콘솔에서 실행하세요.

15. NeoFlow 컴포넌트

NeoFlow 전용 컴포넌트는 팔레트의 NeoFlow 폴더에 있습니다. BizDate 처리용 두 컴포넌트와 병렬 실행용 한 컴포넌트로 구성됩니다.



▲ 팔레트 NeoFlow 패밀리 — BizDate(tNeoFlowPost·tNeoFlowPre) · Process(tNeoFlowThread)

컴포넌트	팔레트 위치	용도
tNeoFlowPre	NeoFlow/BizDate	서버에서 처리 기준일(BizDate) 구간을 가져옴
tNeoFlowPost	NeoFlow/BizDate	처리 완료된 BizDate 워터마크를 서버에 커밋
tNeoFlowThread	NeoFlow/Process	연결된 서브잡들을 병렬 스레드로 실행(시차 기동)

> **참고** 세 컴포넌트 모두 데이터 플로우(행) 연결이 없는 트리거 전용 시작(STARTABLE) 컴포넌트입니다. OnSubjobOk, OnComponentOk, RunIf 등 트리거 링크로만 연결합니다.

배치 규칙 — tPrejob / tPostjob 과의 순서

BizDate 컴포넌트 두 개는 Job 의 사전·사후 구간에 각각 배치합니다. 순서가 어긋나면 BizDate 구간이 채워지기 전에 본 처리가 실행되거나, 처리가 끝나기 전에 워터마크가 커밋될 수 있습니다.

- tNeoFlowPre 는 tPrejob 컴포넌트 뒤에 배치합니다 — Job 의 본 처리보다 먼저 실행되어 BizDate 구간을 globalMap 에 채워야 합니다.
- tNeoFlowPost 는 tPostjob 컴포넌트 뒤에 배치합니다 — 본 처리가 모두 끝난 뒤 워터마크를 커밋해야 합니다.
- 두 연결 모두 OnComponentOk 트리거 링크를 사용합니다.

```
tPrejob   --OnComponentOk-->  tNeoFlowPre   --OnComponentOk-->  [ main subjob ]
tPostjob  --OnComponentOk-->  tNeoFlowPost  --OnComponentOk-->  [ close / cleanup ]
```

> **참고** tPrejob / tPostjob 은 Talend Open Studio 표준 컴포넌트로, 각각 Job 시작 전과 종료 후에 실행되는 구간을 정의합니다. NeoFlow BizDate 컴포넌트는 이 구간의 첫 번째 컴포넌트로 두는 것을 권장합니다.

15.1 tNeoFlowPre — BizDate 구간 가져오기

역할

NeoFlow 서버에서 이번 실행이 처리해야 할 기준일 구간(시작/종료), 로드 타입, 실행 ID 를 가져와 globalMap 에 채워 넣습니다. Job 파라미터는 context 변수에 넣지 않습니다(파라미터는 16 장). tPrejob 컴포넌트 뒤에 배치하여 Job 의 본 처리보다 먼저 실행되게 합니다. 이후 SQL 컴포넌트 등에서 시작/종료 일자를 WHERE 조건으로 사용합니다.

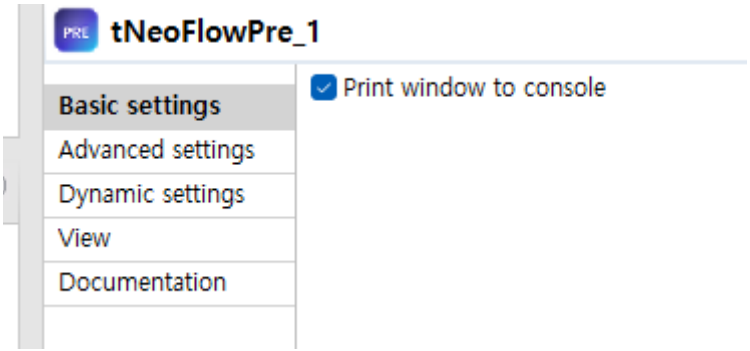
> **참고** Never-fail 원칙 — 서버에 연결되지 않거나 바인딩이 없어도 Job 을 실패시키지 않습니다. 경고만 로그로 남기고 진행합니다(이 경우 BizDate 값이 채워지지 않음).

동작 우선순위

1. 서버 오케스트레이션 실행 — 환경변수/시스템 프로퍼티 `NEOFLOW_BASE_URL`·`NEOFLOW_TOKEN`·`EXECUTION_ID` 가 모두 있으면 그 실행 컨텍스트를 사용합니다(아래 Advanced 파라미터 무시).
2. Studio 테스트 실행 — 위 환경이 없으면 컴포넌트에 설정된 Base URL/Token/Job ID(보통 `context.NEOFLOW_*`)로 서버에서 값을 가져옵니다. 이 경로는 개발·검증 서버에서만 동작합니다. 운영(prod) 서버는 개발 토큰을 거부하므로 값을 받지 못하고 다음 단계로 넘어갑니다(14.6).
3. 둘 다 없으면 경고만 남기고 통과합니다.

파라미터 — Basic settings

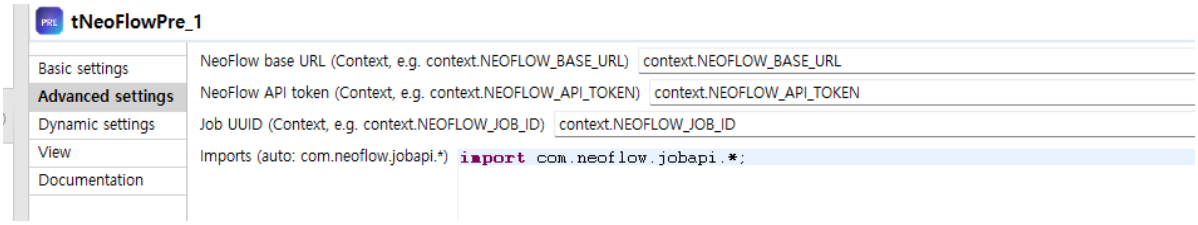
파라미터	라벨	기본값	설명
`PRINT_LOG`	Print window to console	`true`	체크 시 가져온 구간을 콘솔에 출력



▲ *tNeoFlowPre Basic settings — Print window to console 체크박스(기본 체크)*

파라미터 — Advanced settings

파라미터	라벨	기본값	설명
`BASE_URL`	`NeoFlow base URL (Context, e.g. context.NEOFLOW_BASE_URL)`	(빈 값)	Studio 테스트용 서버 URL. Sync/Deploy 시 자동 입력, 서버 실행 시 비워둠
`API_TOKEN`	`NeoFlow API token (Context, e.g. context.NEOFLOW_API_TOKEN)`	(빈 값)	Studio 테스트용 API 토큰. 자동 입력
`JOB_ID`	`Job UUID (Context, e.g. context.NEOFLOW_JOB_ID)`	(빈 값)	Studio 테스트용 Job UUID. 자동 입력
`IMPORT`	`Imports (auto: com.neoflow.jobapi.*)`	(자동)	import com.neoflow.jobapi.* — 자동 생성, 수정 불필요



▲ *tNeoFlowPre Advanced settings — 세 값은 context.NEOFLOW_* 로 바인딩(Sync 가 자동 채움)*

> **참고** Advanced 의 세 값은 일반적으로 직접 입력하지 않습니다. Sync 가 `context.NEOFLOW_*` 로 자동 채웁니다.

반환값 (globalMap)

컴포넌트 실행 후(AFTER) 사용할 수 있는 반환값입니다. `cid` 는 Job 내 컴포넌트 고유 이름입니다.

반환명	타입	globalMap 키
START_DATE	Object(LocalDateTime)	<cid>_START_DATE
END_DATE	Object(LocalDateTime)	<cid>_END_DATE
LOAD_TYPE	String	<cid>_LOAD_TYPE
EXECUTION_ID	Object	<cid>_EXECUTION_ID

다운스트림에서 바로 쓰는 고정 globalMap 키:

키	타입	설명
nf_endDate	LocalDateTime	구간 종료일 (tNeoFlowPost 가 읽음)
nf_startStr	String (yyyy-MM-dd HH:mm:ss)	포맷된 시작일
nf_endStr	String (yyyy-MM-dd HH:mm:ss)	포맷된 종료일

사용 예 — SQL 컴포넌트의 쿼리에서 구간 사용

```
SELECT * FROM sales
WHERE updated_at ≥ '" + (String)globalMap.get("nf_startStr") + "'
  AND updated_at <  '" + (String)globalMap.get("nf_endStr") + "'
```

15.2 tNeoFlowPost — BizDate 워터마크 커밋

역할

`tNeoFlowPre` 가 가져온 구간의 종료일(`nf_endDate`)을 서버에 커밋하여, 해당 로드 구간이 정상 처리되었음을 기록합니다. `tPostjob` 컴포넌트 뒤에 배치하여 데이터 적재가 모두 끝난 뒤 실행되게 합니다.

> **참고** Never-fail 원칙 — 커밋 실패, `nf_endDate` 누락, 예외 발생 시에도 Job 을 실패시키지 않고 로그만 남기고 통과합니다.

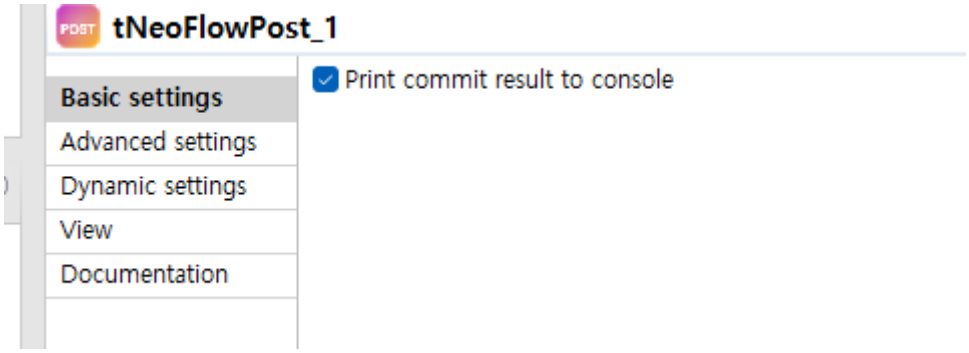
> **주의** 개발·검증 서버에 연결해 Studio 에서 실행하면 `tNeoFlowPost` 가 그 서버 Job 의 실제 워터마크를 커밋합니다. 다음 서버 실행은 이 시각부터 구간을 받으므로, 테스트 뒤 필요하면 NeoFlow Server 의 Job 상세 화면에서 워터마크를 되돌리세요.

전제 조건

- 같은 Job 에 `tNeoFlowPre` 가 먼저 실행되어 있어야 합니다. `tNeoFlowPost` 는 `tNeoFlowPre` 가 채운 고정 키 `nf_endDate` 를 읽습니다.
- `tNeoFlowPre` 가 실행되지 않았거나 바인딩이 없으면 `nf_endDate` 가 없어 커밋을 건너뜁니다(로그: "did tNeoFlowPre run?").
- 배치 순서 — `tNeoFlowPre` 는 `tPrejob` 뒤, `tNeoFlowPost` 는 `tPostjob` 뒤에 두어야 두 컴포넌트의 실행 순서가 보장됩니다.

파라미터 — Basic settings

파라미터	라벨	기본값	설명
`PRINT_LOG`	Print commit result to console	`true`	체크 시 커밋 결과(성공/스킵 사유)를 콘솔에 출력



▲ *tNeoFlowPost Basic settings — Print commit result to console(기본 체크). Advanced settings 항목 없음*

반환값 (globalMap)

반환명	타입	globalMap 키	설명
`COMMITTED`	Boolean	`<cid>_COMMITTED`	커밋 성공 시 true, 스킵/실패 시 false
`SKIP_REASON`	String	`<cid>_SKIP_REASON`	스킵 사유. NON_MONOTONIC = 종료 시각이 기존 워터마크보다 늦지 않음, BACKFILL = 소급 적재 실행. 정상 커밋·오류 시 빈 값

표준 배치 패턴

15 장 배치 규칙과 같습니다. `tNeoFlowPost` 는 본 처리 서브잡에 직접 잇지 않고 `tPostjob` 에 `OnComponentOk` 로 연결합니다.

```
tPrejob   --OnComponentOk-->  tNeoFlowPre   --OnComponentOk-->  [ main subjob ]
tPostjob  --OnComponentOk-->  tNeoFlowPost  --OnComponentOk-->  [ close / cleanup ]
```

15.3 tNeoFlowThread — 서브잡 병렬 실행(시차 기동)

역할

이 컴포넌트에 연결된 여러 서브잡을 각각 별도 스레드로 병렬 실행합니다. 스레드를 동시에 띄우지 않고 설정한 초만큼 시차(ramp-up)를 두고 순차적으로 기동할 수 있습니다(DB 커넥션 폭주 방지 등). 모든 스레드가 끝날 때까지 Job 은 대기합니다.

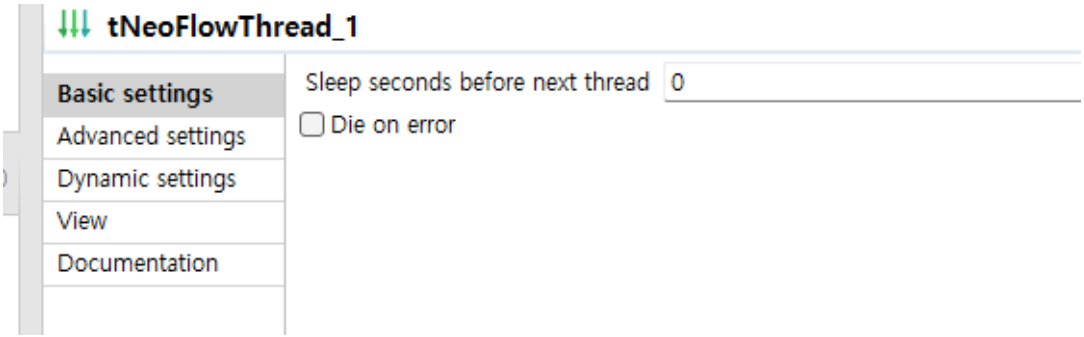
> **참고** `tNeoFlowPre/tNeoFlowPost` 와 달리 에러 처리는 옵션(`DIE_ON_ERROR`)으로 제어합니다. never-fail 이 아닙니다.

연결 방법 — THREAD_PROCESS 링크

- `tNeoFlowThread` 에서 병렬로 돌릴 각 서브잡 시작 컴포넌트로 녹색 점선 트리거(`THREAD_PROCESS`)를 연결합니다.
- 최대 100 개까지 연결 가능하며, 연결 1 개당 스레드 1 개가 생성됩니다.
- 연결한 순서대로 스레드가 기동되고, 각 기동 사이에 시차가 적용됩니다(첫 스레드 제외).

파라미터 — Basic settings

파라미터	라벨	기본값	설명
`SLEEP_SECOND`	Sleep seconds before next thread	`0`	다음 스레드 기동 전 대기(초). 0 이면 즉시 모두 기동. context 변수도 가능
`DIE_ON_ERROR`	Die on error	`false`	체크 시 스레드 중 하나라도 예외가 나면 Job 실패. 해제 시 로그만 남기고 계속



▲ *tNeoFlowThread Basic settings — Sleep seconds(기본 0) · Die on error(기본 해제)*

동작 순서

1. `THREAD_PROCESS` 로 연결된 각 서브잡을 별도 스레드로 생성(스레드명 `<cid>_thread_<index>`).
2. 두 번째 스레드부터 기동 전 `SLEEP_SECOND` 초 대기(시차).
3. 모든 스레드가 끝날 때까지 메인 Job 이 대기(폴링).
4. `DIE_ON_ERROR=true` 면 첫 예외 발생 시 Job 실패, false 면 스택트레이스만 출력하고 완료.
5. 완료 시 콘솔: `[<cid>] Multi-thread process complete (N threads).`

주의

- 모든 스레드가 같은 `globalMap` 을 공유합니다. 서브잡이 공유 상태를 쓴다면 thread-safe(예: `ConcurrentHashMap`) 처리가 필요합니다.
- 반환값(RETURNS) 없음, 별도 jar 의존성 없음.

사용 예

```
tNeoFlowThread
    ├─ THREAD_PROCESS → tDBInput_A → tFileOutput_A
    ├─ THREAD_PROCESS → tDBInput_B → tFileOutput_B
    └─ THREAD_PROCESS → tDBInput_C → tFileOutput_C
```

SLEEP_SECOND=2 로 두면 A → (2 초) → B → (2 초) → C 순으로 기동되어 동시 부하를 분산합니다.

16. Job 파라미터 받기 (NeoFlowContext · NeoFlowJobApi)

NeoFlow Server 의 Job 파라미터와 공통 설정(Global Properties) 값을 Job 안에서 받아 쓰는 방법을 다룹니다. context 변수로 받으려면 `NeoFlowContext` routine(16.2~16.4)을, 컴포넌트 입력 칸에서 바로 조회하려면 `NeoFlowJobApi`(16.5)를 씁니다. 같은 Job 을 기준일·대상 시스템만 바꿔 재사용할 때 필요합니다.

16.1 파라미터가 Job 에 도달하는 경로

NeoFlow 서버는 실행 시 확정된 파라미터를 두 경로로 Job 프로세스에 넘깁니다.

경로	형식	전달 대상
실행 인자	`--context_param <키>=<값>`	비-시크릿 파라미터 (공통 설정의 비밀 속성은 제외)
환경변수	`PARAM_<대문자 키>=<값>`	전체 파라미터 (시크릿 포함)

시크릿을 실행 인자로 넘기지 않는 이유는 커맨드라인이 같은 호스트의 다른 프로세스에서 `/proc/<pid>/cmdline` 으로 읽히기 때문입니다. 즉 **시크릿 파라미터는 환경변수 경로로만 도달**합니다.

시크릿은 키 이름에 `PASSWORD`·`SECRET`·`TOKEN`·`KEY` 가 들어 있는지(대소문자 무시, 단어 일부 포함)로 정해지며, Job 파라미터·공통 설정·실행할 때 입력한 값 모두에 적용됩니다(예: `SORT_KEY` 도 시크릿입니다). 여기에 더해 공통 설정(Global Properties)에서 비밀로 지정한 키도 이름과 관계없이 시크릿으로 다룹니다. 그래서 `DB_CONN_STR` 처럼 이 단어가 없는 비밀 속성도 환경변수로만 도착하며, 실행 인자에 기대는 Job 은 아래 routine 을 두지 않으면 값을 받지 못합니다.

실행 인자 경로는 Talend Open Studio 규칙에 따라 **파라미터 키와 context 변수명이 정확히 일치**해야 바인딩됩니다. 대소문자만 달라도 매칭되지 않습니다. 아래 `NeoFlowContext` routine 이 이 두 가지 빈틈(시크릿·대소문자)을 함께 메웁니다.

16.2 NeoFlowContext routine (기본 포함)

`NeoFlowContext` 는 NeoFlow 가 주입한 `PARAM_*` 환경변수를 같은 이름의 context 변수에 자동으로 매핑하는 routine 입니다. NeoFlow Studio 에 시스템 routine 으로 포함되어 있어 별도로 등록할 필요가 없습니다. Repository 뷰의 [Code] → [Routines] → [system] 폴더에서 확인할 수 있으며, 모든 Job 에서 import 없이 `NeoFlowContext.apply(context)` 로 호출합니다.

> **주의** 같은 이름(`NeoFlowContext`)의 사용자 routine 을 만들지 마세요. 시스템 routine 과 클래스 이름이 겹쳐 Job 빌드가 꼬일 수 있습니다. 이전에 매뉴얼 절차대로 사용자 routine 을 만들어 두었다면 삭제하고 시스템 routine 을 쓰세요.

16.3 Job 에서 사용

`tPrejob` 뒤에 연결한 `tJava` 에 다음 한 줄을 둡니다. 이 한 줄이 모든 파라미터를 처리하므로, 파라미터가 늘어나도 Job 을 고칠 필요가 없습니다.

```
NeoFlowContext.apply(context);
```

이후 Job 어디서든 `context.GS_DATA_DIR` 처럼 평소대로 쓰면 됩니다. Studio 에서 로컬 실행할 때는 `PARAM_*` 환경변수가 없으므로 Context 탭에 넣어 둔 기본값이 그대로 유지됩니다. 따라서 로컬용·서버용 분기 코드를 둘 필요가 없습니다.

routine 을 쓰지 않고 직접 읽으려면 파라미터마다 아래 보일러플레이트를 Job 에 넣어야 합니다.

```
String gsDataDir = System.getenv("PARAM_GS_DATA_DIR");
if (gsDataDir != null && !gsDataDir.isEmpty()) {
    context.GS_DATA_DIR = gsDataDir;
}
```

> **참고** 이 직접 읽기 방식도 정상 동작하며, `--context_param` 을 지원하지 않는 구버전 NeoFlow 서버에서도 동일하게 쓸 수 있습니다. 다만 파라미터 수만큼 코드가 늘어나므로 routine 사용을 권장합니다.

16.4 동작 규칙과 제약

routine 은 다음 규칙으로 동작합니다.

항목	동작
매칭	PARAM_ 접두어를 뗀 이름과 context 변수명을 대소문자 무시로 비교(NeoFlow 가 키를 대문자화하므로)
덮어쓰기 범위	NeoFlow 가 실제로 주입한 변수만. 나머지는 기존 값 유지
지원 타입	String · Integer · Long · Boolean · Double · Float · BigDecimal · Character (대응 primitive 포함)
미지원 타입	건드리지 않음 — 실행 인자가 세팅한 값이나 Context 탭 기본값이 그대로 보존됨
파싱 실패	해당 변수만 건너뛰고 경고를 남긴 뒤 Job 은 계속 진행
Properties 동기화	context.getProperty("X") 로 읽는 컴포넌트를 위해 내부 map 도 함께 갱신
로깅	적용된 변수명만 출력하고 값은 출력하지 않음

> **주의** 파라미터 값을 `System.out.println` 으로 출력하지 마세요. Job 의 표준출력은 NeoFlow 실행 로그로 그대로 적재되며 이 경로에는 서버측 시크릿 마스킹이 적용되지 않으므로, 값을 찍으면 패스워드 파라미터가 평문으로 남습니다.

제약

- **Date 타입 context 변수는 매핑되지 않습니다.** 올바른 파싱은 Studio 에 선언된 변수별 포맷 패턴에 달려 있는데 routine 은 그 패턴을 알 수 없습니다. 문자열로 선언해 Job 안에서 파싱하거나, 서버의 실행 인자 경로에 위임하세요.
- 적재 유형은 `PARAM_LOAD_TYPE` 이 아니라 `LOAD_TYPE` 으로 주입되므로 이 routine 의 대상이 아닙니다. 필요하면 `System.getenv("LOAD_TYPE")` 을 직접 읽습니다.
- 파라미터 키와 context 변수명이 이름 자체로 다르면 어느 경로로도 값이 전달되지 않습니다. routine 이 흡수해 주는 것은 대소문자 차이뿐입니다.

> **참고** 서버가 실행 인자(`--context_param`)로도 값을 보내지만 routine 호출은 계속 두는 것이 좋습니다. 시크릿은 환경변수 경로로만 전달되고, 실행 인자를 지원하지 않는 이전 서버에서도 routine 은 같은 방식으로 동작하기 때문입니다. 실행 인자 주입은 서버 설정 `execution.context-param.enabled`(기본 `true`)로 끌 수 있으며, 꺼도 환경변수 경로는 유지되므로 routine 을 쓰는 Job 은 영향을 받지 않습니다.

16.5 NeoFlowJobApi 로 서버 값 조회

`NeoFlowJobApi`(패키지 `com.neoflow.jobapi`)는 Job 안에서 NeoFlow Server 의 실행 컨텍스트를 조회하는 정적 메서드 모음입니다. NeoFlow 컴포넌트와 함께 배포되는 `neoflow-job-api-1.0.0.jar` 에 들어 있고, `tNeoFlowPre` 가 Job 에 `import com.neoflow.jobapi.*;` 를 자동으로 넣으므로 `tNeoFlowPre` 가 있는 Job 에서는 클래스 이름만으로 호출할 수 있습니다.

메서드	반환 타입	설명
`getParameter(String key)`	`Object`	이번 실행의 파라미터 값 하나. 없으면 null. 호출할 때마다 서버에 한 번 조회합니다.
`pullRuntimeContext()`	`RuntimeContext`	BizDate 구간·적재 유형·실행 ID·파라미터 전체 (tNeoFlowPre 가 내부에서 사용)
`commitBizDate(LocalDateTime end)`	`CommitResult`	BizDate 워터마크 커밋 (tNeoFlowPost 가 내부에서 사용)
`commitNow()`	`CommitResult`	현재 시각으로 워터마크 커밋
`getEnv()`	`Map<String, String>`	연결 설정 확인용. 토큰 값은 ********로 가려서 돌려줍니다.

사용 예 — DB 접속 정보를 공통 설정에서 받기

컴포넌트의 입력 칸에 Java 식으로 적습니다. 값은 문자열로 변환해 씁니다.

```
Host     : (String)NeoFlowJobApi.getParameter("MYSQL_HOST")
Port     : (String)NeoFlowJobApi.getParameter("MYSQL_PORT")
Password : (String)NeoFlowJobApi.getParameter("MYSQL_PASSWORD")
```

공통 설정 값은 문자열로 돌아옵니다. 실행할 때 입력한 값에 숫자를 넣으면 숫자형(`Long`)으로 돌아오므로, 숫자가 올 수 있는 파라미터는 `String.valueOf(NeoFlowJobApi.getParameter("키"))` 로 변환하면 안전합니다.

값의 출처

- NeoFlow Server 에서 실행될 때 — 그 실행의 파라미터를 서버가 발급한 실행 토큰으로 조회합니다. 값은 실행 시 입력 > Job 파라미터 기본값 > 공통 설정(활성 프로파일) 순으로 정해지며, 비밀 속성도 평문으로 받습니다.
- Studio 테스트 실행 — 같은 실행에서 `tNeoFlowPre` 가 연 개발용 세션으로 서버 기본값(Job 파라미터 기본값 > 공통 설정)을 조회합니다. 개발·검증 서버에서만 동작하며, 운영(prod) 서버에서는 세션이 열리지 않아 호출이 예외로 끝나고 Job 이 실패합니다(14.6).

> **참고** `NeoFlowContext` 와의 차이 — `NeoFlowContext` 는 서버가 넣어 준 `PARAM_*` 환경변수를 context 변수에 옮길 뿐 서버를 호출하지 않으므로, 로컬 실행에서는 Context 탭 기본값이 그대로 남습니다. `NeoFlowJobApi.getParameter()` 는 context 변수 없이 서버에서 직접 값을 받지만, 서버 연결이 필요하고 `tNeoFlowPre` 가 있는 Job 에서만 쓸 수 있습니다.

> **주의** `getParameter()` 로 받은 값을 `System.out.println` 등으로 출력하지 마세요. 비밀 속성도 평문으로 돌아오며, Job 의 표준출력은 실행 로그에 그대로 남습니다.

17. Sync to NeoFlow Server

작성한 Job 을 빌드하여 NeoFlow 서버로 메타데이터 동기화 + 빌드 산출물 업로드까지 한 번에 수행합니다.

> **주의** 전제 조건 — 유효한 License(3 장)와 NeoFlow Server 프로파일(14 장)이 등록되어 있어야 합니다. License 가 유효하지 않으면 메뉴 실행 시 에러 다이얼로그가 뜨고 마법사가 열리지 않습니다.

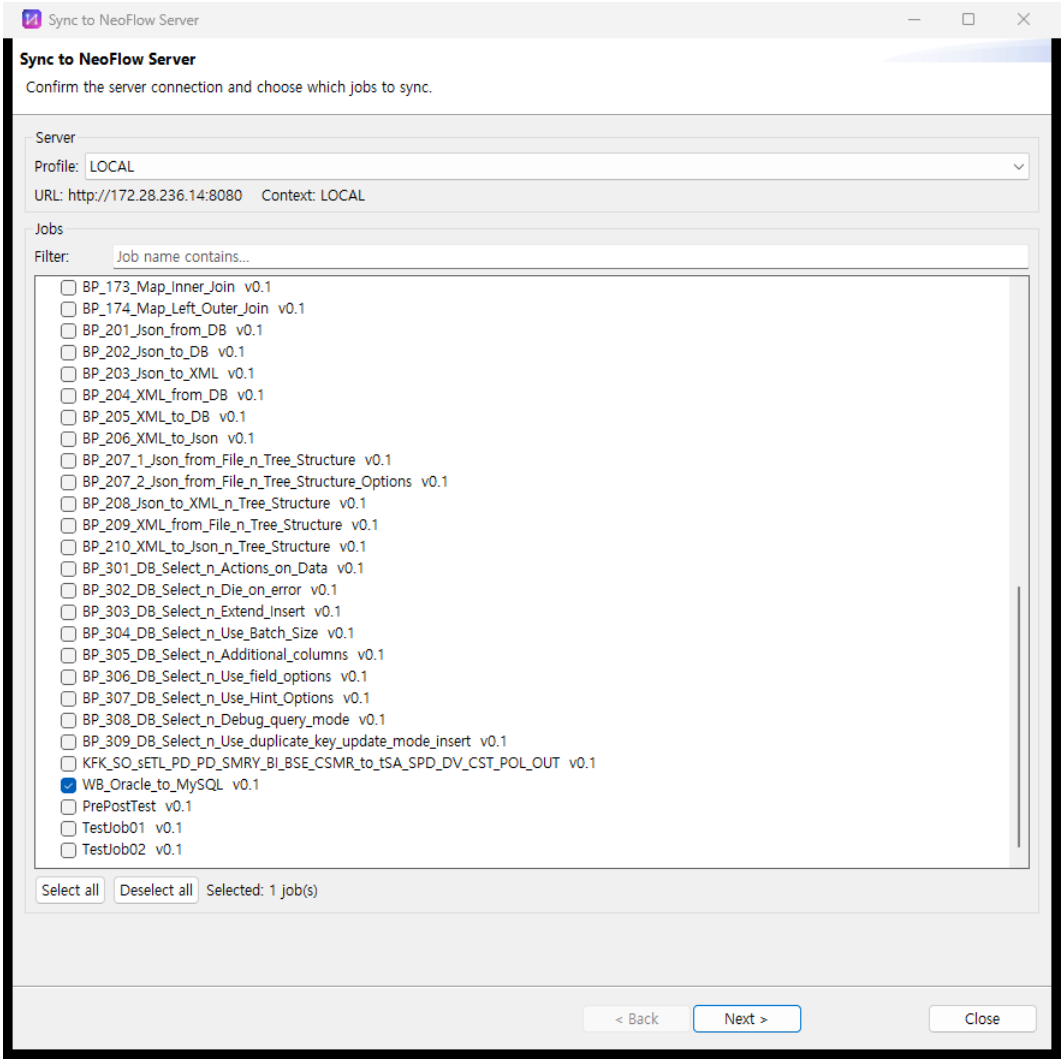
17.1 실행

Repository 뷰에서 Job(또는 Job 이 든 폴더)을 하나 이상 선택한 뒤 우클릭 → [Sync to NeoFlow Server...] 를 실행합니다.

17.2 마법사 단계

Page 1 — 서버/잡 선택

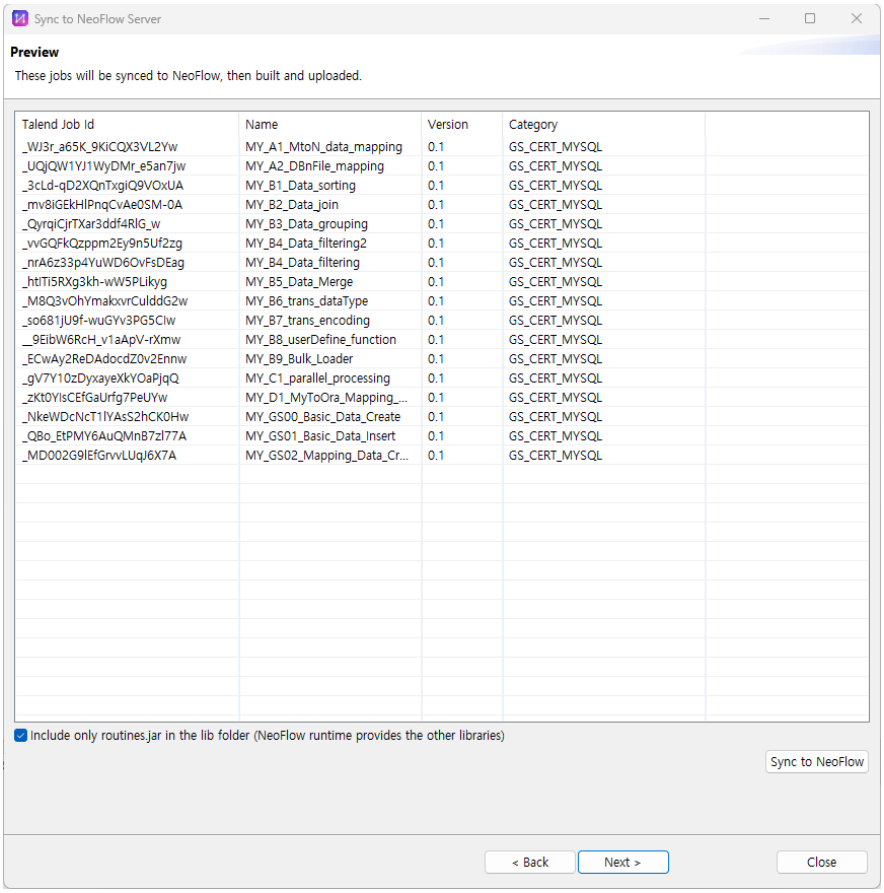
- Profile — 사용할 서버 프로파일 선택(아래에 URL/Context 표시). 기본 프로파일이 미리 선택됩니다.
- Filter — Job 이름으로 목록을 필터링합니다.
- Job 목록(체크 트리)에서 동기화할 Job 을 체크합니다. 각 행은 잡이름 v 버전 으로 표시됩니다.
- Select all / Deselect all 버튼과 하단 `Selected: N job(s)` 표시를 제공합니다.
- 진행 조건 — Job 1 개 이상 선택 + 프로파일에 Base URL 과 API token 이 있어야 합니다.



Sync 마법사 Page 1 — Profile 선택 + Job 체크(Selected: N job(s))

Page 2 — Preview

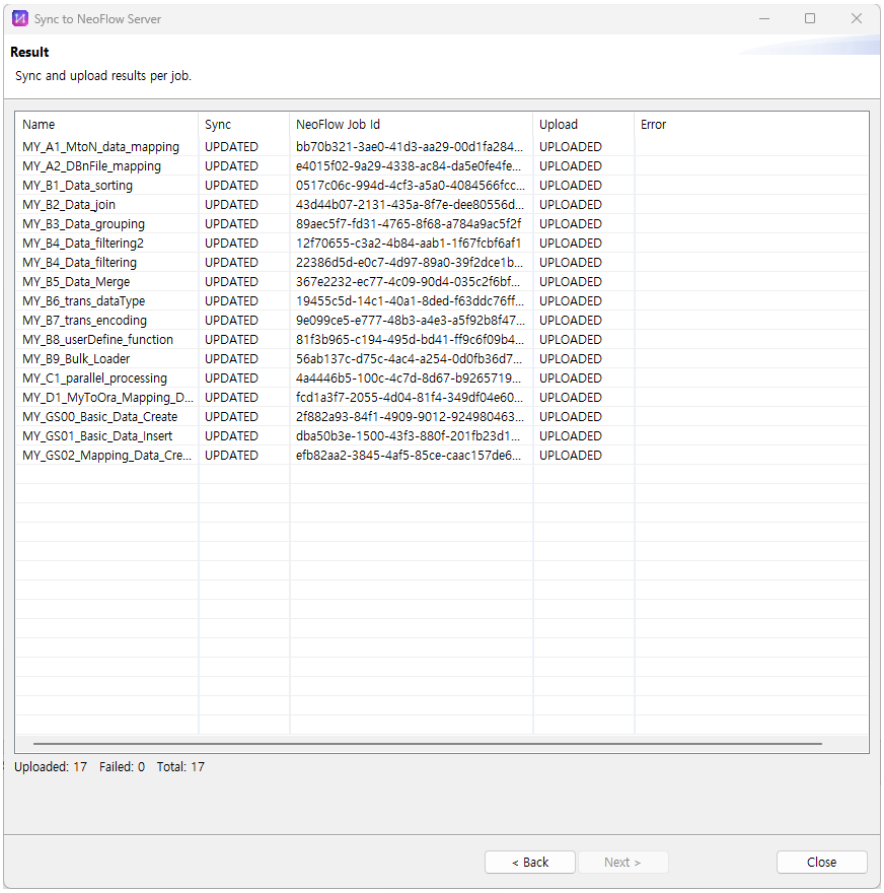
- 선택한 Job 의 메타데이터 표시 — `Talend Job Id`, `Name`, `Version`, `Category`.
- Include only routines.jar in the lib folder (기본 체크) — 체크 시 산출물 `lib/`에 `routines.jar` 만 넣고, 나머지 라이브러리는 실행할 때 서버의 공유 라이브러리(NEO_LIBS)에서 받습니다. 서버에 없는 라이브러리를 쓰는 Job 은 관리자가 웹 콘솔의 「공유 라이브러리 (NEO_LIBS)」에 올리거나, 이 옵션을 해제하고 Sync 하세요(해제하면 `lib/`의 jar 가 업로드 때 공유 라이브러리에 등록됩니다).
- Sync to NeoFlow 버튼 — 동기화 + 빌드 + 업로드 과정을 시작합니다.



◢ *Sync 마법사 Page 2 — 메타데이터 확인 + routines.jar 옵션 + Sync to NeoFlow*

Page 3 — Result

- Job 별 결과 표시 — `Name`, `Sync`, `NeoFlow Job Id`, `Upload`, `Error`.
- 하단 요약 — `Uploaded: N  Failed: N  Total: N`.
- 행 우클릭 — Copy error(에러 복사), Copy row(행 전체 TSV 복사).



◢ *Sync 마법사 Page 3 — Job 별 결과(UPDATED/UPLOADED) + 요약*

> **참고** 이 다이얼로그는 비모달입니다. 빌드가 오래 걸려도 Studio 의 다른 작업이 가능하며, Close 버튼으로 닫습니다.

17.3 내부 동작 요약

1. 메타데이터 동기화 — 선택한 Job 정보를 서버에 등록하고 NeoFlow Job ID 를 발급받습니다.
2. context 주입 — 빌드 전, 프로파일의 `Build context` 환경에 `NEOFLOW_BASE_URL`/`NEOFLOW_API_TOKEN`/`NEOFLOW_JOB_ID` 를 기록하고, 이 환경을 Job 의 기본 context 로 설정합니다.
3. 빌드 — Job 을 standalone POJO zip 으로 빌드합니다(원본 `.item` 소스 포함, source code 제외).
4. 라이브러리 정리 — 위 체크박스가 켜져 있으면 `lib/`에서 `routines.jar` 만 남기고 제거합니다.
5. 업로드 — 빌드된 zip 을 서버에 업로드합니다. 서버는 zip 안 context 파일의 `NEOFLOW_API_TOKEN` 값(및 `nf_` 로 시작하는 토큰 값)을 비워서 저장하므로, 서버에 저장되거나 다운로드되는 패키지에는 API 토큰이 남지 않습니다. 서버에서 실행할 때는 서버가 넣어 주는 값을 쓰므로 실행에는 영향이 없습니다.

> **주의** Studio 작업 공간의 Job(`.item`)과 로컬에 만든 빌드 zip 에는 API 토큰이 평문으로 남아 있습니다. Job 파일이나 로컬 빌드 zip 을 다른 사람에게 전달할 때는 context 의 `NEOFLOW_API_TOKEN` 값을 지우고, 이미 전달했다면 [설정] → [API 토큰]에서 해당 토큰을 폐기하십시오.

> **참고** 처음 Sync 한 Job 은 NeoFlow Server 에 비활성 상태로 등록됩니다. 실행하거나 스케줄에 올리려면 웹 콘솔 Job 목록에서 [활성화] 하세요(여러 개는 선택한 뒤 [선택 활성화], ADMIN·매니저). 이미 등록된 Job 을 다시 Sync 하면 상태는 그대로 유지됩니다.

> **참고** Job 안에 `tNeoFlowPre/tNeoFlowPost` 가 있으면 context 주입이 수행되고, `tNeoFlowPre` 의 `BASE_URL/API_TOKEN/JOB_ID` 파라미터가 `context.NEOFLOW_*` 로 자동 연결됩니다. `tNeoFlowPost` 에는 이 파라미터가 없으며, 서버 실행 때 서버가 주입하는 실행 토큰을 씁니다(14.6). NeoFlow 컴포넌트가 없는 Job 은 context 가 수정되지 않습니다.

17.4 Sync 전후의 Job 처리

Sync 와 Build Job 의 Deploy(18 장)는 Job 을 서버에 올리기 전후에 다음 두 가지를 자동으로 처리합니다.

열린 Job 편집기 저장·닫기

[Sync to NeoFlow] 를 누르면 선택한 Job 가운데 편집기에 열려 있는 Job 을 먼저 저장하고 닫은 뒤 진행합니다. 편집기에 열린 Job 은 저장된 Job 과 별개의 편집 상태를 가지므로, 열어 둔 채로 서버 연결 정보를 기록하면 다음 저장 때 덮어써지기 때문입니다. 저장하지 않은 변경이 있으면 저장 여부를 묻는 창이 뜹니다.

상황	메시지 (화면 표시 그대로)
Sync 중 저장 창에서 취소	`Sync cancelled: save or close the open job(s) first.`
Deploy 중 저장 창에서 취소	`Deploy cancelled: save or close the open job(s) first.`
Sync 진행 중 취소	`Sync cancelled.`
서버 오류 등으로 Sync 중단	`Sync aborted: <오류 내용>`

Job 사본의 NeoFlow 연결 해제

Repository 에서 [Duplicate] 하거나 [Copy] → [Paste] 로 만든 Job 사본은 아직 서버에 등록되지 않은 새 Job 입니다. 그래서 사본의 context 변수 `NEOFLOW_BASE_URL`·`NEOFLOW_API_TOKEN`·`NEOFLOW_TOKEN`·`NEOFLOW_JOB_ID` 값을 비워, 원본이 등록된 서버 Job 과 분리합니다. 변수 자체는 남겨 두므로 `tNeoFlowPre` 의 설정은 그대로 유지됩니다.

> **참고** 사본을 서버에 올리려면 사본을 Sync 하세요. Sync 가 사본 자신의 NeoFlow Job ID 로 값을 다시 채웁니다.

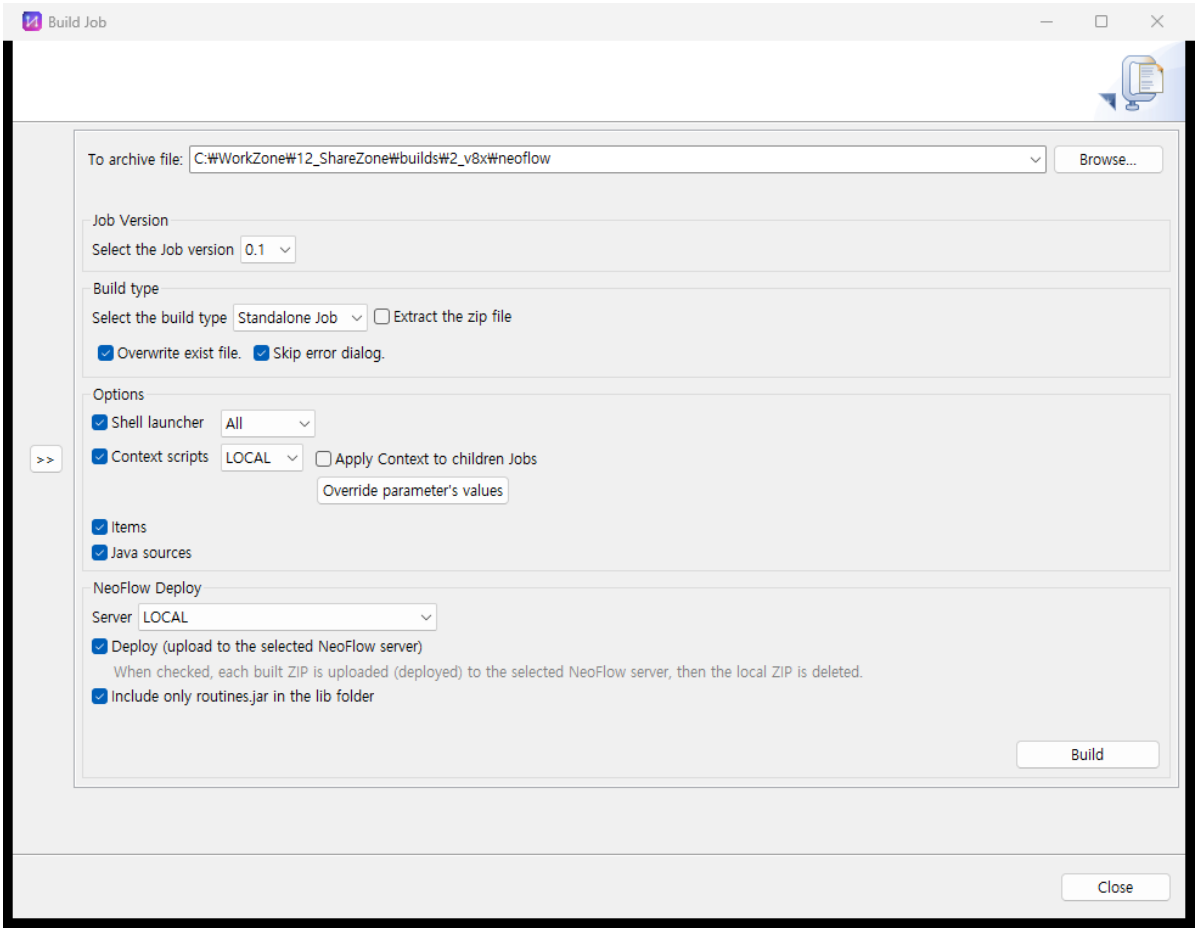
18. Build Job (NeoFlow Deploy)

Job 을 로컬 zip 으로 빌드하면서, 옵션으로 NeoFlow 서버에 배포(Deploy)까지 한 번에 할 수 있습니다. Sync 마법사가 "서버 등록 + 업로드"에 특화돼 있다면, Build Job 은 표준 빌드 산출물을 만들면서 같은 배포를 곁들이는 방식입니다.

> **주의** 전제 조건 — Sync 와 동일하게 유효한 License 와 NeoFlow Server 프로파일이 필요합니다. License 가 유효하지 않으면 Build Job 메뉴 실행 시 에러 다이얼로그가 뜹니다.

18.1 실행

Repository 에서 Job(또는 폴더)을 우클릭 → [Build Job] 을 실행합니다.



▲ Build Job 다이얼로그 — 표준 export 옵션 + NeoFlow Deploy 섹션

18.2 주요 항목

- To archive file — 빌드 산출물(zip)을 저장할 경로.
- Job Version / Build type — 버전, 빌드 타입(Standalone Job 등), Overwrite/Skip error 옵션.
- Options — Shell launcher, Context scripts(환경 선택), Items(원본 `.item` 포함), Java sources 등 표준 export 옵션.

NeoFlow Deploy (NeoFlow 전용)

- Server — 배포할 NeoFlow 서버 프로파일(14 장에서 등록).

- Deploy (upload to the selected NeoFlow server) — 기본 체크. 체크 시 빌드된 zip 을 선택한 NeoFlow 서버로 업로드(배포)합니다. 로컬 zip 만 만들려면 해제합니다.
- Include only routines.jar in the lib folder — 기본 체크. 체크 시 `lib/`에 `routines.jar` 만 넣고 나머지는 실행할 때 서버의 공유 라이브러리(NEO_LIBS)에서 받습니다(Sync 의 동일 옵션과 같음, 17.2).

> **주의** Deploy 안내 문구: "When checked, each built ZIP is uploaded (deployed) to the selected NeoFlow server, then the local ZIP is deleted." — 배포 후 로컬 zip 은 삭제됩니다(서버 업로드가 목적이므로). 로컬 보관이 필요하면 Deploy 를 해제하고 빌드만 하세요.

> **참고** Deploy 가 켜진 Build Job 은 내부적으로 Sync 와 동일한 동작(메타데이터 동기화 + `context.NEOFLOW_*` 주입 + 업로드)을 수행합니다(17.3 참고).

18.3 배포 실행과 결과

[Deploy] 를 켠 채 [Build] 를 누르면 Sync 와 같은 순서(열린 편집기 정리 → 메타데이터 동기화 → context 주입 → 빌드 → 업로드)로 처리한 뒤, `NeoFlow Deploy Result` 창에 Job 별 결과를 보여 줍니다.

- 결과 창의 열 — `Name`, `Sync`, `Upload`, `Error`.
- 하단 요약 — `Uploaded: N  Failed: N  Total: N`.

Build Job 창은 비모달 창입니다. 빌드가 진행되는 동안에도 Studio 의 다른 작업을 할 수 있으며, [Close] 버튼이 계속 활성화되어 있어 진행 중에 중단을 요청할 수 있습니다.

배포 중 문제가 있으면 제목이 `NeoFlow Deploy` 인 창에 다음 문구가 표시됩니다.

상황	메시지 (화면 표시 그대로)
서버 프로파일 없음	`No NeoFlow server profile is configured. Set one in Preferences > NeoFlow Server, or uncheck Deploy to build locally only.`
선택한 Job 없음	`No jobs selected to deploy.`
편집기 저장 창에서 취소	`Deploy cancelled: save or close the open job(s) first.`
진행 중 취소	`Deploy cancelled.`
서버 오류 등으로 중단	`Deploy aborted: <오류 내용>`

Deploy 를 해제하고 빌드하다 실패하면 `N job(s) did not build:` 로 시작하는 창에 실패한 Job 과 `See the Error Log view for the full stack trace.` 가 표시됩니다. Error Log 뷰에서 원인을 확인하세요.

> **참고** 메시지 속 `Preferences > NeoFlow Server` 는 화면 문구 그대로이며, 실제 위치는 [Window] → [Preferences] → [NeoFlow] → [NeoFlow Server] 입니다(14.2).

19. 따라 하기

앞 장에서 살펴본 화면과 개념을 실제로 써 봅니다. 실습 1 은 아무 준비 없이 바로 할 수 있는 «Hello World» 이고, 실습 2 부터는 DB 를 사용해 조회·변환·적재까지 실제 ETL 흐름을 만들어 봅니다. 순서대로 진행하면 뒤 실습이 앞 실습의 결과를 사용합니다.

실습	내용	핵심 컴포넌트
1	첫 Job 만들기 — 콘솔에 데이터 출력	`tFixedFlowInput, tLogRow`
2	DB 연결과 테이블 생성	`tDBConnection, tDBRow`
3	테스트 데이터 생성·적재	`tRowGenerator, tMap, tDBOutput`
4	조회 → 정렬 → 파일 출력	`tDBInput, tSortRow, tFileOutputDelimited`
5	두 소스 조인	`tFileInputDelimited, tJoin`
6	필터링과 조건 분기	`tFilterRow, tMap 출력 필터`
7	그룹핑·집계	`tAggregateRow`
8	자식 Job 병렬 호출	`tRunJob, tNeoFlowThread`
9	NeoFlow Server 로 동기화	`Sync to NeoFlow Server`

19.1 실습 준비

실습 1 은 준비물이 없습니다. 실습 2 부터는 접속 가능한 데이터베이스가 필요합니다.

- **DB** — MySQL 8.0 기준으로 설명합니다. 테이블을 만들고 지울 수 있는 계정이 필요합니다(조회 전용 계정으로는 실습 2~7 을 할 수 없습니다).
- **작업 폴더** — 파일 출력 실습에서 사용할 폴더를 하나 만들어 둡니다(예: `C:\neoflow-lab`).
- **DB 확인 도구** — 적재 결과를 눈으로 확인할 DB 클라이언트(DBeaver 등)가 있으면 편합니다.

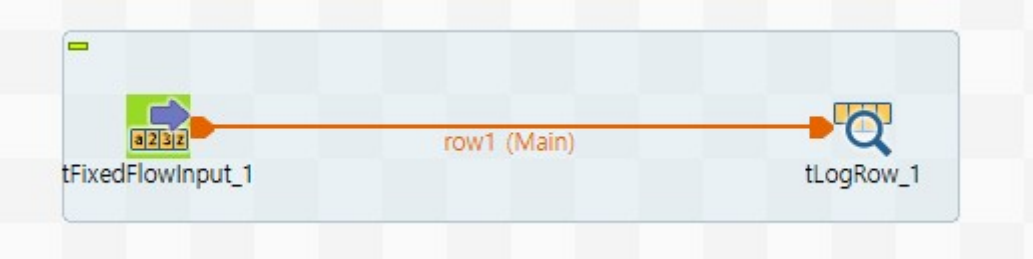
> **참고** 이 장의 화면 캡처는 실제로 작성·실행한 예제 Job 에서 가져왔습니다. 예제 Job 들은 접속 정보를 NeoFlow Server 가 주입한 실행 파라미터에서 읽도록 되어 있어 캡처의 Host·Port 칸이 `NeoFlowJobApi.getParameter(...)` 로 보입니다. **처음 실습할 때는 그 자리에 접속 정보를 직접 입력하거나 context 변수를 쓰면 됩니다.**

> **참고** 이 장의 Run 콘솔 화면은 개발·검증 서버에 연결해 Studio 에서 실행한 결과입니다. 운영(prod) 프로파일 서버에 연결하면 `tNeoFlowPre·tNeoFlowPost` 로그 대신 `studio context unavailable` 경고가 남고, `getParameter` 로 접속 정보를 받는 예제 Job 은 DB 에 연결하지 못합니다(14.6). 예제 Job 의 파일 경로는 `context.GS_DATA_DIR` 을 기준으로 합니다.

19.2 실습 1 — 첫 Job 만들기

항목	내용
목표	컴포넌트를 배치하고 연결해 Job 을 실행하는 기본 흐름을 익힌다. DB 없이 진행한다.
사용 컴포넌트	tFixedFlowInput, tLogRow

1. Repository 에서 `Job Designs` 우클릭 → `[Create job]` 을 선택하고 Name 에 `HELLO_NEOFLOW` 를 입력한 뒤 [Finish] 를 누릅니다. 빈 캔버스가 열립니다.
2. 팔레트 검색란에 `tFixedFlowInput` 을 입력해 찾은 뒤 캔버스로 드래그합니다. 이 컴포넌트는 직접 입력한 값으로 데이터 행을 만들어 줍니다.
3. 같은 방법으로 `tLogRow` 를 캔버스에 배치합니다. 흐름에 들어온 데이터를 Run 콘솔에 출력하는 컴포넌트입니다.
4. `tFixedFlowInput` 을 클릭하고 Component 탭 → `Basic settings` → [Edit schema] 를 눌러 컬럼 두 개를 추가합니다 — `MESSAGE`(String), `SEQ`(Integer).
5. Mode 에서 `Use Single Table` 을 선택하고 Value 에 각각 `"Hello NeoFlow"` 와 `1` 을 입력합니다. Number of rows 는 `1` 로 둡니다.
6. `tFixedFlowInput` 에서 우클릭 → `[Row] → [Main]` 을 선택한 뒤 `tLogRow` 를 클릭해 두 컴포넌트를 연결합니다.
7. `tLogRow` 를 클릭하고 Mode 를 `Table (print values in cells of a table)` 로 바꿉니다. 결과가 표 형태로 보기 좋게 출력됩니다.
8. `Ctrl+S` 로 저장한 뒤 `F6` 을 누릅니다. Configuration 영역이 Run 탭으로 바뀌면서 Job 이 실행됩니다.

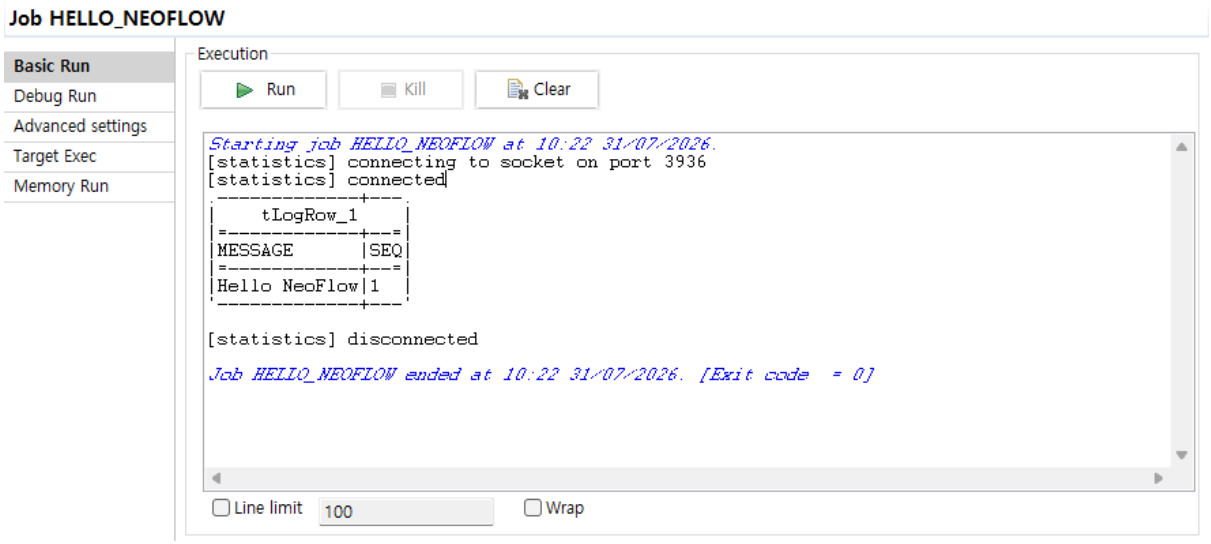


▲ *실습 1 — HELLO_NEOFLOW Job 캔버스*

Run 콘솔에 아래와 같은 출력이 나오고 마지막 줄에 `Exit code = 0` 이 찍히면 성공입니다.

```
Starting job HELLO_NEOFLOW at 10:22 31/07/2026.
[statistics] connecting to socket on port 3936
[statistics] connected
.-------------+---.
|    tLogRow_1    |
|=------------+--=|
|MESSAGE      |SEQ|
|=------------+--=|
|Hello NeoFlow|1  |
'-------------+---'

[statistics] disconnected
Job HELLO_NEOFLOW ended at 10:22 31/07/2026. [Exit code  = 0]
```



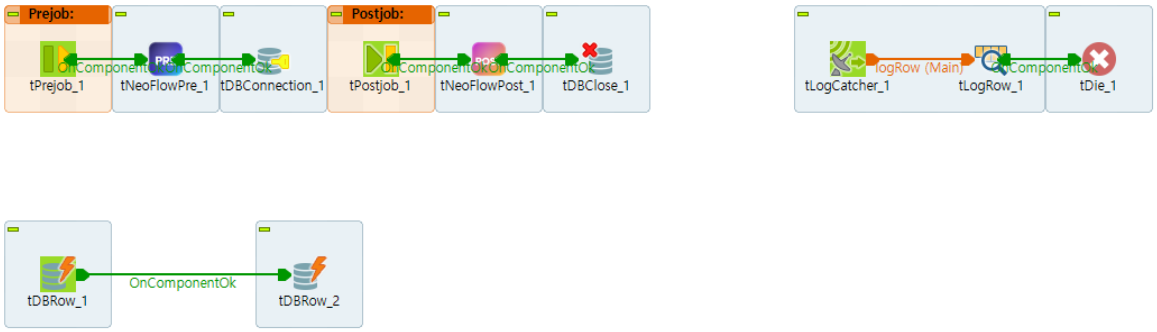
▲ 실습 1 — Run 콘솔 실행 결과

> **TIP** `tLogRow` 는 ETL 개발에서 가장 자주 쓰는 디버깅 도구입니다. 흐름 중간이 궁금할 때 연결선 위에 `tLogRow` 를 끼워 넣으면 그 지점의 데이터를 그대로 확인할 수 있습니다.

> **주의** 실행했는데 캔버스에 빨간 X 표시가 있고 Run 이 되지 않는다면 Problems 탭을 확인하세요. 스키마 미정의, 필수 항목 누락이 대부분의 원인입니다.

19.3 실습 2 — DB 연결과 테이블 생성

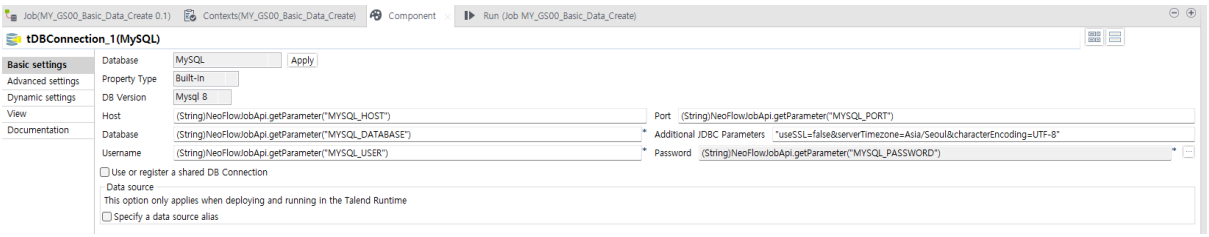
항목	내용
목표	DB 연결을 한 번 맺어 Job 전체에서 재사용하고, 임의 SQL(DDL)을 실행한다.
사용 컴포넌트	tDBConnection, tDBRow
예제 Job	MY_GS00_Basic_Data_Create



▲ 실습 2 — 테이블 생성 Job 캔버스

1. 새 Job 을 만들고 팔레트에서 `tDBConnection`(또는 `tMysqlConnection`)을 배치합니다.
2. Component 탭에서 DB Type·Host·Port·Database·Username·Password 를 입력합니다. Additional parameters 에 `useSSL=false&serverTimezone=Asia/Seoul&characterEncoding=UTF-8` 을 넣어 타임존·문자셋을 고정합니다.

3. tDBRow 를 두 개 배치합니다. 첫 번째는 기존 테이블 삭제, 두 번째는 테이블 생성용입니다.
4. 두 tDBRow 모두 Use an existing connection 을 체크하고 앞서 만든 연결을 선택합니다. 이렇게 하면 접속 정보를 한 곳에서만 관리하면 됩니다.
5. 첫 번째 tDBRow 의 Query 에 DROP TABLE IF EXISTS MEMBER 를 입력합니다. IF EXISTS 를 쓰면 테이블이 없어도 오류가 나지 않아 반복 실행이 가능합니다.
6. 두 번째 tDBRow 의 Query 에 아래 CREATE TABLE 문을 입력합니다.
7. 첫 번째 tDBRow 에서 우클릭 → [Trigger] → [On Component Ok] 로 두 번째 tDBRow 에 연결합니다. 삭제가 성공했을 때만 생성이 실행됩니다.
8. Ctrl+S 저장 후 F6 으로 실행합니다.

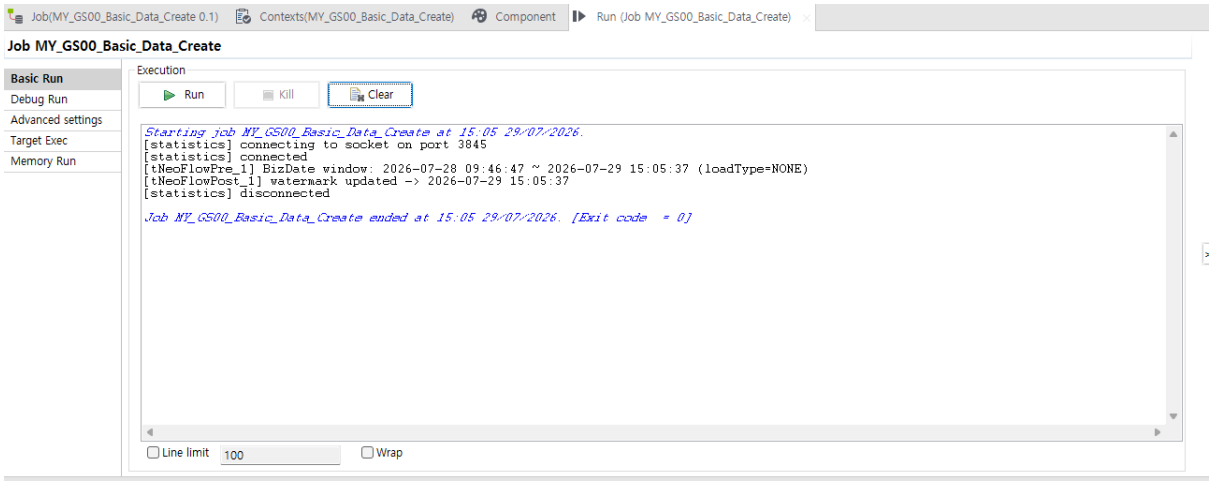


◢ *tDBConnection — 접속 정보 설정*

```
CREATE TABLE MEMBER (
    SEQ         INT          NOT NULL AUTO_INCREMENT,
    FIRST_NAME  VARCHAR(50)  NULL,
    LAST_NAME   VARCHAR(50)  NULL,
    GENDER      VARCHAR(50)  NOT NULL,
    AGE         INT          NOT NULL,
    CITY        VARCHAR(50)  NOT NULL,
    JOIN_TYPE   VARCHAR(50)  NOT NULL DEFAULT 'ONLINE',
    CREATE_DT   TIMESTAMP    NOT NULL,
    MODIFY_DT   TIMESTAMP    NOT NULL,
    PRIMARY KEY (SEQ)
) DEFAULT CHARSET=utf8mb4
```



◢ *tDBRow — CREATE TABLE 쿼리 입력*

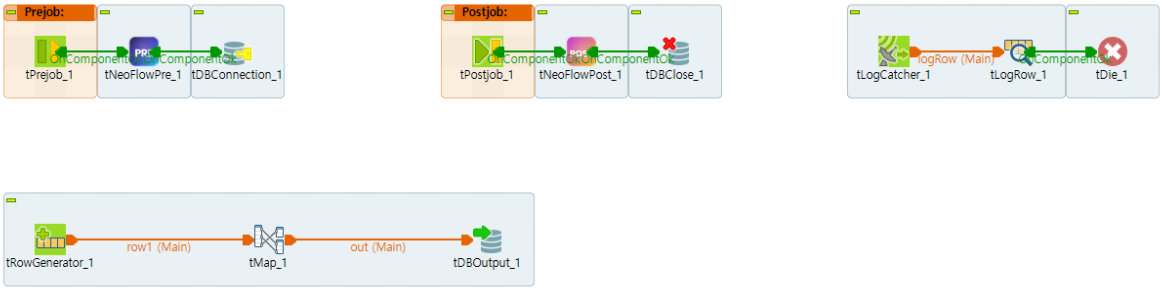


▲ *실습 2 — 실행 결과 (Exit code = 0)*

> TIP `tDBConnection` 으로 연결을 공유하면 컴포넌트마다 접속하지 않아 성능이 좋아지고, **하나의 트랜잭션으로 묶을 수 있습니다.** Job 끝에 `tDBClose` 를 두어 연결을 닫는 것이 좋은 습관입니다.

19.4 실습 3 — 테스트 데이터 생성·적재

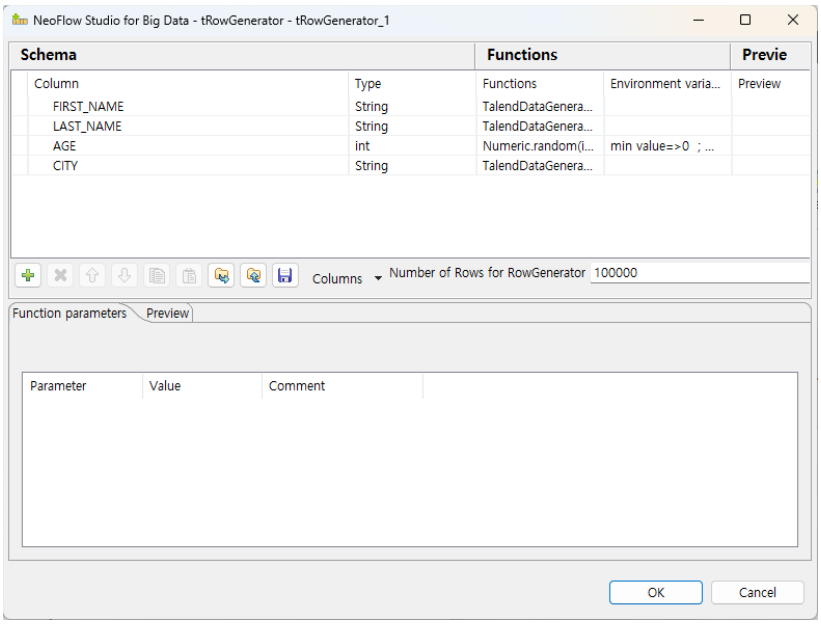
항목	내용
목표	가상 데이터를 만들고 tMap 으로 컬럼을 파생시킨 뒤 테이블에 적재한다.
사용 컴포넌트	tRowGenerator, tMap, tDBOutput
예제 Job	MY_GS01_Basic_Data_Insert



▲ *실습 3 — 데이터 생성·적재 Job 캔버스*

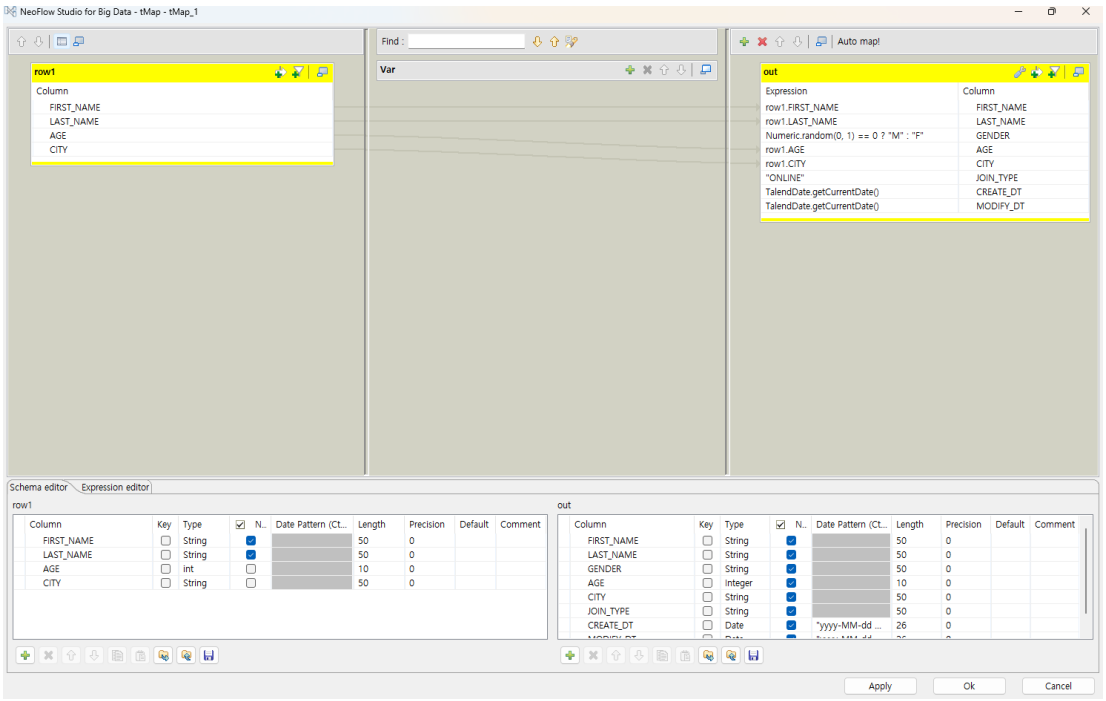
1. 실습 2 와 같은 방식으로 `tDBConnection` 을 배치해 연결을 만듭니다.
2. `tRowGenerator` 를 배치하고 더블 클릭해 편집기를 엽니다. 컬럼 4 개 (`FIRST_NAME`·`LAST_NAME`·`AGE`·`CITY`)를 추가하고 각각 값 생성 함수를 지정합니다.
3. Number of Rows for RowGenerator 를 `100000` 으로 설정합니다.
4. `tMap` 을 배치하고 `tRowGenerator` 에서 `[Row] → [Main]` 으로 연결합니다.
5. `tMap` 을 더블 클릭해 편집기를 열고, 오른쪽에 출력 테이블 `out` 을 만든 뒤 아래 식대로 컬럼 8 개를 매핑합니다(`SEQ` 는 AUTO_INCREMENT 라 매핑하지 않습니다). 입력에 없는 컬럼 (GENDER·JOIN_TYPE·CREATE_DT·MODIFY_DT)은 식으로 새로 만듭니다.
6. `tDBOutput` 을 배치해 `tMap` 의 `out` 을 연결하고, Table 에 `MEMBER`, Action on data 에 `Insert` 를 지정합니다. Action on table 은 `Default`(테이블을 건드리지 않음) 그대로 둡니다.
7. 대량 적재이므로 Commit every 와 Batch size 를 `10000` 으로 올립니다.

8. 저장 후 F6 으로 실행하고, DB 도구로 100,000 건이 들어왔는지 확인합니다.



▲ *tRowGenerator 편집기 — 컬럼별 생성 함수와 행 수*

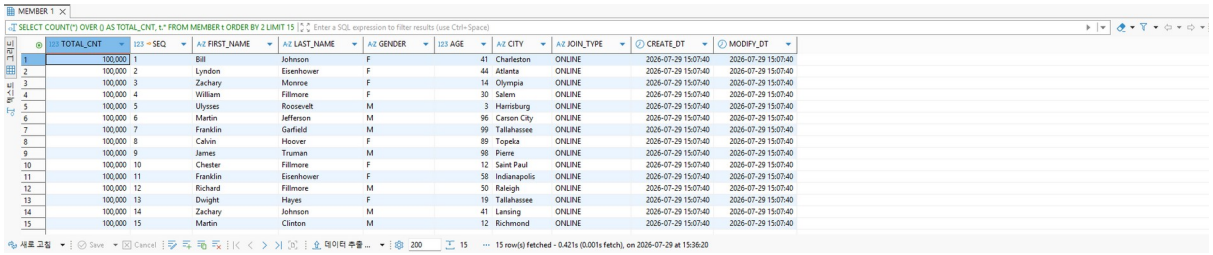
```
FIRST_NAME : TalendDataGenerator.getFirstName()
LAST_NAME  : TalendDataGenerator.getLastName()
AGE        : Numeric.random(0, 100)
CITY       : TalendDataGenerator.getUsCity()
Rows       : 100000
```



▲ *tMap 편집기 — 입력 매핑과 파생 컬럼 식*

```
FIRST_NAME : row1.FIRST_NAME
LAST_NAME  : row1.LAST_NAME
GENDER     : Numeric.random(0, 1) == 0 ? "M" : "F"
```

```
AGE         : row1.AGE
CITY        : row1.CITY
JOIN_TYPE   : "ONLINE"
CREATE_DT   : TalendDate.getCurrentDate()
MODIFY_DT   : TalendDate.getCurrentDate()
```

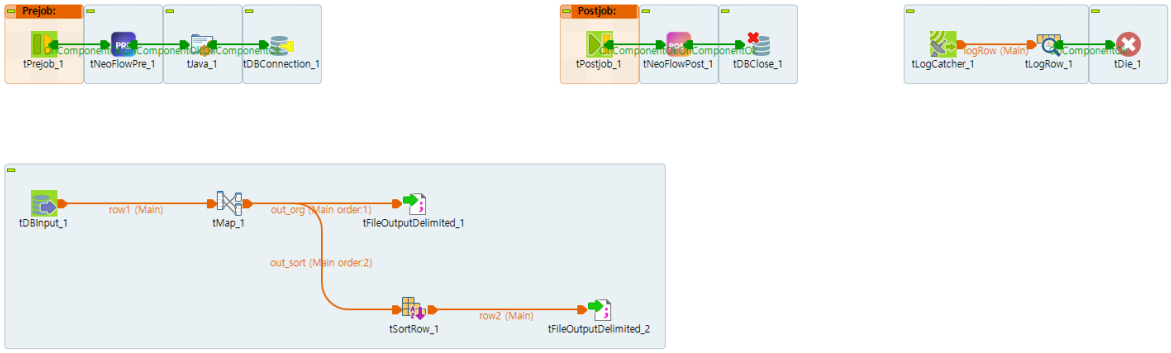


▲ 적재 결과 확인 — MEMBER 테이블 100,000 건

> **참고** tMap 은 이 도구에서 가장 중요한 컴포넌트입니다. 왼쪽이 입력, 오른쪽이 출력이며 가운데에서 선을 끌어 매핑합니다. 컬럼명이 같으면 왼쪽 위 **Auto map** 버튼으로 한 번에 연결할 수 있습니다.

19.5 실습 4 — 조회 → 정렬 → 파일 출력

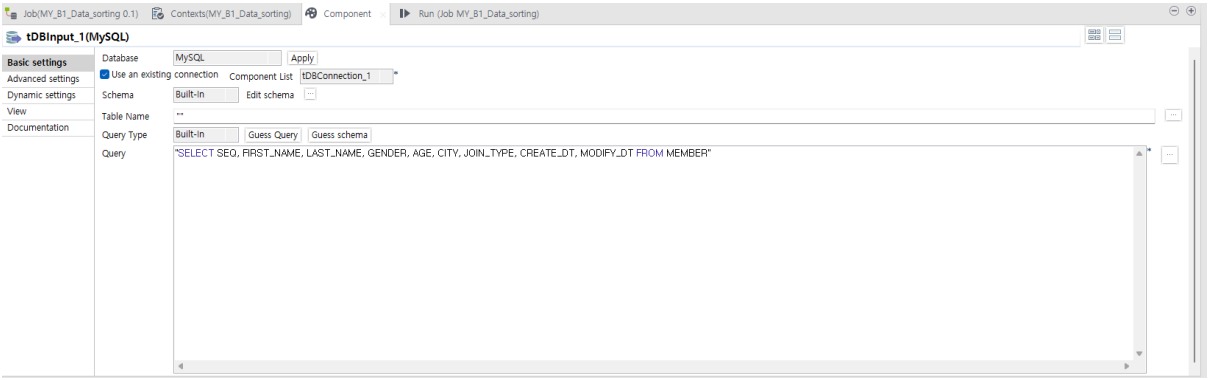
항목	내용
목표	DB 에서 읽은 데이터를 두 갈래로 나눠 하나는 그대로, 하나는 정렬해서 CSV 로 내보낸다.
사용 컴포넌트	tDBInput, tMap(다중 출력), tSortRow, tFileOutputDelimited
예제 Job	MY_B1_Data_sorting



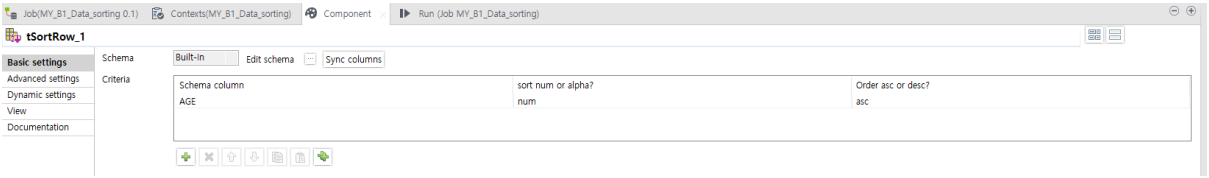
▲ 실습 4 — 정렬 전·후 두 파일로 출력하는 Job

1. tDBConnection 으로 연결을 만들고, tDBInput 을 배치해 Use an existing connection 을 체크합니다.
2. Query 에 SELECT SEQ, FIRST_NAME, LAST_NAME, GENDER, AGE, CITY, JOIN_TYPE, CREATE_DT, MODIFY_DT FROM MEMBER 를 입력하고 [Guess schema] 로 스키마 9 개 컬럼을 가져옵니다.
3. tMap 을 연결하고 편집기에서 **출력 테이블을 두 개** 만듭니다 — out_org(정렬 전)와 out_sort(정렬 후). 입력 9 개 컬럼을 양쪽에 동일하게 매핑합니다.
4. out_org 는 tFileOutputDelimited 로 바로 연결합니다. File name 에 C:/neoflow-lab/member_org.csv 를 지정합니다.

5. `out_sort` 는 `tSortRow` 를 거쳐 두 번째 `tFileOutputDelimited` 로 연결하고, File name 에 `C:/neoflow-lab/member_sorted.csv` 를 지정합니다.
6. `tSortRow` 의 Criteria 표에 정렬 기준을 추가합니다 — Schema column `AGE`, sort `num`, order `asc`.
7. 두 파일 컴포넌트 모두 Row Separator 를 `\n`, Field Separator 를 `;` 로 두고 Advanced settings 에서 Encoding 을 `UTF-8` 로 지정합니다.
8. 저장 후 실행하고, 두 CSV 파일을 열어 AGE 컬럼 순서가 다른지 비교합니다.



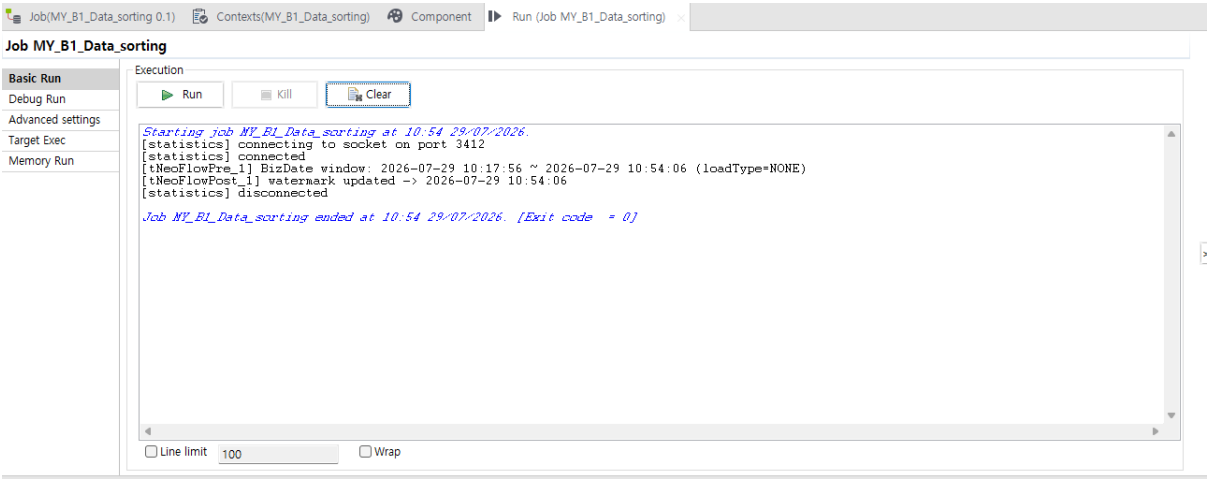
▲ *tDBInput — 쿼리와 스키마 설정*



▲ *tSortRow — 정렬 기준(AGE / num / asc)*

> **주의** `tSortRow` 에서 숫자 컬럼을 `alpha`(문자 정렬)로 두면 `10` 이 `9` 보다 앞에 오는 사전순 정렬이 됩니다. 숫자 컬럼은 반드시 `num` 을 선택하세요.

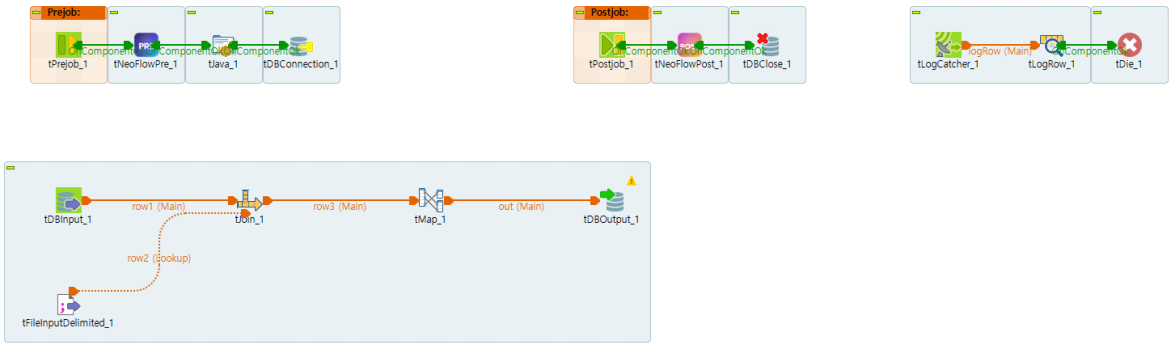
> **TIP** 하나의 입력을 여러 갈래로 나눌 때는 `tMap` 에 출력 테이블을 여러 개 만드는 방법이 가장 간단합니다. 데이터를 두 번 읽지 않아도 되고, 갈래마다 다른 컬럼 구성·필터를 줄 수 있습니다.



▲ *실습 4 — Run 콘솔 실행 결과*

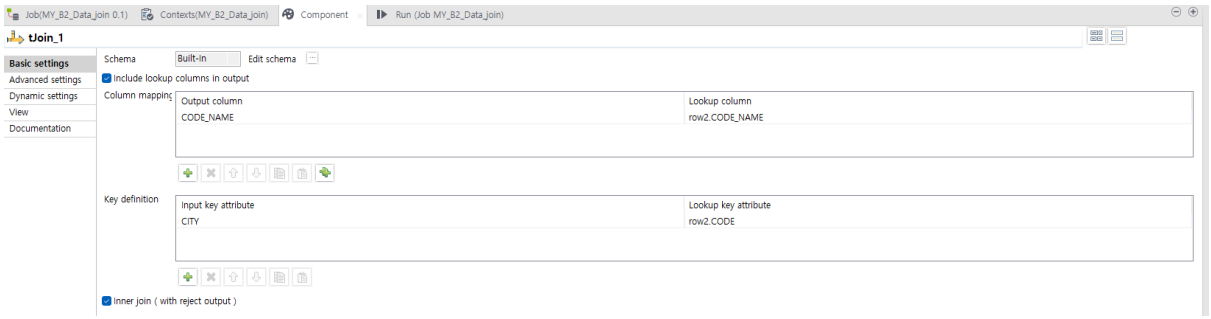
19.6 실습 5 — 두 소스 조인

항목	내용
목표	DB 테이블과 CSV 파일을 조인해 코드값을 이름으로 치환한다.
사용 컴포넌트	tFileInputDelimited, tJoin
예제 Job	MY_B2_Data_join



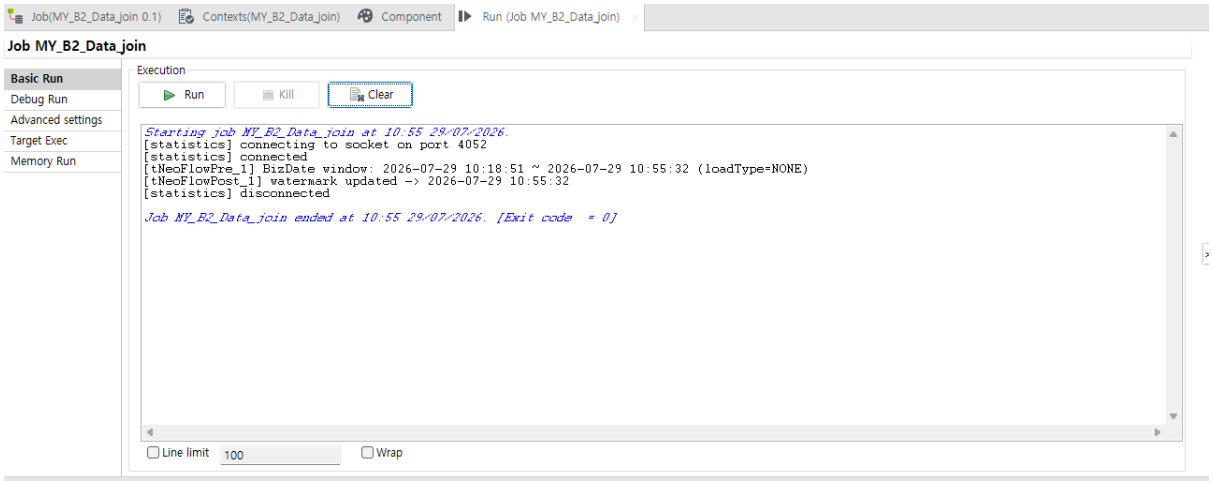
▲ *실습 5 — DB(주 입력)와 파일(룩업)을 tJoin 으로 결합*

1. 실습 4 처럼 `tDBInput` 으로 `MEMBER` 를 조회합니다. 이 흐름이 **주 입력(Main)** 이 됩니다.
2. `tFileInputDelimited` 를 배치하고 룩업용 CSV(`code_master.csv` — CODE, CODE_NAME 두 컬럼)를 지정합니다. Field Separator 는 `,`, Encoding 은 `UTF-8` 입니다.
3. `tJoin` 을 배치합니다. **먼저** `tDBInput` 에서 `[Row] → [Main]` 으로 연결하고, **그 다음** `tFileInputDelimited` 에서 연결합니다. 나중에 연결한 쪽이 자동으로 Lookup 이 됩니다.
4. `tJoin` 의 **Key definition** 표에 조인 키를 지정합니다 — Input key attribute `CITY` ↔ Lookup key attribute `row2.CODE`.
5. **Column mapping** 표에 가져올 컬럼 `CODE_NAME` 을 추가하고, `Include lookup columns in output` 이 체크돼 있는지 확인합니다.
6. 조인 결과를 `tMap` 으로 받아 적재 스키마에 맞춰 정리한 뒤 `tDBOutput` 으로 내보냅니다.
7. 저장 후 실행하고 결과 테이블에 CODE_NAME 이 채워졌는지 확인합니다.



▲ *tJoin — Join Key 와 Lookup columns 설정*

> **참고** 조인은 `tJoin` 과 `tMap` 두 가지 방법이 있습니다. `tJoin` 은 조인 키와 가져올 컬럼만 지정하는 단순 결합이고, `tMap` 은 조인과 동시에 변환·필터·다중 출력까지 처리합니다. 변환이 필요 없는 단순 코드 매칭이면 `tJoin` 이 읽기 쉽습니다.

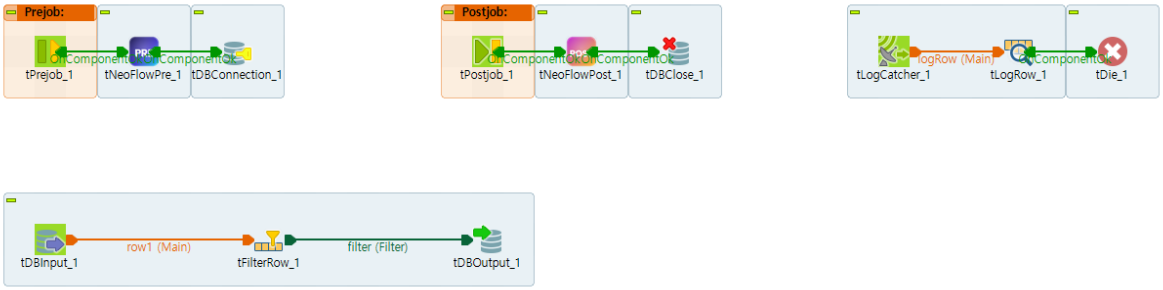


▲ *실습 5 — Run 콘솔 실행 결과*

19.7 실습 6 — 필터링과 조건 분기

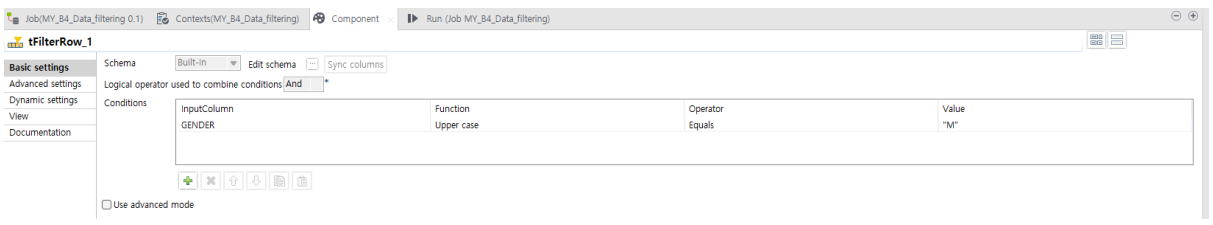
항목	내용
목표	조건에 맞는 행만 남기는 필터링과, 조건별로 서로 다른 대상에 나눠 담는 분기를 각각 만들어 본다.
사용 컴포넌트	tFilterRow, tMap 출력 필터
예제 Job	MY_B4_Data_filtering / MY_B4_Data_filtering2

(1) 조건에 맞는 행만 남기기 — tFilterRow



▲ *필터링 — GENDER 가 M 인 행만 통과*

1. `tDBInput` 으로 `MEMBER` 를 조회하고 `tFilterRow` 를 연결합니다.
2. Conditions 표에 조건을 추가합니다 — InputColumn `GENDER`, Function `Upper case`, Operator `Equals`, Value `"M"`.
3. `tFilterRow` 에서 우클릭 → `[Row] → [Filter]` 로 `tDBOutput` 에 연결합니다. 조건을 통과한 행만 이 링크로 흐릅니다.
4. 저장 후 실행하고, 적재 건수가 원본보다 줄었는지 확인합니다.

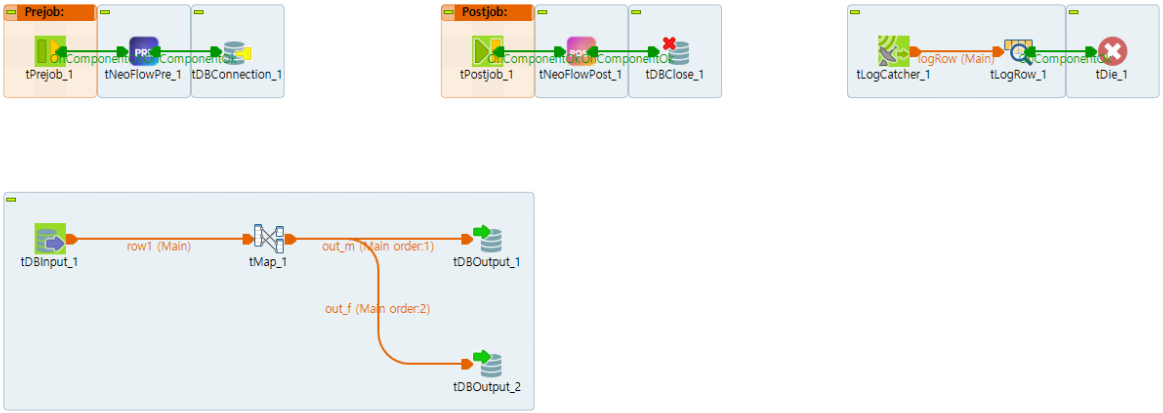


▲ *tFilterRow — 조건 설정*

> TIP Upper case 같은 Function 을 함께 지정하면 대소문자 차이를 흡수할 수 있습니다. 생성되는 조건식은 `$source == null ? false : ...` 형태라 값이 null 이어도 오류 없이 걸러집니다.

(2) 조건별로 나눠 담기 — tMap 출력 필터

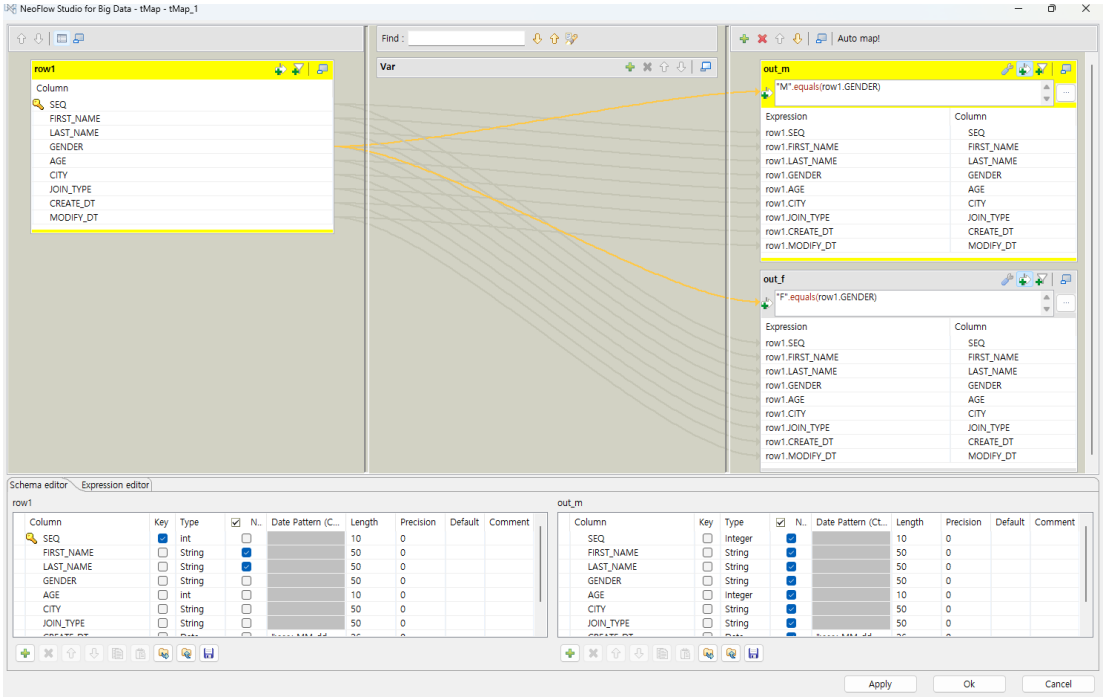
필터링이 «맞는 것만 남기기»라면, 분기는 «모두 살리되 갈 곳을 나누기»입니다. `tMap` 의 각 출력 테이블에 필터 식을 걸면 하나의 입력을 여러 대상으로 분리할 수 있습니다.



▲ 조건 분기 — 성별에 따라 두 테이블로 분리

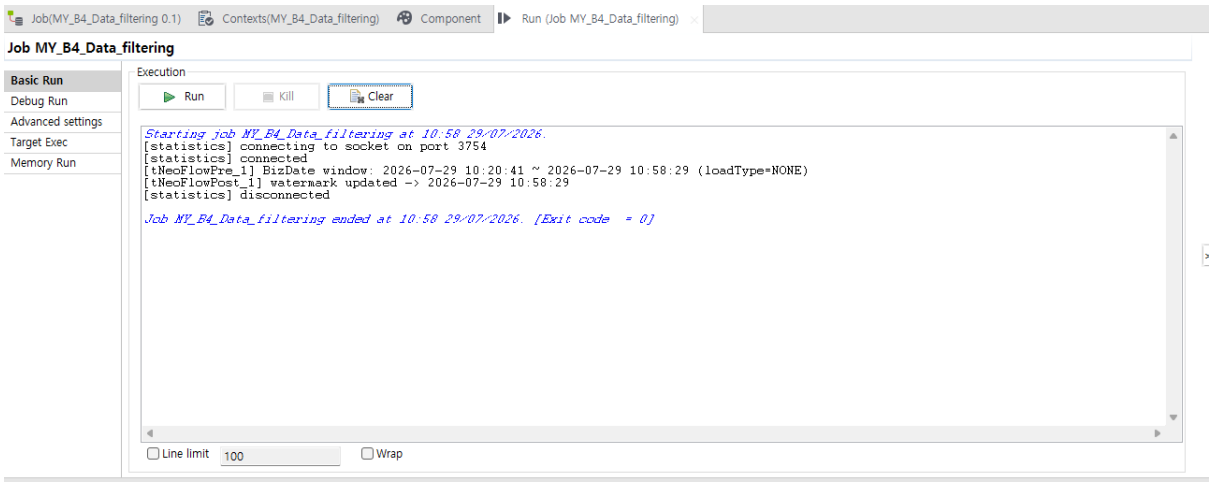
1. `tDBInput` 에 `tMap` 을 연결하고 편집기에서 출력 테이블 두 개(`out_m`, `out_f`)를 만듭니다.
2. 입력 9 개 컬럼을 양쪽 출력에 동일하게 매핑합니다.
3. 각 출력 테이블 머리의 필터 아이콘을 눌러 Expression filter 를 켜고 조건식을 입력합니다.
4. `out_m` 은 `tDBOutput_1`(MEMBER_B4_M), `out_f` 는 `tDBOutput_2`(MEMBER_B4_F)로 각각 연결합니다.
5. 실행 후 두 테이블 건수의 합이 원본 건수와 같은지 확인합니다. 같으면 행 손실 없이 분기된 것입니다.

```
out_m  Expression filter : "M".equals(row1.GENDER)
out_f  Expression filter : "F".equals(row1.GENDER)
```



▲ *tMap 편집기 — 출력별 Expression filter*

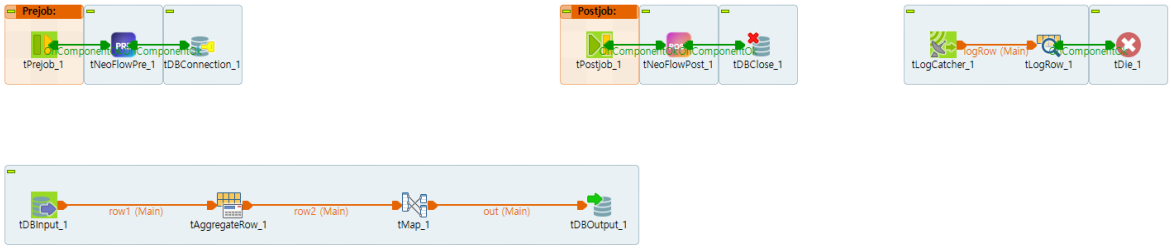
> **TIP** 조건식은 `row1.GENDER.equals("M")` 보다 **리터럴을 앞에 두는** `"M".equals(row1.GENDER)` 형태를 쓰세요. 컬럼 값이 null 이어도 NullPointerException 이 나지 않습니다.



▲ *실습 6 — Run 콘솔 실행 결과*

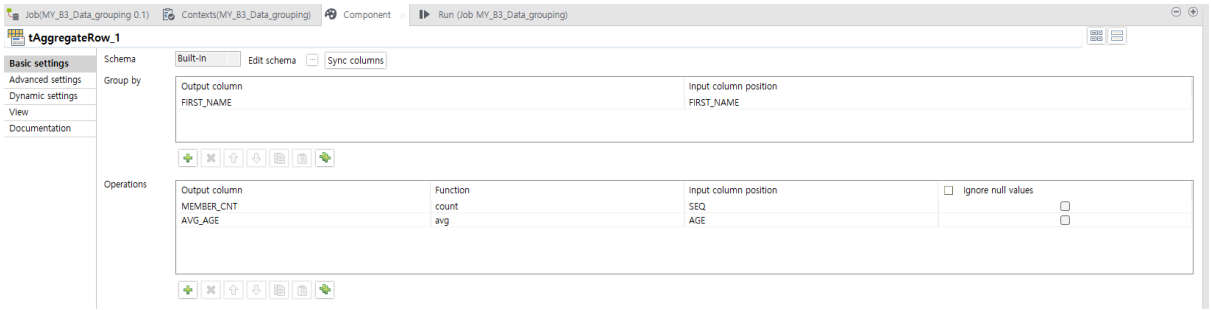
19.8 실습 7 — 그룹핑·집계

항목	내용
목표	그룹 키별로 건수와 평균을 계산해 요약 테이블을 만든다.
사용 컴포넌트	tAggregateRow, tMap
예제 Job	MY_B3_Data_grouping



▲ *실습 7 — 그룹별 집계 Job*

1. `tDBInput` 으로 `MEMBER` 를 조회하고 `tAggregateRow` 를 연결합니다.
2. Group by 표에 그룹 키를 지정합니다 — Output column `FIRST_NAME` ← Input column `FIRST_NAME`.
3. Operations 표에 집계 두 건을 추가합니다 — `MEMBER_CNT = count(SEQ)`, `AVG_AGE = avg(AGE)`.
4. `avg` 결과는 Double 이므로 `tMap` 에서 `(int) Math.round(row2.AVG_AGE)` 로 정수 변환합니다.
5. `tDBOutput` 으로 결과 테이블에 적재하고 실행합니다.

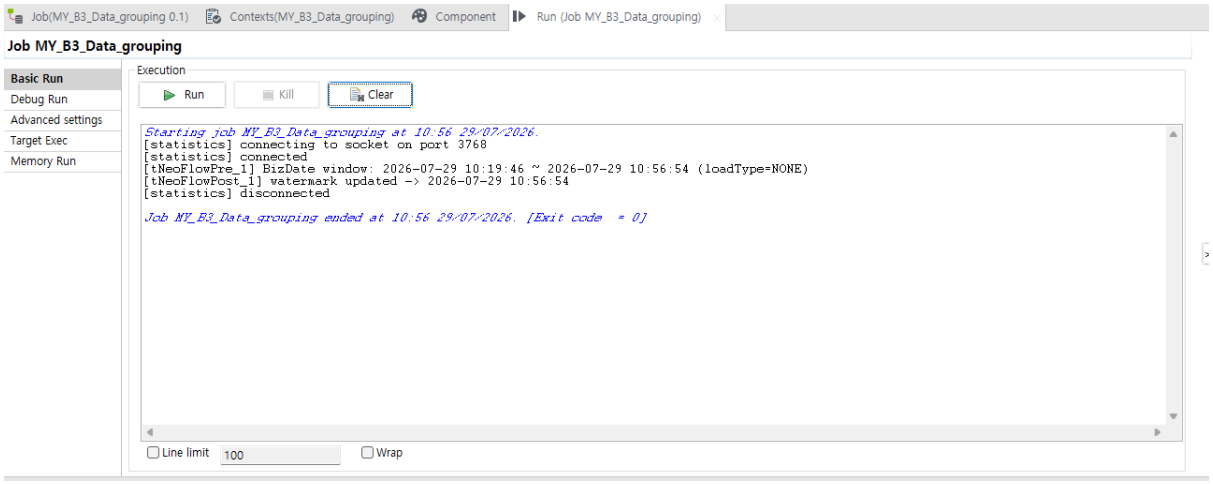


▲ *tAggregateRow — Group by 와 Operations*

```
Group by
  FIRST_NAME  ←  FIRST_NAME

Operations
  MEMBER_CNT  =  count(SEQ)
  AVG_AGE     =  avg(AGE)
```

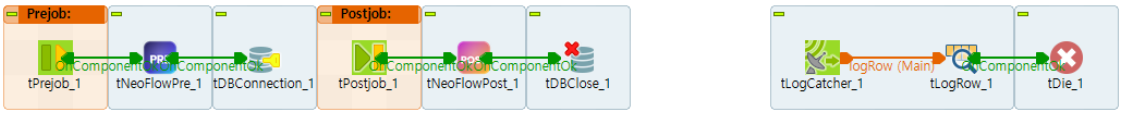
> **참고** 집계를 거치면 **행 수가 원본보다 크게 줄어듭니다**(고유 그룹 키 개수만큼). 집계 후 컬럼 구성도 바뀌므로, 뒤에 붙는 컴포넌트의 스키마를 반드시 다시 확인하세요.

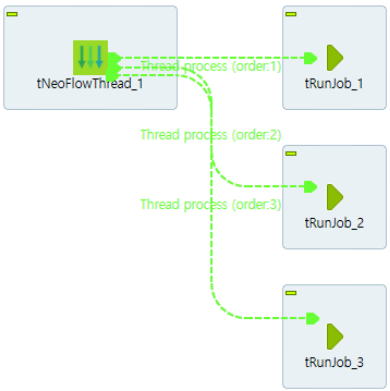


◢ *실습 7 — Run 콘솔 실행 결과*

19.9 실습 8 — 자식 Job 병렬 호출

항목	내용
목표	만들어 둔 Job 들을 하나의 Job 에서 동시에 실행한다.
사용 컴포넌트	tRunJob, tNeoFlowThread
예제 Job	MY_C1_parallel_processing

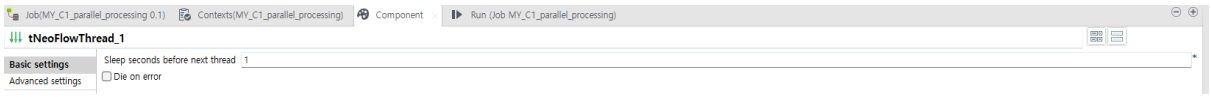




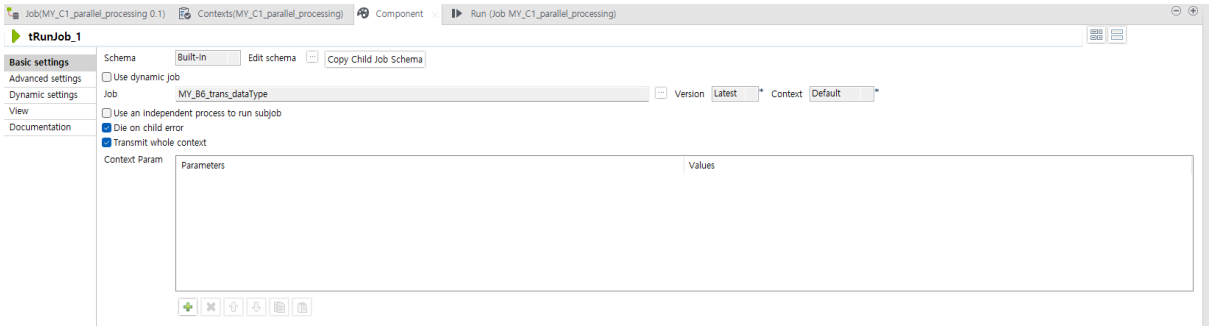
◢ *실습 8 — 자식 Job 3 개를 병렬 기동*

1. 새 Job 을 만들고 팔레트의 `NeoFlow` 패밀리에서 `tNeoFlowThread` 를 배치합니다.
2. `Sleep seconds before next thread` 를 1 로 설정합니다. 스레드를 1 초 간격으로 띄워 DB 동시 접속이 몰리는 것을 완화합니다.
3. `tRunJob` 을 세 개 배치하고, `tNeoFlowThread` 에서 각각 `Thread process` 링크로 연결합니다.
4. 각 `tRunJob` 의 Job 항목에서 실행할 자식 Job 을 선택합니다. Version 은 `Latest`, Context 는 `Default` 로 둡니다.

5. 저장 후 실행하고, Run 콘솔에서 세 Job 의 시작 로그가 1 초 간격으로 나온 뒤 서로 겹쳐 진행되는지 확인합니다.

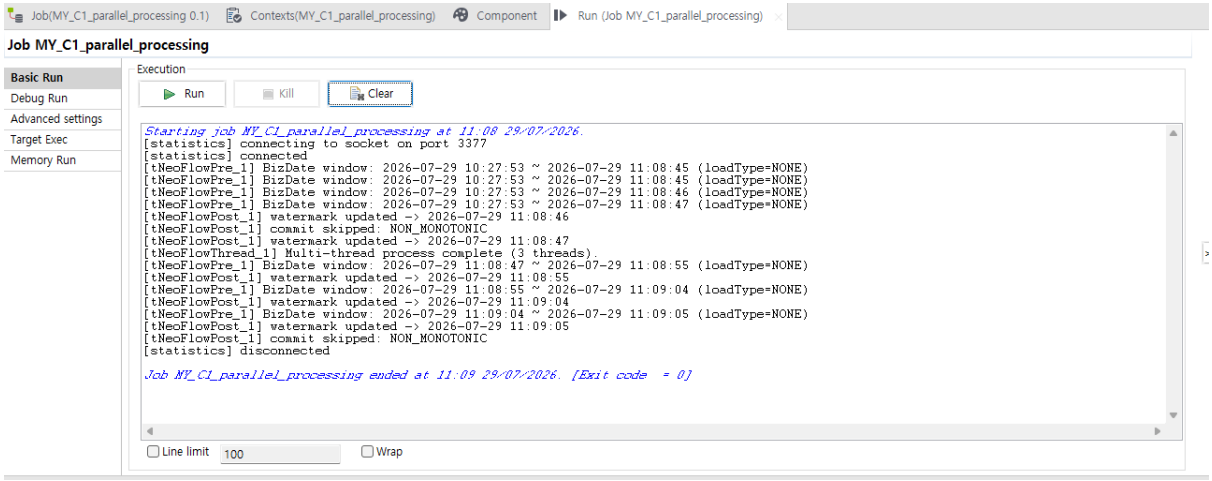


▲ *tNeoFlowThread — 스레드 기동 간격 설정*



▲ *tRunJob — 실행할 자식 Job 지정*

> **참고** 링크에 표시되는 order 는 **스레드를 띄우는 순서**일 뿐입니다. 일단 기동되면 세 Job 은 서로를 기다리지 않고 동시에 진행됩니다. 반대로 순차 실행이 필요하면 `tRunJob` 을 `OnSubjobOk` 트리거로 일렬 연결하세요.

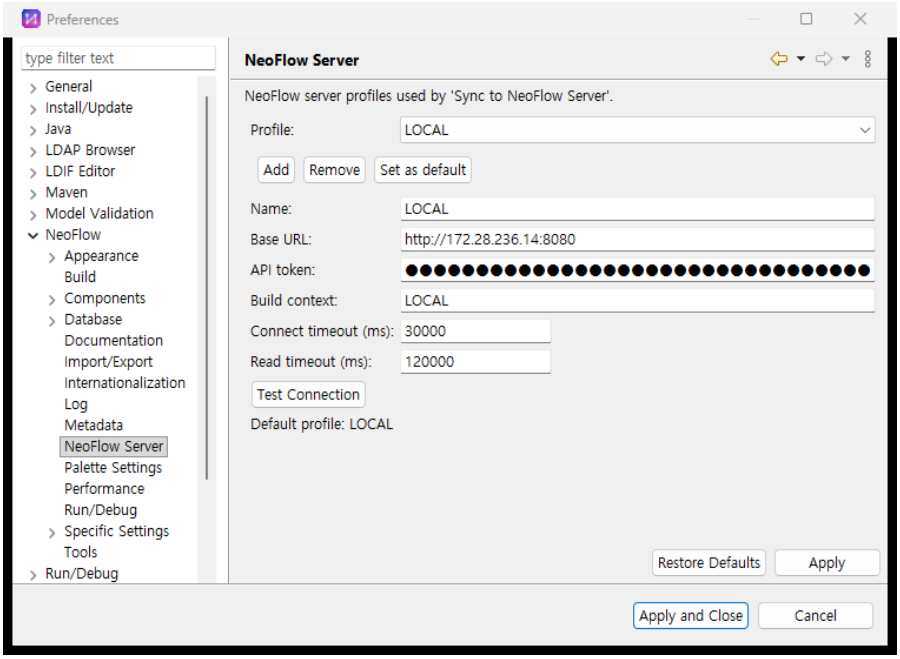


▲ *실습 8 — 병렬 실행 Run 콘솔 로그*

19.10 실습 9 — NeoFlow Server 로 동기화

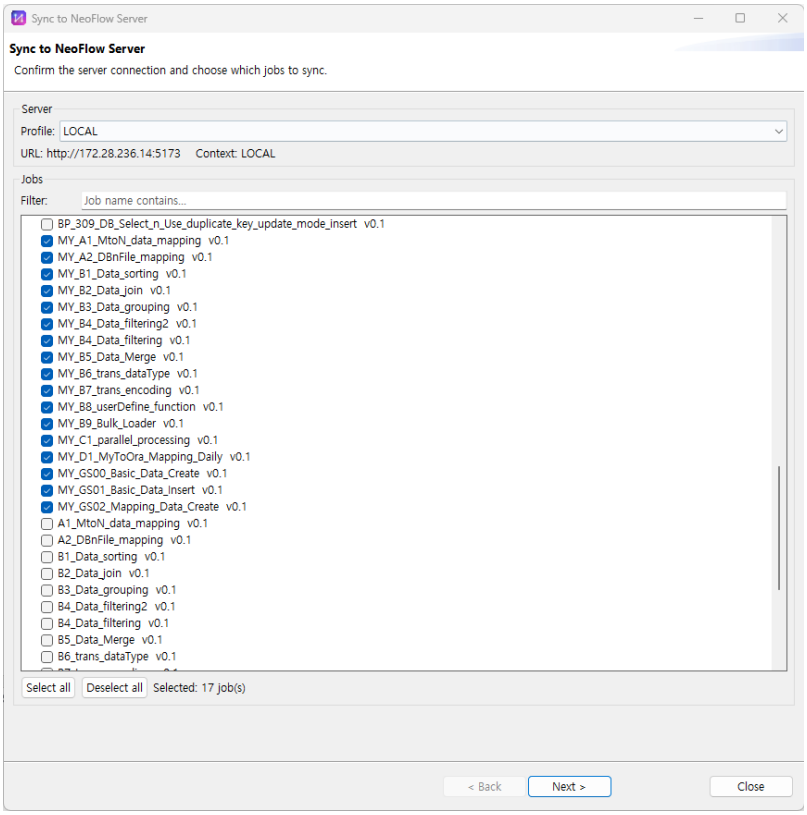
항목	내용
목표	Studio 에서 만든 Job 을 NeoFlow Server 에 올려 운영 스케줄에 편입한다.
사용 컴포넌트	Sync to NeoFlow Server

Job 이 완성되면 마지막 단계는 서버 등록입니다. Repository 에서 Job 을 우클릭해 `[Sync to NeoFlow Server...]` 를 선택하면 마법사가 열립니다. 사전에 [Window] → [Preferences] → [NeoFlow] → [NeoFlow Server] 에서 서버 연결 정보(Base URL·API token)를 등록해 두어야 합니다(14 장).

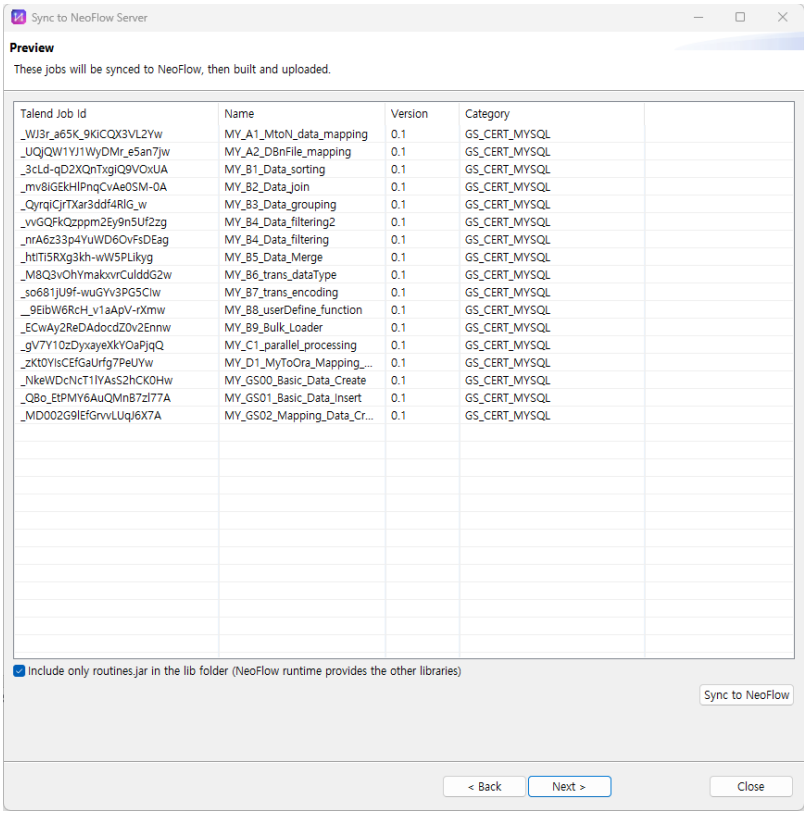


◢ *Preferences — NeoFlow Server 연결 정보 등록*

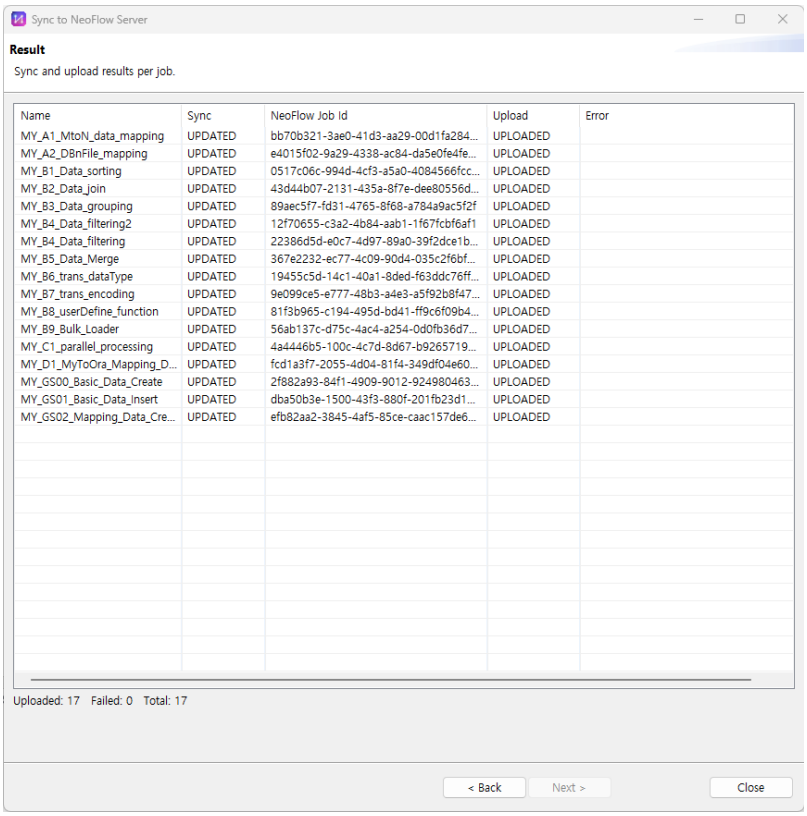
1. 1 단계 — Profile 을 확인하고 동기화할 Job 을 체크합니다.
2. 2 단계(Preview) — Job 메타데이터를 확인하고 [Sync to NeoFlow] 를 누릅니다.
3. 3 단계(Result) — Job 별 결과에서 업로드가 성공했는지 확인하고 창을 닫습니다.



◢ *Sync to NeoFlow Server — 1 단계*

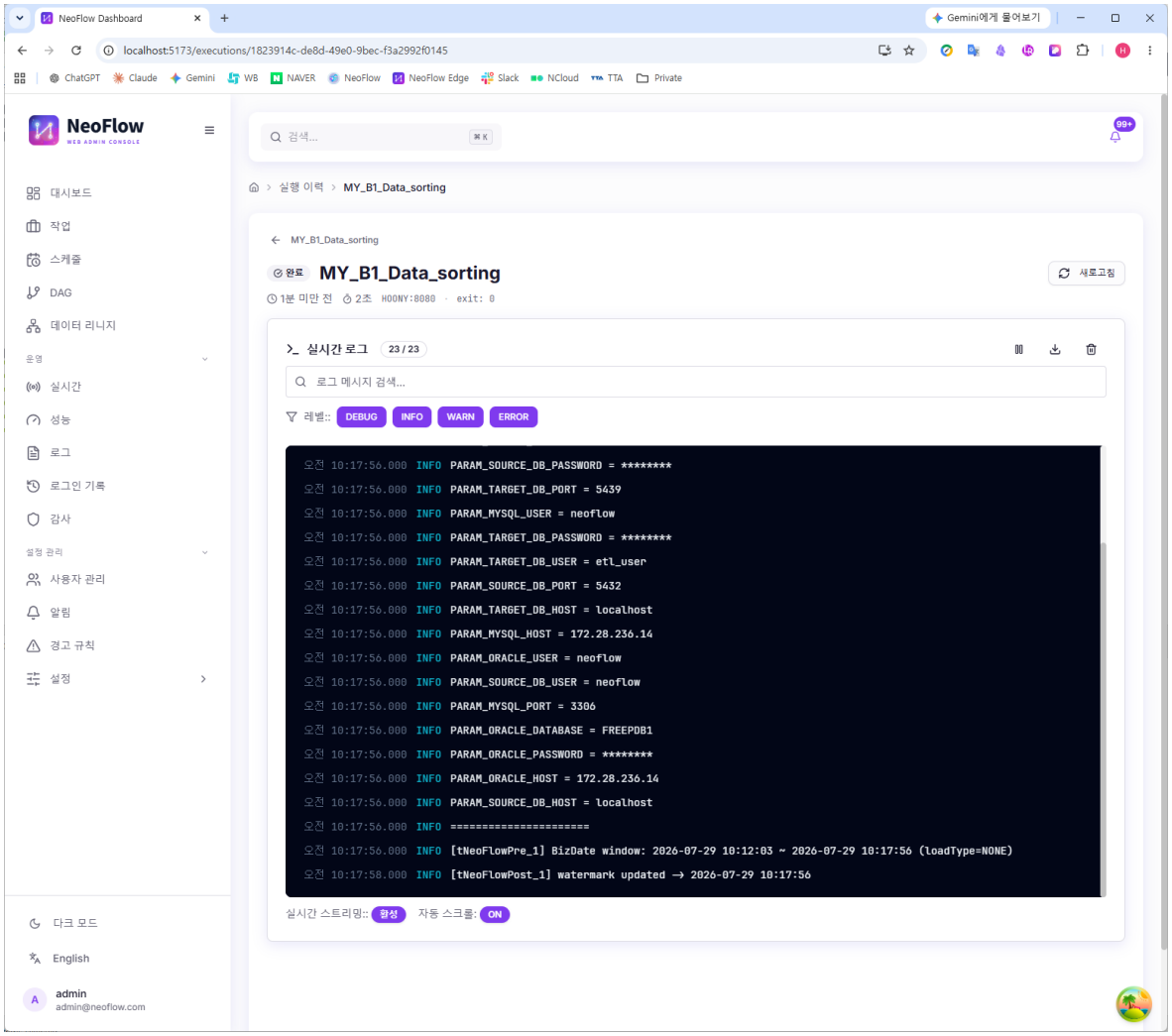


◢ *Sync to NeoFlow Server — 2 단계*



◢ *Sync to NeoFlow Server — 3 단계*

동기화가 끝나면 NeoFlow Server 웹 콘솔의 Job 목록에 나타나고, 서버에서 실행·스케줄·모니터링할 수 있습니다.



▲ NeoFlow Server — 실행 이력과 실시간 로그 (개발 환경 화면 예시)

> **참고** Sync 마법사의 각 단계 항목은 17 장, Build Job(NeoFlow Deploy) 옵션은 18 장, API token 발급 절차는 14.1 에서 자세히 다룹니다. 서버 콘솔 조작은 «NeoFlow Server 사용자 매뉴얼»을 참조하세요.

20. 부록

20.1 빠른 점검 체크리스트

Job 을 서버에 올리기 전에 다음을 순서대로 확인하세요.

- 설치 — `NEO_BD-win-x86_64.ini` 맨 위의 `-vm` 경로가 JDK 11(64bit) `javaw.exe` 를 가리키고, 창 제목에 `NeoFlow Studio for Big Data` 와 버전(1.0.0 으로 시작)이 표시되는지 확인.
- License — [Help] → [License Registration]에서 `License key is valid.` 확인.
- API token — NeoFlow Server 관리 콘솔 [설정] → [API 토큰]에서 토큰 발급 후 값 복사(발급 직후 1회만 표시).
- Server 프로파일 — [Window] → [Preferences] → [NeoFlow] → [NeoFlow Server]에서 Base URL/API token 입력, Test Connection 성공, Set as default 지정.
- Job 설계 — BizDate 사용 시 `tPrejob` 뒤에 `tNeoFlowPre`, `tPostjob` 뒤에 `tNeoFlowPost` 를 `OnComponentOk` 로 연결. 병렬 처리 시 `tNeoFlowThread`.
- Sync — Repository 에서 Job 우클릭 → [Sync to NeoFlow Server...] → Job 선택 → Sync → Result 에서 `Uploaded` 확인.

20.2 자주 쓰는 단축키

단축키	기능	비고
`Ctrl + S`	저장	실행 전 자동 저장 설정 가능
`F6`	Job 실행	캔버스에 포커스를 둔 상태에서
`Ctrl + Space`	context·변수 자동 완성	입력란·코드 편집기에서
`Ctrl + 마우스 휠`	캔버스 확대 / 축소	
`Ctrl + A`	캔버스 전체 선택	
`Ctrl + Z` / `Ctrl + Y`	실행 취소 / 다시 실행	
`Ctrl + Shift + F`	생성 코드 서식 정리	Code 탭
`F5`	테이블명 입력란에서 context 변수 생성	일부 입력란
`Delete`	선택한 컴포넌트·연결 삭제	
우클릭 → `Deactivate`	컴포넌트·연결 비활성화	삭제하지 않고 실행에서 제외
우클릭 → `Sync columns`	바뀐 스키마를 뒤 컴포넌트에 다시 전파	

> **TIP** 캔버스를 클릭한 뒤 컴포넌트 이름을 그냥 타이핑하면 검색 상자가 열립니다. 팔레트를 스크롤하는 것보다 훨씬 빠릅니다.

20.3 자주 쓰는 globalMap 변수

컴포넌트는 실행 결과(처리 건수·현재 값·에러 등)를 globalMap 에 <컴포넌트 ID>_<키> 형태로 남깁니다. <컴포넌트 ID> 는 캔버스의 고유 이름입니다(예: tFileInputDelimited_1).

globalMap 키 (대표 예)	의미
<comp>_NB_LINE	읽은/쓴 행 수 (input/output 공통)
tDBOutput_1_NB_LINE_INSERTED	삽입된 행 수 (UPDATED/DELETED/REJECTED 동형)
tFileList_1_CURRENT_FILEPATH	tFileList 현재 반복 파일 전체 경로
tFileList_1_NB_FILE	tFileList 현재 처리 순번
<comp>_ERROR_MESSAGE	Die on error 해제 시 컴포넌트 에러 메시지
tRunJob_1_CHILD_RETURN_CODE	tRunJob 자식 Job 종료 코드
tLoop_1_CURRENT_ITERATION	tLoop 현재 반복 인덱스

```
// globalMap 값 꺼내기 — 반드시 형 변환
int rows = ((Integer) globalMap.get("tFileInputDelimited_1_NB_LINE"));
String f = ((String)  globalMap.get("tFileList_1_CURRENT_FILEPATH"));
```

20.4 컴포넌트 빠른 참조

하고 싶은 것	컴포넌트
CSV/텍스트 읽기·쓰기	tFileInputDelimited / tFileOutputDelimited
DB 조회·적재	tDBInput / tDBOutput
매핑·조인·변환	tMap
조건 분기	tFilterRow / tMap 출력 필터
집계·정렬·중복제거	tAggregateRow / tSortRow / tUniqRow
다른 Job 호출	tRunJob
행마다 반복 / 파일마다 반복	tFlowToIterate / tFileList
로그 출력 / 중단 / 에러 수집	tLogRow / tDie / tLogCatcher
테스트 데이터 / 임의 코드	tFixedFlowInput / tJava·tJavaRow
업무기준일(BizDate) 구간 받기 / 커밋	tNeoFlowPre / tNeoFlowPost
서브잡 병렬 실행(시차 기동)	tNeoFlowThread

20.5 자주 묻는 질문

Q. Studio 가 실행되지 않고 바로 닫힙니다.

A. JDK 경로 문제일 가능성이 큽니다. `NEO_BD-win-x86_64.ini` 의 `-vm` 항목이 실제 `javaw.exe` 경로를 가리키는지 확인하세요. JDK 11(64bit)이 설치되어 있어야 하며, 32bit JRE 로는 실행되지 않습니다.

Q. 캔버스에 빨간 X 표시가 있고 실행되지 않습니다.

A. Configuration 영역의 `Problems` 탭을 확인하세요. 스키마 미정의, 필수 입력 항목 누락, 연결이 끊긴 컴포넌트가 대부분의 원인입니다. 문제 항목을 더블 클릭하면 해당 컴포넌트로 이동합니다.

Q. 컴포넌트에 느낌표가 뜨면서 «Schema is not synchronized» 경고가 납니다.

A. 앞 컴포넌트의 스키마를 바꿨는데 뒤 컴포넌트가 예전 구조를 그대로 갖고 있는 상태입니다. 해당 컴포넌트를 우클릭해 `[Sync columns]` 를 선택하면 맞춰집니다.

Q. DB 연결 테스트에서 «driver not found» 오류가 납니다.

A. JDBC 드라이버(jar)가 없는 경우입니다. 연결 마법사에서 안내하는 모듈을 내려받거나, Configuration 영역 `Modules` 탭 우측 상단 버튼으로 직접 jar 를 추가하세요. 일부 상용 DB 드라이버는 라이선스 때문에 기본 포함되지 않습니다.

Q. Job 은 성공했는데 데이터가 DB 에 없습니다.

A. 커밋이 되지 않았을 가능성이 큽니다. `tDBConnection` 으로 연결을 공유하는 구조라면 Job 끝에 `tDBCommit`(또는 `tDBOutput` 의 Commit every 설정)이 필요합니다. `tDBClose` 만 두면 커밋 없이 연결이 닫힐 수 있습니다.

Q. Job 이름을 한글로 지었더니 빌드가 실패합니다.

A. Job 이름은 Java 클래스명으로 변환되므로 영문·숫자·언더스코어만 사용할 수 있습니다. 한글 설명이 필요하면 Purpose·Description 항목에 쓰세요. 압축 해제 경로에 한글·공백이 있어도 같은 문제가 생깁니다.

Q. Job 을 열었더니 잠금(locked) 상태라고 나옵니다.

A. 같은 Job 을 다른 곳에서 열고 있거나, 이전에 Studio 가 비정상 종료되어 잠금이 남은 경우입니다. Studio 를 정상 종료하면 해제되며, `Read job` 으로는 언제든 열람할 수 있습니다.

Q. 화면 배치가 엉망이 되었습니다.

A. `[Window] → [Perspective] → [Reset Perspective...]` 를 실행하면 기본 배치로 돌아갑니다. 실수로 닫은 뷰는 `[Window] → [Show view...]` 에서 다시 열 수 있습니다.

Q. Sync/Build Job 메뉴를 눌렀더니 에러 다이얼로그만 뜹니다.

A. License 가 없거나 만료/불일치 상태입니다. 3 장을 참고해 라이선스를 다시 등록하세요.

Q. Test Connection 이 실패하거나 Sync 에서 인증 오류가 납니다.

A. Test Connection 창에 표시된 문구로 원인을 구분합니다. `Invalid or expired API token` 은 토큰이 만료·폐기됐거나 잘못 붙여넣은 경우이고(토큰을 발급한 계정이 비활성이어도 같습니다), `Token lacks the ADMIN/JOB_MANAGER role` 은 토큰을 발급한 계정이 ADMIN·매니저가 아닌 경우입니다. `Cannot reach server:` 뒤에 `PKIX path building failed` 가 나오면 서버 인증서를 Studio 가 쓰는 JDK 의 `cacerts` 에 등록하고 Studio 를 다시 시작하세요(14.5). 토큰 문제라면 NeoFlow Server 관리 콘솔 [설정] → [API 토큰]에서 상태와 만료일을 확인하고, 필요하면 새 토큰을 발급해 프로파일에 다시 입력하세요(14.1).

Q. 발급받은 토큰을 복사하지 못하고 모달을 닫았습니다.

A. 평문 토큰은 다시 볼 수 없습니다. 목록에서 해당 토큰을 [토큰 폐기]로 폐기하고 새로 발급하세요.

Q. tNeoFlowPre 를 넣었는데 nf_startStr/nf_endStr 가 비어 있습니다(null).

A. 서버 바인딩이 없는 상태입니다. Studio 단독 테스트라면 Sync 를 한 번 수행해 `context.NEOFLOW_*`가 채워졌는지 확인하세요. 연결한 서버가 운영(prod) 프로파일이면 로컬 테스트 실행에서 개발 토큰이 거부되어 값이 채워지지 않습니다(14.6). `tNeoFlowPre` 는 never-fail 이라 바인딩이 없으면 값을 채우지 않고 통과합니다.

Q. Studio 에서 Job 을 실행하니 「studio context unavailable」 로그가 남고 DB 연결 단계에서 실패합니다.

A. 연결한 NeoFlow Server 가 운영(prod) 프로파일이면 로컬 테스트 실행용 개발 토큰 발급이 403 으로 거부됩니다. 그러면 `tNeoFlowPre` 가 구간을 받지 못하고, 접속 정보를 `NeoFlowJobApi.getParameter()` 로 받는 컴포넌트도 값을 조회하지 못해 실패합니다. 운영 서버에서는 Sync·활성화한 뒤 웹 콘솔에서 실행하고, 로컬 테스트는 개발·검증 서버 프로파일로 하세요(14.6).

Q. tNeoFlowPost 가 커밋을 건너뜁니다.

A. 같은 Job 에 `tNeoFlowPre` 가 먼저 실행되어 `nf_endDate` 를 채웠는지 확인하세요. 배치가 어긋나 `tNeoFlowPost` 가 먼저 실행되는 경우도 원인이 됩니다 — `tNeoFlowPre` 는 `tPrejob` 뒤, `tNeoFlowPost` 는 `tPostjob` 뒤에 두어야 합니다(15 장 배치 규칙). 커밋은 됐는데 건너뛴 경우에는 `SKIP_REASON` 을 확인하세요. `NON_MONOTONIC` 은 이번 종료 시각이 이미 커밋된 워터마크보다 늦지 않은 경우(같은 Job 의 실행이 겹쳐 다른 실행이 먼저 커밋했거나, 워터마크를 수동으로 앞당겨 둔 경우 등), `BACKFILL` 은 소급 적재 실행이라 워터마크를 옮기지 않은 경우입니다.

Q. NeoFlowContext routine 을 따로 등록해야 하나요?

A. 아닙니다. `NeoFlowContext` 는 시스템 routine 으로 Studio 에 포함되어 있습니다([Code] → [Routines] → [system]). 같은 이름의 사용자 routine 은 만들지 마세요(16.2).

Q. Studio 를 실행하면 Java 를 찾을 수 없다는 오류 창이 뜨고 시작되지 않습니다.

A. `NEO_BD-win-x86_64.ini` 둘째 줄의 JDK 경로가 실제 설치 위치와 다릅니다. JDK 11(64bit)의 `bin\javaw.exe` 전체 경로로 고친 뒤 다시 실행하세요(2.3).

Q. tNeoFlowThread 에서 일부 스레드가 실패해도 Job 이 성공으로 끝납니다.

A. `Die on error` 가 해제되어 있기 때문입니다. 실패 시 Job 을 중단하려면 체크하세요.

20.6 용어 정리

용어	뜻
Job	ETL 처리 한 단위. Studio 에서 만들고 실행하는 대상
Subjob (서브잡)	Row 연결로 이어진 컴포넌트 묶음. 캔버스에서 회색 배경으로 구분됨
Component (컴포넌트)	Job 을 구성하는 기능 블록. 이름이 `t` 로 시작
Connection (연결)	컴포넌트를 잇는 선. Row·Iterate·Trigger 세 종류
Schema (스키마)	흐름을 통과하는 데이터의 컬럼 구조
Context	환경별로 달라지는 값(접속 정보·경로·날짜)을 담는 변수
Routine (루틴)	Job 에서 호출하는 사용자 정의 Java 함수
Repository	프로젝트 자원을 담는 좌측 트리
Palette (팔레트)	사용 가능한 컴포넌트 목록(우측)
Workspace (워크스페이스)	프로젝트가 저장되는 폴더
Build	Job 을 Studio 없이 실행할 수 있는 아카이브로 만드는 작업

용어	뜻
Sync	Studio 의 Job 을 NeoFlow Server 에 등록·갱신하는 작업
서버 프로파일	Studio 가 연결할 NeoFlow Server 의 Base URL·API token 묶음
API token	Studio 가 NeoFlow Server 에 인증할 때 쓰는 토큰. 서버 관리 콘솔에서 발급
BizDate (업무기준일)	Job 이 처리할 데이터 구간(시작·종료 일시). `tNeoFlowPre` 가 서버에서 받음
워터마크	처리를 마친 BizDate 구간의 종료 시각. `tNeoFlowPost` 가 서버에 커밋

20.7 참고 문서

- NeoFlow Server 사용자 매뉴얼 — 서버 웹 콘솔에서의 Job 등록·스케줄·실행 이력·알림 운영.
- Talend Studio User Guide 8.0 — Working with components (공식): help.talend.com / help.qlik.com 의 studio-user-guide 8.0.
- Talend Open Studio Components Reference Guide — 컴포넌트별 전체 파라미터·시나리오.

> **참고** 특정 컴포넌트의 모든 파라미터·예외 동작이 필요하면, Studio 에서 해당 컴포넌트를 선택한 뒤 Component 뷰의 Documentation 탭과 위 공식 Reference Guide 를 함께 확인하세요.